

Programmazione Distribuita



Programmazione Distribuita

Capitolo 1 – Introduzione

1.1 I sistemi distribuiti

Un **sistema distribuito** consiste di un insieme di macchine, ognuna gestita in maniera autonoma, connesse attraverso una rete. Ogni **nodo del sistema distribuito** esegue un insieme di componenti che comunicano e coordinano il proprio lavoro attraverso uno strato di software detto **middleware**, in maniera che l'utente percepisce il sistema come un'unica entità integrata.

Diverse **leggi empiriche** elaborate negli anni hanno predetto la velocità di evoluzione dei vari sistemi. Ad esempio, la **legge di Moore** afferma che la densità dei transistor nei processori si raddoppia ogni 18 mesi. I sistemi distribuiti rappresentano, quindi, la strada per poter utilizzare al meglio quello che la tecnologia hardware continua a produrre, ma spesso si scontrano con varie difficoltà di progettazione, sviluppo, gestione ed evoluzione. Tra i vari problemi abbiamo l'elevato costo per produrre un software complesso e problemi dovuti alla difficoltà nella realizzazione di sistemi eterogenei e complessi. La priorità più importante per gli ambienti di sviluppo per sistemi distribuiti è la riusabilità e le integrabilità di soluzioni diverse.

1.2 Le leggi che regolano le Reti

Lo **sviluppo di una rete** dipende dall'utilità che questa porta ad un insieme di utenti: quanto più sono utilizzabili i servizi, tanto maggiore è il loro valore commerciale. Il valore di una rete con riferimento al numero di connessioni è definito dalle **leggi empiriche di Sarnoff, Metcalfe e Reed**.

- **Sarnoff's law**. Il valore o utilità di una rete broadcast è direttamente proporzionale al numero di utenti:
 $V = a \cdot N$ (**lineare**).
- **Metcalfe's law**. Il valore di una rete è direttamente proporzionale al quadrato del numero degli utenti:
 $V = a \cdot N + b \cdot N^2$ (**quadratico**). In questo caso posso comunicare con ogni altro nodo della rete.
- **Reed's law**. Il valore di una rete sociale è direttamente proporzionale ad una funzione esponenziale in N : $V = a \cdot N + b \cdot N^2 + c \cdot 2^N$ (**esponenziale**). Quindi il valore di una rete cresce in modo esponenziale se è associata a gruppi con interessi comuni che condividono idee, interessi e obiettivi.

1.3 Caratteristiche di un Sistema Distribuito

I sistemi distribuiti hanno varie caratteristiche tra cui:

- Un sistema distribuito è **remoto** perché le sue componenti devono poter essere locali o remote, quindi potenzialmente localizzate su macchine diverse.
- Un sistema distribuito è per sua natura **concorrente**, cioè abbiamo l'esecuzione contemporanea di due o più istruzioni su macchine diverse, e non esistono strumenti come *lock* e *semafori* che permettono di gestire in maniera più semplice la sincronizzazione.
- **Assenza di uno stato globale**. Non esiste una maniera per poter determinare lo stato globale del sistema, in quanto la distanza e la eterogeneità del sistema non permette di definire con certezza lo stato in cui si trova ciascun nodo.
- **Malfunzionamenti parziali**. Ogni componente del sistema distribuito può smettere di funzionare e questo non deve interferire con le funzionalità del sistema distribuito.

- **Eterogeneità.** Un sistema distribuito per sua stessa definizione è eterogeneo per tecnologia sia hardware che software.
- **Autonomia.** Un sistema distribuito non ha un singolo punto dal quale può essere controllato, coordinato e gestito.
- **Evoluzione.** I sistemi distribuiti devono essere flessibili nel migrare verso ambienti diversi senza costi eccessivi.
- **Mobilità.**

1.4 Un modello di riferimento: RM-ODP

Allo scopo di facilitare lo sviluppo dei sistemi distribuiti, è importante l'utilizzo di un **modello comune** che serva come astrazione per produttori, progettisti o sviluppatori. Infatti, *ISO/IEC* ha proposto un modello formale per le architetture dei sistemi distribuiti: questa specifica si chiama "*The Reference Model of Open Distributed Processing*" (**RM-ODP**).

L'obiettivo di questo modello è di favorire la diffusione dei servizi di un sistema distribuito in un ambiente eterogeneo. Il modello *RM-ODP* si basa ed integra il modello *ISO/OSI*, infatti, questo modello gestisce i problemi di **comunicazione** rispetto ai problemi di **connessione** che vengono trattati principalmente dal modello *ISO/OSI*.

RM-ODP estende ed ingloba, il modello *ISO/OSI*, usando quest'ultimo come modalità per la comunicazione tra componenti eterogenee.

1.5 Requisiti non funzionali di un sistema distribuito

La realizzazione di un sistema distribuito non è facile e comporta la necessità di considerare vari aspetti che non hanno a che fare direttamente con le funzionalità del sistema distribuito stesso ma hanno invece a che fare con i cosiddetti **requisiti non funzionali**, che indicano la qualità del sistema.

Questi *requisiti non funzionali* specificano che la progettazione deve puntare realizzare sistemi distribuiti che:

- ... siano **aperti**, in modo da supportare la portabilità per evitare di rimanere legati ad un singolo fornitore.
- ... siano **integrati** così da incorporare al proprio interno sistemi e risorse differenti senza dover utilizzare strumenti *ad-hoc*.
- ... siano **flessibili** per poter evolvere e far evolvere i sistemi distribuiti in maniera da integrare sistemi *legacy* al proprio interno. Un sistema distribuito dovrebbe anche poter gestire modifiche a *run-time*.
- ... siano **modulari** in modo da permettere ad ogni componente di poter essere autonoma ma anche indipendente verso il resto del sistema.
- ... supportino la **federazione di sistemi**, in modo da unire diversi sistemi dal punto di vista amministrativo oltre che architetturale.
- ... siano **facilmente gestibili**, in modo da permettere il controllo, la gestione e la manutenzione per configurare i servizi, la qualità e le politiche di accesso.
- ... siano **scalabili** perché qualsiasi sistema distribuito accessibile da Internet può essere soggetto a picchi di carico non prevedibili.

- ... siano **sicuri** così che utenti non autorizzati non possono accedere a dati sensibili.
- ... offrano la **trasparenza** mascherando i dettagli e le differenze del sistema sottostante.

1.6 Trasparenza: un Requisito non funzionale importante

La definizione di **trasparenza** di un sistema equivale a dire che **un sistema distribuito appare come un'unica entità all'utente**. I vantaggi della trasparenza sono una maggior produttività attraverso l'astrazione del modello ed un alto riuso delle applicazioni sviluppate. Abbiamo diversi tipi di trasparenza, interconnessi su tre livelli:

1. Trasparenza (**livello di base**): di accesso, di locazione
2. Trasparenza (**livello di funzionalità**): di migrazione, di replica, di persistenza, di transazioni
3. Trasparenza (**livello di efficienza**): scalabilità, prestazioni, malfunzionamenti

Trasparenza di accesso

La **trasparenza di accesso** nasconde le differenze nella rappresentazione dei dati e nell'invocazione per permettere l'interoperabilità tra oggetti. Questo significa che gli oggetti devono essere accessibili attraverso la stessa interfaccia, sia che siano acceduti da locale sia che siano acceduti da remoto. In questo modo, un oggetto può essere spostato a run-time da un nodo ad un altro.

Trasparenza di locazione

La **trasparenza di locazione** non permette di utilizzare informazioni sulla locazione di un particolare componente nel sistema che viene identificato ed utilizzato in maniera indipendente dalla sua posizione. Questo tipo di trasparenza è fondamentale in quanto senza di esso non si potrebbero spostare componenti da un nodo ad un altro.

Trasparenza di migrazione

Il compito di questo tipo di trasparenza è quello di nascondere la possibilità che il sistema faccia migrare un oggetto da un nodo ad un altro. Questo viene utilizzato per ottimizzare le prestazioni del sistema come il bilanciamento del carico tra i nodi oppure riducendo la latenza o anche per anticipare i malfunzionamenti. La **trasparenza di immigrazione** dipende dalla **trasparenza di accesso** e dalla **trasparenza di locazione**.

Trasparenza di replica

Con questo tipo di trasparenza il sistema maschera il fatto che una singola componente viene replicata con un certo numero di copie dette **repliche** che vengono posizionate su altri nodi del sistema e che offrono esattamente lo stesso tipo di servizio della componente originale. **Anche questo tipo di trasparenza dipende da quello di accesso e di locazione.**

Le repliche vengono utilizzate per diversi scopi: per le prestazioni in modo da minimizzare la latenza oppure sono utilizzate per poter far scalare il sistema in presenza di aumento del carico di lavoro.

Trasparenza alle transazioni

La **trasparenza alle transazioni** nasconde all'utente le attività che vengono svolte per assicurare la consistenza dello stato degli oggetti in presenza della concorrenza.

Trasparenza ai malfunzionamenti

La **trasparenza ai malfunzionamenti** nasconde ad un oggetto il malfunzionamento di oggetti con i quali sta interoperando. **Questo tipo di trasparenza si poggia sulla trasparenza di replica, in quanto quest'ultima**

fornisce la possibilità di poter ripetere le operazioni che si erano iniziate su una replica di un oggetto, potendo rieseguirle su un'altra replica. Ma si basa anche sulla trasparenza alle transazioni, in quanto operazioni complesse se interrotte a causa di un malfunzionamento non vengono confermate e quindi non alterano lo stato della risorsa e possono essere ripetute su un'altra replica.

Trasparenza alla persistenza

Questo tipo di trasparenza nasconde all'utente le operazioni che compie il sistema per rendere persistente un oggetto durante una fase di non utilizzo. Questo tipo di trasparenza si basa sulla trasparenza di locazione in quanto l'accesso indipendente dalla posizione fisica dell'oggetto permette una riattivazione dell'oggetto anche su nodi diversi da quelli su cui era stato de-attivato.

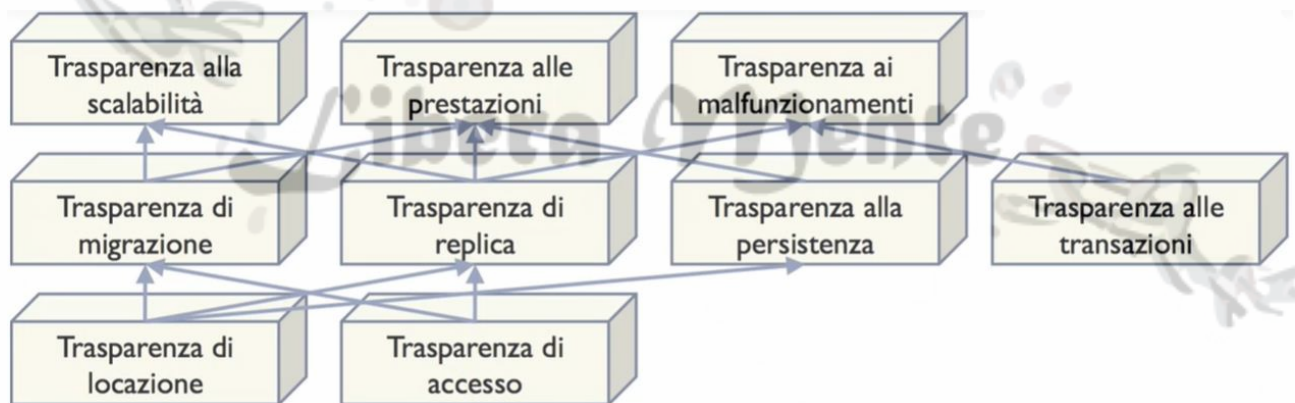
Trasparenza alla scalabilità

La scalabilità è uno dei principali motivi a favore di un sistema distribuito rispetto ad uno centralizzato. Un sistema viene detto scalabile quando in grado di poter servire carichi di lavoro via via crescenti senza dover modificare la propria architettura e proprio organizzazione. Progettare un sistema scalabile è necessario visto il gran numero di utenti.

Trasparenza alle prestazioni

Questo tipo di trasparenza rende il progettista/sviluppatore ignaro dei meccanismi che vengono utilizzati per ottimizzare le prestazioni del sistema durante la fornitura di servizi. In particolare, il sistema può provvedere ad implementare politiche di bilanciamento del carico spostando componenti da nodi più saturi a nodi meno saturi.

La trasparenza alle prestazioni si appoggia sulla trasparenza alla migrazione, alla replica e alla persistenza.

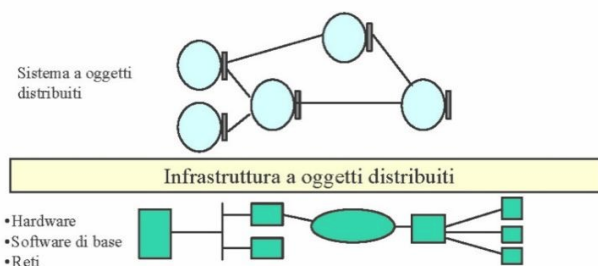


1.7 Middleware ad Oggetti Distribuiti

I sistemi distribuiti basati su oggetti distribuiti sono uno degli strumenti utilizzati nei sistemi distribuiti per assicurare estendibilità, affidabilità e scalabilità, rendendo minimo lo sforzo per progettare, sviluppare e mantenere sistemi complessi. Gli oggetti distribuiti separano due aree della tecnologia software:

- i **sistemi distribuiti** che puntano a realizzare un unico sistema integrato basato sulle risorse offerte da diversi calcolatori messi in rete;
- lo **sviluppo e la programmazione orientata agli oggetti** che si focalizzano sulle modalità per ridurre la complessità dei sistemi.

Gli *oggetti distribuiti* hanno come obiettivo quello di realizzare servizi distribuiti riutilizzabili, efficienti, flessibili, sicuri e robusti sia per hardware che per software.



Questa integrazione viene realizzata attraverso il **middleware ad oggetti distribuiti**, un software che risiede tra le **applicazioni** e lo **strato del sistema operativo, protocolli di rete e hardware**.

È chiaro che possiamo programmare un sistema distribuito utilizzando le primitive di comunicazione di ogni singolo nodo, questo però risulta essere complesso e costoso; quindi, **una soluzione è il middleware il quale fornisce le astrazioni appropriate per i programmatori**. Il **middleware ad oggetti distribuiti** può essere suddiviso in 3 strati:

1. **Middleware d'infrastruttura**. Si occupa della comunicazione tra OS diversi e della gestione della concorrenza in maniera da essere portabile (un esempio è la Java Virtual Machine).
2. **Middleware di distribuzione**: automatizza compiti comuni per la comunicazione come:
 - il **marshalling**: invio di parametri per le invocazioni remote ad altri nodi;
 - multiplexing dello stesso canale di comunicazione per più invocazioni;
 - gestione della semantica delle invocazioni (unicast, multicast, attivazione on-demand);
 - riconoscimento gestione dei malfunzionamenti;
3. **Middleware per i servizi comuni**: serve a fornire servizi comuni a tutte le applicazioni distribuite e quindi riutilizzabili in tutti i contesti.

Quindi, l'astrazione fornita dal **middleware** permette di focalizzarsi sullo sviluppo dell'applicazione, favorisce il riuso delle soluzioni adottate e rende sviluppo efficace ed efficiente.

1.8 Il progenitore: RPC

Un meccanismo di base molto importante per consentire il dialogo di applicazioni presenti su macchine diverse è il meccanismo **Remote Procedure Call (RPC)** che permette ad una procedura in esecuzione su una macchina di invocarne un'altra che si trovasse su un'altra macchina (**invocazione di procedure remote come se fossero locali**). L'obiettivo era di facilitare il compito del programmatore il quale può concentrarsi sulla suddivisione delle funzionalità in procedure senza curarsi del fatto che siano remote o locali.

Infatti, **RPC** è la prima tecnologia a fornire la traduzione dei tipi di dato a livello applicazione in maniera tale che:

- fosse possibile trasmettere i dati utilizzando stream di byte su socket;
- si superassero le differenze nella rappresentazione di interi e stringhe.

Inoltre, **RPC** imponeva che fosse rispettata la **sincronia** dell'invocazione di funzioni (metodi) bloccando il client fino a quando il server non avesse risposto all'invocazione remota.

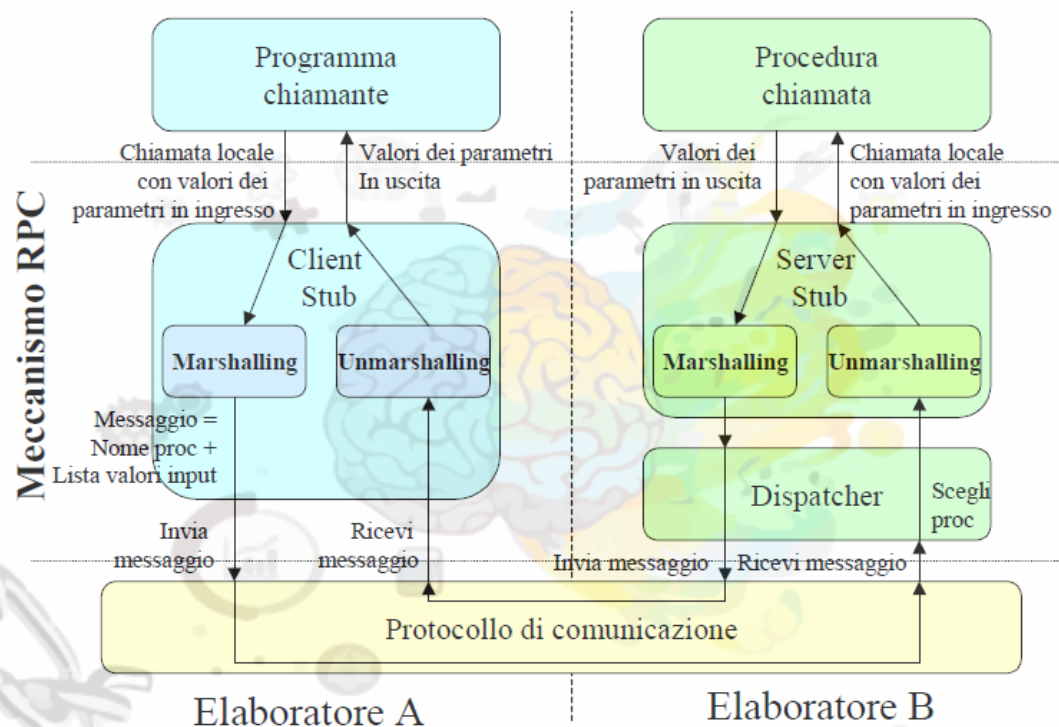
Le invocazioni **asincrone**, invece, si hanno quando la funzione (processo, thread, ...) chiamante continua la computazione, concorrentemente alla computazione della funzione chiamata.

Le operazioni di sincronizzazione, marshaling, data representation e comunicazione tra client e server venivano implementate dai sistemi di **RPC** attraverso i **client stub** e **server stub**. La scrittura degli **stub** venne poi automatizzata, utilizzando un linguaggio chiamato **Interface Definition Language (IDL)**.

Il passaggio da *RPC* agli *Oggetti Distribuiti* è stato motivato dall'evoluzione dei sistemi distribuiti divenuti più complessi, eterogenei e difficili da gestire e da mantenere a causa delle migliorie hardware e di comunicazione. Innanzitutto, *RPC*:

- usava un paradigma procedurale e non ad oggetti;
- i tipi di dato sono solamente elementari;
- mancanza della gestione delle eccezioni.

Con il passaggio al paradigma ad oggetti abbiamo nuovi meccanismi come il *polimorfismo*, *l'ereditarietà*, la *gestione delle eccezioni* che non offriva il paradigma procedurale.



1.9 Esempi di Middleware: CORBA, Java RMI, .NET

Tre sono le famiglie basate su oggetti distribuiti: **CORBA**, **Java RMI**, **.NET Remoting**.

CORBA

CORBA è uno standard che permette ad oggetti distribuiti, scritti in diversi linguaggi e quindi eterogenei, di comunicare e collaborare per realizzare un'applicazione distribuita. L'architettura di **CORBA** si basa sull'invocazione di un servizio (metodo). Le implementazioni degli oggetti remoti possono essere in C, C++, Java, ed altri linguaggi per i quali esiste un **binding** con **CORBA**.

Le invocazioni remote vengono realizzati attraverso **Object Request Broker (ORB)** chi si occupa di fornire diversi tipi di trasparenza a d'invocazione ma anche di fornire accesso a servizi di supporto.

Enterprise JavaBeans (con Java RMI)

Ha come obiettivo limitare la complessità per realizzare nuovi servizi basati su oggetti facilitandone il riuso. Abbiamo una netta separazione del layer di presentazione da quello di business in modo da bilanciare il carico attraverso le trasparenze di migrazione, locazione, accesso, etc.

Enterprise JavaBeans è **un'architettura a componenti**: una componente è un modulo software che può essere ricombinato anche sotto forma di binario a differenza di librerie, oggetti tradizionali etc. Il server che gestisce le componenti si chiama *container* (o application server). Inoltre, è un Middleware implicito.

.NET Remoting

.NET Framework è la soluzione di Microsoft per la realizzazione di applicazioni ed include un ambiente di esecuzione chiamato **Common Language Runtime (CLR)** in modo che le applicazioni possono essere scritte in uno dei linguaggi supportati da Microsoft.

.NET Remoting è una tecnologia per realizzare una comunicazione remota. Con questa tecnologia la parte di middleware di infrastruttura è assicurato dal **CLR** e la parte di middleware di servizi viene realizzata da altre librerie di **.NET**.

1.10 Componenti Distribuite

Una **componente distribuita** è un blocco riutilizzabile di software che può essere usato in un sistema distribuito per realizzare funzionalità. All'interno di una componente risiedono servizi e applicazioni che espongono le proprie funzionalità tramite un'interfaccia. A differenza dei moduli software riutilizzabili (come gli oggetti), le componenti possono essere combinati sotto forma di eseguibili binari, piuttosto che sotto forma di azioni da compiere sul codice sorgente e, inoltre, il modello a componenti si basa sul cosiddetto **middleware implicito** contrapposto al **middleware esplicito**. Un esempio di middleware esplicito è *Java RMI*.

Il **middleware implicito** è in grado di fornire servizi comuni ad ogni componente senza che essa debba esplicitamente richiederli all'interno del codice. Il server che gestisce la componente (detta *application server o container*) fornisce questi servizi sulla base delle richieste codificate non nel codice ma in un file di metadati di descrizione durante la fase di deployment, cioè quando la componente viene messa a disposizione sul server.

In questa maniera i servizi vengono messi a disposizione in maniera trasparente allo sviluppatore di software della componente, realizzando una maggiore interoperabilità tra produttori di software diversi.

Il **middleware esplicito** i servizi venivano esplicitamente richiesti dal progettista di applicazioni inserendo all'interno del codice le chiamate ai servizi forniti dal sistema. Questo presentava alcuni problemi:

- Complessità;
- mancanza di controllo del sistema su errori;
- mancanza di portabilità.

Capitolo 2 – Programmazione Concorrente

2.1 Thread in Java

Un **thread** è un **percorso di controllo** all'interno di un processo. Un **processo**, invece, è un programma in esecuzione con un unico percorso di controllo e con uno spazio di memoria privato.

Molti sistemi operativi moderni permettono che un processo possa avere più percorsi di controllo, ovvero possedere dei **multi-thread**.

Un processo tradizionale è chiamato **processo pesante** (*heavyweight process*) in riferimento allo spazio di indirizzamento, mentre, i *thread* sono anche detti **processi leggeri** perché hanno un contesto più semplice rispetto ai processi. I *thread* sono **shared memory**: condividono la memoria per comunicare.

Uno dei vantaggi principali della programmazione *multithread* è dato dal fatto che i *thread* appartenenti allo stesso processo condividono tra loro memoria e file aperti. In particolari circostanze, ogni *thread* può necessitare di una copia privata di certi dati, chiamata **dati specifici dei thread**.

Nei **sistemi multicore** i *thread* possono essere eseguiti in parallelo, poiché ogni *thread* può essere assegnato ad un core specifico.

2.2 Multi-Threading

Il **multi-threading** è una forma di parallelizzazione che consente l'elaborazione simultanea. Invece di assegnare un carico di lavoro elevato a un singolo core, i programmi threadizzati dividono il lavoro in più thread software. Questi thread vengono quindi elaborati in parallelo da diversi core della CPU per risparmiare tempo.

La tecnologia **Hyper-Threading** è un'innovazione hardware che consente l'esecuzione di più thread su ciascun core. Più thread attivi significano più lavoro svolto in parallelo.

Quando la tecnologia *Hyper-Threading* è attiva, la CPU genera due contesti di esecuzione per ogni core fisico. Questo significa che un singolo core fisico funziona come due "core logici", in grado di gestire diversi thread software.

I problemi con i Multi-Core riguardano l'accesso alla memoria, infatti, si potranno avere:

- **cache misses** (breve);
- **page faults** (lungo);
- **context-switch dallo scheduler** (molto lungo).

2.3 Dennard Scaling e il Power Wall

Il **Dennard Scaling** afferma che, man mano che i transistor si riducono, la loro densità di potenza rimane costante, in modo che l'utilizzo di energia rimanga proporzionato all'area.

Sfortunatamente, *Dennard Scaling* ignora i **leakege current** (perdite di energia) e il **threshold voltage**, che stabiliscono un limite di potenza per transistor. Questo limite definisce il **power wall** in cui la frequenza di un processore è stata limitata intorno ai 4 GHz fino al 2006.

Quindi, con l'era del *Dennard Scaling* si puntava ad aumentare sempre più la frequenza dei processori trascurando i core.

Solo successivamente (dopo il 2005) si è passati ad aumentare il numero di core mantenendo costante le frequenze di clock della CPU, infatti, tramite l'uso di più core le performance aumentarono grazie all'uso del **parallelismo**.

2.4 Programmazione Concorrente

La programmazione distribuita implica la conoscenza della **programmazione concorrente** che coinvolge diversi processi che vengono eseguiti insieme. Abbiamo tre tipi di programmazione concorrente:

1. programmazione concorrente eseguita su calcolatori diversi;
2. processi concorrenti sulla stessa macchina (multitasking);
3. processo padre che genera processi figli per `fork()`;

Quando parliamo di programmazione concorrente nello stesso processo ci riferiamo ai **thread**.

2.5 Multitask vs Multithread

- ❖ Il **multitask** crea l'illusione (per l'utente) di una macchina completamente dedicata ma durante l'interazione dell'utente con il proprio programma, il SO ha il tempo di servire altri utenti.
- ❖ Il **multithread**, invece, è l'estensione del multitask riferita ad un singolo programma in grado di eseguire più thread "contemporaneamente".

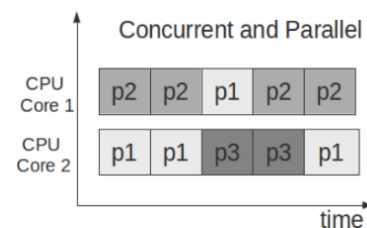
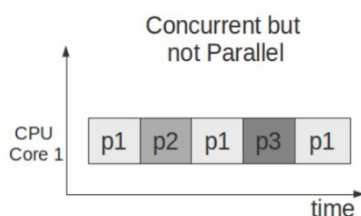
2.6 Parallelismo e Concorrenza

La **concorrenza** è una tecnica utilizzata per ridurre il tempo di risposta del sistema utilizzando una singola CPU. Un'attività è divisa in più parti e ogni parte viene elaborata contemporaneamente ma non nello stesso istante. Produce l'illusione del parallelismo, ma in realtà i pezzi di un'attività non vengono elaborati parallelamente.

La concorrenza conferisce l'accesso a più parti delle risorse condivise e funziona su un thread quando sta facendo progressi utili; quindi, arresta il thread e passa a un thread diverso quando non sta facendo progressi utili.

Il **parallelismo** è concepito allo scopo di aumentare la velocità di calcolo utilizzando più processori. È una tecnica per eseguire simultaneamente i diversi compiti nello stesso istante. Coinvolge diverse CPU che operano ed eseguono parallelamente attività al fine di aumentare la velocità di calcolo e migliorare la produttività.

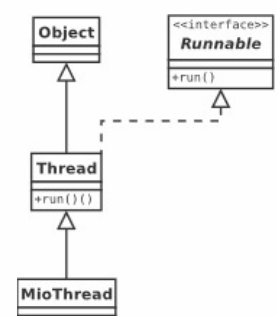
Il parallelismo determina la sovrapposizione delle attività di CPU e I/O in un processo con le attività di CPU e I/O di un altro processo.



2.7 Thread in Java

I thread in Java sono **oggetti**, istanze quindi della classe `Thread`. L'evoluzione di Java ha portato a due modalità di gestione dei thread:

1. istanziare un oggetto `Thread` ogni volta che serve un task asincrono (creazione e gestione a cura del programmatore);
2. astrarre la gestione, passando un task ad un `Executor`.



Per scrivere i thread in Java bisogna:

1. estendere la classe `java.lang.Thread`;
2. riscrivere (override) il metodo `run()` nella sottoclasse di `Thread`;
3. creare un'istanza di questa classe derivata;
4. richiamare il metodo `start()` su questa istanza.

```
public class HelloThread extends Thread {
    public void run() {
        System.out.println("Hello from a thread!");
    }
    public static void main(String args[]) {
        (new HelloThread()).start();
    }
}
```

1) Esempio

```
public class HelloRunnable implements Runnable{
    public void run() {
        System.out.println("Hello from a
        thread!");
    }

    public static void main(String args[]) {
        (new Thread(new HelloRunnable())).start();
    }
}
```

2) Esempio

2.7.1 Metodi della classe `Thread`

- **`sleep()`**: cessa temporaneamente l'esecuzione del thread che lo esegue per un certo numero di millisecondi. È un metodo statico ciò vuol dire che può essere invocato sulla classe `Thread` e non sugli oggetti.
- **`join()`**: consente a un thread di attendere il completamento di un altro thread. Se `t` è un oggetto `Thread` il cui thread è attualmente in esecuzione, allora `t.join()` farà in modo che `t` venga terminato prima che l'istruzione successiva venga eseguita dal programma. È possibile anche specificare un periodo di attesa in millisecondi come parametro. `join()` risponde ad un interrupt generando `InterruptedException`.
- **`start()`**: quando il programma chiama questo metodo, viene creato un nuovo thread e il codice all'interno del metodo `run()` viene eseguito nel nuovo thread.
- **`run()`**: se si chiama il metodo `run()` direttamente non verrà creato alcun nuovo thread e il codice all'interno di `run()` verrà eseguito direttamente nel thread corrente.

Un'altra differenza tra `start()` e `run()` è che non è possibile chiamare `start()` due volte: una volta avviata, la seconda `start()` chiamata verrà lanciata `IllegalStateException` in mentre è possibile chiamare il metodo `run()` più volte poiché è solo un metodo normale.

2.8 Gli Interrupt

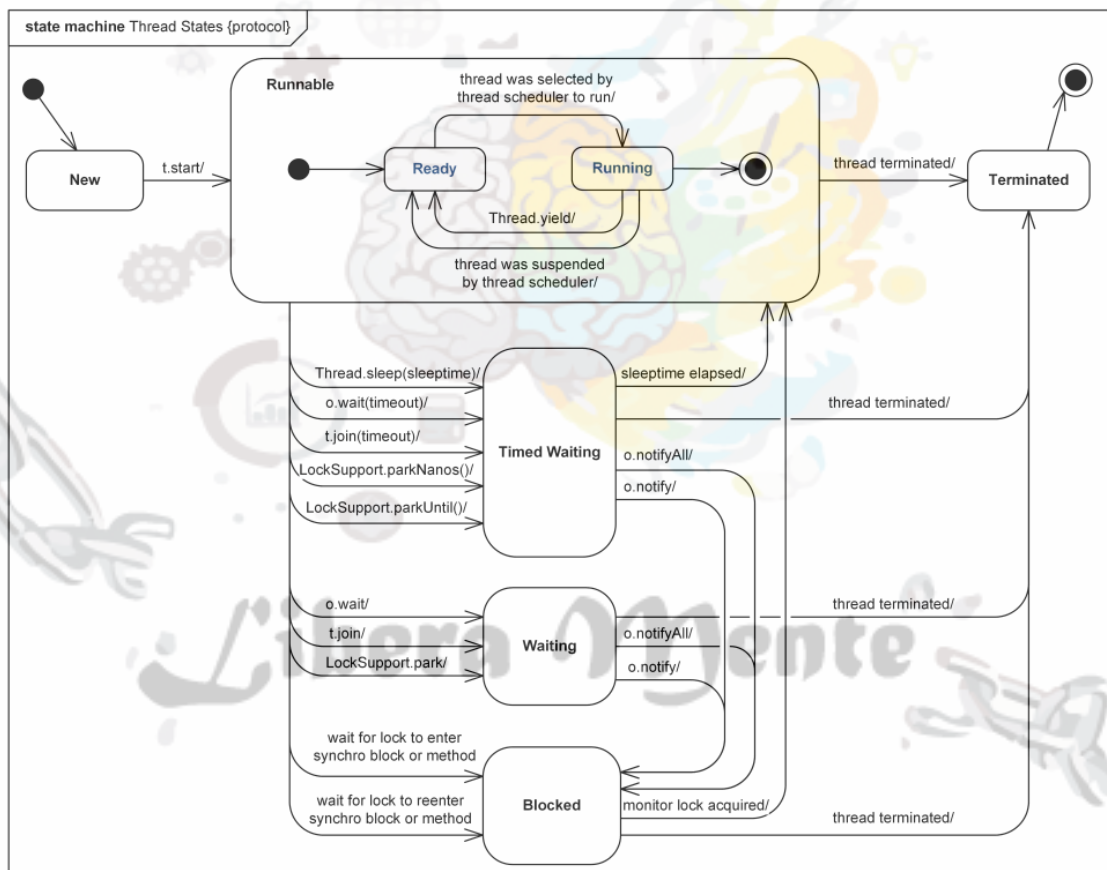
Un **interrupt** indica al thread di fermare quello che sta facendo e fare qualcosa altro ed è il programmatore a decidere cosa fare. Se un thread non invoca un metodo che lancia l'eccezione `InterruptedException`, allora il programmatore può controllare se è stato interrotto utilizzando **`Thread.interrupted()`**.

2.9 Gli Stati dei Thread (nella JVM)

Un thread può trovarsi in uno dei seguenti stati:

- **NEW**: un thread che non è ancora stato avviato.
- **RUNNABLE**: un thread in esecuzione nella macchina virtuale Java.
- **BLOCKED**: un thread bloccato in attesa di un blocco del monitor.
- **WAITING**: un thread che attende indefinitamente che un altro thread esegua una determinata azione.
- **TIMED_WAITING**: un thread in attesa che un altro thread esegua un'azione fino a un tempo di attesa specificato.
- **TERMINATED**: un thread che è stato terminato.

Un thread può trovarsi in un solo stato in un determinato momento. Questi stati sono stati della macchina virtuale che non riguardano gli stati dei thread del sistema operativo.



2.10 Comunicazione tra Thread

I thread comunicano principalmente condividendo e accedendo a:

- **campi** (tipi primitivi);
- **campi che contengono riferimenti a oggetti.**

I tipi di errori con la programmazione concorrente sono principalmente due:

1. **Interferenza**: si verifica quando due o più thread accedono concorrentemente provocando una **race condition**. La **race condition** si verifica quando il risultato di un'operazione dipende dall'ordine di esecuzione di diversi thread. Gli errori dovuti ad una *race condition* sono tipicamente transienti e difficili da riprodurre (debugging difficile). Inoltre, l'uso del debugger può alterare l'ordine di esecuzione dei task. Questi bug vengono anche detti *Heisenbug* che richiamano il principio di indeterminazione di Heisenberg in fisica quantistica ("Non è possibile misurare simultaneamente la posizione ed il momento di una particella").

2. **Inconsistenza della memoria**: si verifica quando thread diversi hanno visioni diverse dei dati. Le cause possono essere: protocolli di coerenza di cache, ottimizzazioni hardware/software, etc. La soluzione è l'**happens-before** in cui la memoria scritta da un thread è visibile da un altro thread.

Esempio:

Il campo contatore è condiviso tra due thread, A e B

- supponiamo che A incrementi il contatore: `counter++`;
- supponiamo che subito dopo B esegue la stampa: `System.out.println(counter)`;
- ... può capitare che la modifica di A non sia visibile a B (che stampa 0).
- bisogna stabilire una relazione **happens-before**.

Per stabilire una relazione *happens-before* usiamo la **sincronizzazione**, in particolare useremo i metodi `start()` e `join()` per introdurre una relazione *happens-before*.

- `Thread.start()`: gli effetti del codice sono visibili al nuovo thread;
- `Thread.join()`: metterà il thread corrente in attesa fino a quando il thread su cui è chiamato non termina. Se il thread viene interrotto, la `join` genererà un'`InterruptedException`.

Esempio:

Se `t` è un oggetto `Thread` il cui thread è attualmente in esecuzione, allora `t.join()` farà in modo che `t` venga terminato prima che l'istruzione successiva venga eseguita dal programma.

Un'altra maniera è rendere la variabile **volatile** assicurando la relazione *happens-before* per ogni successiva lettura della variabile (da parte di qualsiasi thread).

La **volatile** è usata quando il valore di una variabile viene scritto in memoria da un thread e tutti gli altri thread leggeranno tale valore. In questo modo non vengono utilizzati i meccanismi di caching e vengono disabilitate alcune ottimizzazioni di riordino delle istruzioni del compilatore.

Infatti, dichiarando **volatile** una variabile si costringe il thread ad aggiornare di volta in volta il valore senza memorizzarlo in cache.

Per risolvere questi problemi è necessaria la **sincronizzazione** che a sua volta genera problemi di contesa: cioè quando più thread cercano di accedere alla stessa risorsa simultaneamente (**deadlock** e **livelock**).

2.11 Legge di Amdahl: Speedup

La **legge di Amdahl** viene usata per predire l'aumento massimo teorico di velocità che si ottiene usando più processori. Chiameremo **speedup S** di un programma X, il rapporto tra il tempo impiegato da un processore per eseguire X rispetto al tempo impiegato da n processori per eseguire X.

Sia p la parte del programma X che è possibile **parallelizzare** con n processori, la parte parallela ha tempo $\frac{p}{n}$ mentre la parte sequenziale ha tempo $(1 - p)$.

Quindi, lo **speedup** che si ottiene eseguendo il programma X su n processori, dove p è la parte di X che si può parallelizzare è:
$$S(n) = \frac{1}{(1 - p) + \frac{p}{n}}$$

La **legge di Amdahl** ci dice che la parte sequenziale del programma rallenta molto qualsiasi **speedup** che possiamo pensare di ottenere. Quindi, per velocizzare un programma non basta investire sull'hardware ma è assolutamente necessario impegnarsi a rendere la parte parallela predominante rispetto alla parte sequenziale.

2.12 Sincronizzazione: metodo `synchronized()`

I **metodi sincronizzati** (`synchronized`) sono un costrutto del linguaggio Java, che permette di risolvere gli errori di concorrenza al costo di inefficienza. Per rendere un metodo sincronizzato, basta aggiungere `synchronized` alla sua dichiarazione:

```
public synchronized void increment () {  
    c++;  
}
```

Non è possibile che due esecuzioni dello stesso metodo sullo stesso oggetto siano **interfogliate** (sovrapposte). Quando un thread esegue un metodo sincronizzato per un oggetto, gli altri thread che invocano metodi sincronizzati dello stesso oggetto sono sospesi fino a quando il primo thread non ha finito. Quando un thread esce da un metodo sincronizzato, allora si stabilisce una relazione **happens-before** cioè i cambi allo stato, effettuati dal thread appena uscito sono visibili a tutti i thread.

I costruttori non possono essere sincronizzati solo il thread che crea dovrebbe avere accesso all'oggetto in costruzione.

`synchronized(this)`, il thread è sincronizzato sull'oggetto corrente, quindi, si può avere un'istanza per thread. Ciò è utile per impedire a più thread di aggiornare un oggetto contemporaneamente, il che potrebbe creare uno stato incoerente. Lo stesso vale su un metodo statico sincronizzato.

`synchronized(SomeClass.class)` è sincronizzato sulla classe dell'oggetto corrente in modo che solo un thread alla volta possa accedere a qualunque istanza di quella classe. Questo potrebbe essere utilizzato per proteggere i dati condivisi tra tutte le istanze di una classe dall'entrare in uno stato incoerente.

2.13 Lock intrinseci

Un **lock intrinseco** o **monitor lock** è una entità associata ad ogni oggetto che garantisce sia **accesso esclusivo** sia **accesso consistente** (relazione *happens-before*).

Un thread deve:

- acquisire il lock di un oggetto;
- rilasciarlo quando ha terminato.

Quando il lock che possedeva viene rilasciato, viene stabilita la relazione *happens-before*.

Quando un thread esegue un metodo sincronizzato di un oggetto ne acquisisce il lock, e lo rilascia al termine (anche se c'è una eccezione).

2.14 Sincronizzazione con Azioni Atomiche

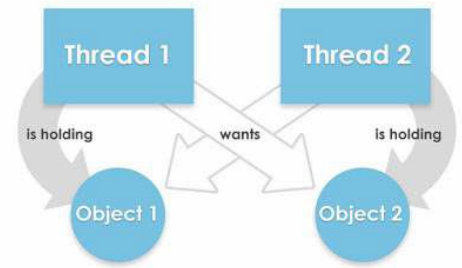
Sono azioni che non sono interrompibili e si completano (del tutto) oppure per niente. Si possono specificare **azioni atomiche** in Java per:

- `read` e `write` su variabili di riferimento e su tipi primitivi (a parte `long` e `double`);
- `read` e `write` su tutte le variabili `volatile`.

Write a variabili `volatile` stabiliscono una relazione *happens-before* con le letture successive. Per usare le azioni atomiche bisogna importare `java.util.concurrent.atomic`.

2.15 Problemi di Sincronizzazione: Deadlock, Starvation, Livelock

- Un **deadlock** si verifica quando due thread sono bloccati, ognuno in attesa dell'altro.



- La **starvation** si verifica quando un thread non è in grado di ottenere un accesso alle risorse condivise e non è in grado di fare progressi. Ciò accade quando le risorse condivise non vengono rese disponibili per lunghi periodi da thread "avid". Ad esempio, supponiamo che un oggetto ha un metodo sincronizzato che impiega molto tempo per essere restituito. Se un thread richiama frequentemente questo metodo, anche altri thread che richiedono un accesso sincronizzato frequente allo stesso oggetto verranno spesso bloccati.
- La **livelock** si verifica quando un thread agisce in risposta all'azione di un altro thread. Se l'azione dell'altro thread è anche una risposta all'azione di un altro thread, potrebbe verificarsi il **livelock**. Come con il **deadlock**, i thread **livelock** non sono in grado di fare ulteriori progressi. Tuttavia, i thread non sono bloccati: sono semplicemente troppo occupati a risponderci l'un l'altro per riprendere il lavoro. Questo è paragonabile a due persone che tentano di incrociarsi in un corridoio: Alphonse si sposta alla sua sinistra per far passare Gaston, mentre Gaston si sposta alla sua destra per far passare Alphonse. Vedendo che si stanno ancora bloccando a vicenda, Alphonse si sposta alla sua destra, mentre Gaston si sposta alla sua sinistra e così via.

2.16 Multithreading a grana fine e a grana grossa

Nel **multithread a grana fine**, i thread emettono istruzioni in modo round-robin mentre nel **multithreading a grana grossa**, i thread emettono istruzioni fino a quando non si verifica uno stallo.

Il **multithreading a grana fine** è un meccanismo di multithreading in cui il passaggio tra i thread avviene nonostante la mancanza di cache causata dall'istruzione del thread.

Il **multithreading a grana grossa** è un meccanismo di multithreading in cui lo switch avviene solo quando il thread in esecuzione provoca uno stallo, sprecando così un ciclo di clock. Tale multithreading è meno efficiente del **multithread a grana fine** perché causa un gap / perdita di ciclo quando si passa da un thread all'altro. Inoltre, la grana grossa richiede meno thread per mantenere la CPU occupata rispetto al **multithreading a grana fine**.

2.17 Modello fork-join

Nel **calcolo parallelo**, il **modello fork-join** è un modo per eseguire programmi paralleli attraverso un approccio **divide et impera**. In pratica, si suddivide l'attività in sottoattività più piccole e indipendenti fino a quando non sono abbastanza semplici da essere eseguite in modo asincrono.

Dopodiché, inizia la parte **"join"**, in cui i risultati di tutte le sottoattività vengono unite ricorsivamente in un unico risultato, o nel caso di un'attività che restituisce `void`, il programma attende semplicemente che ogni sottoattività venga eseguita.

2.18 Singleton

Il **singleton** è un noto design pattern della programmazione ad oggetti che permette di istanziare **una ed una sola istanza** di una classe. L'allocazione avviene solo quando l'oggetto è utilizzato per la prima volta (lazy allocation).

Il singleton avrà un **costruttore privato**, perché non deve essere invocato dall'esterno, e un **metodo pubblico statico `getInstance()`** il quale istanzia un solo oggetto, verificando che sia stato creato per la prima volta, e lo restituisce. Inoltre, essendo una inizializzazione statica, si stabilisce una relazione *happens-before* con tutte le altre operazioni della classe.

```
public static Singleton getInstance() {
    if (instance == null) {
        instance = new Singleton();
    }
    return instance;
}
```

Questo metodo funziona se ci troviamo in ambito **single-thread**. Nella programmazione **multi-thread** si possono verificare condizioni di *race condition* perché due thread vogliono accedere nello stesso momento al metodo `getInstance()` e verranno creati due singleton.

Una soluzione è di rendere `getInstance()` un metodo sincronizzato ma questo porta lo svantaggio di avere una grande inefficienza.

Una soluzione migliore è il **Double-checked locking**: se non esiste l'istanza si entra in una sezione critica, si controlla che nel waiting non sia cambiata la situazione e se è il caso viene creato un nuovo Singleton e si esce dalla sezione critica. Una cosa importante è che il campo `instance` deve essere **volatile** per prevenire problemi di incoerenza della cache (solo con versioni dopo la JDK5). Questo metodo però non è funzionante al 100% per la mancanza di compatibilità con le versioni precedenti di Java.

```
private static volatile Singleton instance;
public static Singleton getInstance() {
    if (instance == null) {
        synchronized (Singleton.class)
            if (instance == null) {
                instance = new Singleton();
            }
    }
    return instance;
}
```

Capitolo 3 – Socket

3.1 Programmare con i socket

Java fornisce le API per la programmazione di rete all'interno del package `java.net` dove vengono definite le classi per trattare gli indirizzi di rete che gestiscono IPv4 e IPv6.

La comunicazione tra programmi su Internet avviene utilizzando la suite di protocolli **TCP/IP**. Questi protocolli forniscono un livello di astrazione chiamata **socket** che permette di ricevere e trasmettere dati. Questa astrazione permette al programmatore di vedere una comunicazione su rete come un semplice stream di dati.

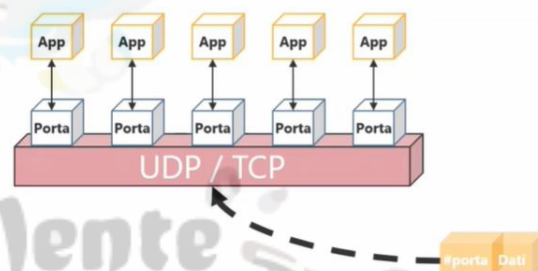
- **TCP** offre una connessione affidabile tra i due punti della comunicazione.
- **UDP** non fornisce una connessione tra due punti ma permette di inviare i pacchetti di dati (datagram) da un'applicazione ad un'altra. In questo caso non viene fornito alcun meccanismo per garantire il recapito dei pacchetti, a differenza di *TCP*, quindi **UDP è inaffidabile**.

La scelta di quale protocollo utilizzare per la comunicazione all'interno di un'applicazione distribuita dipende:

- dalla natura dell'applicazione, cioè se richiede lo scambio di stream di dati oppure di messaggi di dimensione limitata;
- dalla quantità di overhead che si vuole pagare: le comunicazioni *TCP* richiedono maggiori risorse di calcolo per offrire l'affidabilità della comunicazione;

La trasmissione avviene assegnando un *socket* ad una specifica porta che serve ad identificare quali sono i pacchetti destinati ad una certa applicazione.

- Gli **stream socket** sono *TCP*,
- I **datagram socket** sono *UDP*.



3.2 Socket TCP

A livello programmatico, un *socket* è un identificatore univoco che rappresenta un canale di comunicazione attraverso cui viene trasmessa l'informazione. La comunicazione basata su *socket* è indipendente dal linguaggio di programmazione, inoltre, *client* e *server* devono concordare sul protocollo (TCP o UDP) e sul numero di porta.

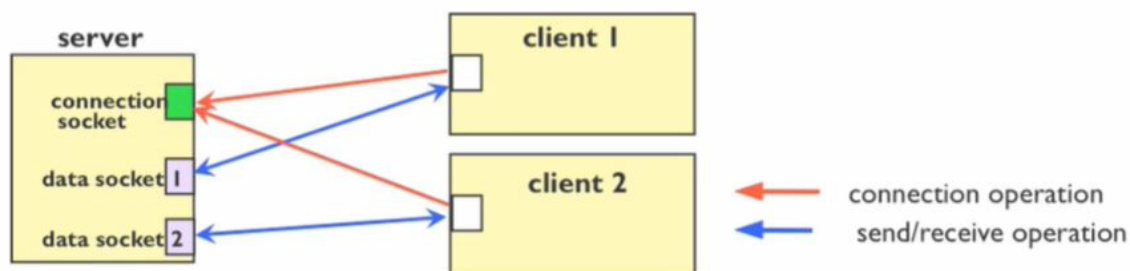
Ad ogni *socket* viene assegnato un numero di porta che serve ad identificare l'applicazione in esecuzione sul computer al fine di ricevere dati. Quindi, un *socket* è dato dalla combinazione di indirizzo IP e numero di porta.

I due computer che vogliono comunicare, tramite un socket, prendono il nome di **client** e **server**:

- il **server** è in esecuzione ed attende che qualche *client* richieda la connessione;
- dal lato **client**, il programma conosce l'indirizzo IP della macchina (su cui è in esecuzione il *server*) e il numero di porta. Inoltre, il *client* deve anche comunicare al *server* il numero di porta locale sul quale riceverà i dati.

Il **procedimento di connessione** prevede che il *server* accetti la connessione e assegni un nuovo socket per la comunicazione bidirezionale tra *client* e *server*. In questa maniera, il *server* può tornare ad accettare

connessioni da altri *client*. In questo caso, il *server* lancia un thread per ogni *socket* stabilito con il *client*, in modo da permettere la gestione concorrente delle comunicazioni con tutti i *client*.



Il package `java.net` offre due classi per i socket: la classe **ServerSocket** e la classe **Socket**. La classe **ServerSocket** implementa un socket che attende le richieste dei client; quando ne riceve una, assegna un socket alla connessione bidirezionale, restituendo l'oggetto **Socket** che viene usato per la comunicazione tra *client* e *server*.

I **metodi** di **ServerSocket** sono:

- **ServerSocket(int port)**: crea un server socket su una specifica porta;
- **Socket accept() throws IOException**: aspetta connessioni su questo socket e le accetta. È un metodo bloccante fino a quando non viene fatta una connessione. Inoltre, questo metodo crea un nuovo **Socket** e lo restituisce;
- **public void close() throws IOException**: chiude il socket;
- **void setSoTimeout(int timeout) throws SocketException**: setta un timeout (in ms) per una call ad accept e se il tempo passa senza una connessione, viene lanciata un'eccezione `java.io.InterruptedIOException`.

Costruzione di uno stream Socket

- 1) Il *server* stabilisce un server socket `sd1` con l'indirizzo locale per ascoltare le connessioni. Il *client* stabilisce una connessione con il server.
- 2) Il *server* accetta una connessione e crea un nuovo socket `sd2` per lo scambio di dati con il *client*.
- 3) Il *client* effettua una `send` di dati e il *server* effettua una `receive` per ogni `send` inviata dal *client*.
- 4) Il *server* invia la risposta e il *client* effettua una `receive`.
- 5) Quando il protocollo termina, il *server* chiude ed elimina il socket `sd2`, e anche il *client* chiude il suo socket.

3.3 Gli Stream

La comunicazione tra client e server avviene attraverso la scrittura e la lettura di **stream** (flussi) associati ad un socket. Gli stream permettono la trasmissione di oggetti Java tra client e server, attraverso il meccanismo della serializzazione.

La *serializzazione* trasforma una classe Java in un array di byte, mentre, "**serializzando un oggetto**" lo si rende persistente, ovvero l'oggetto sopravvive dopo la chiusura del programma: ciò significa salvare lo stato dell'oggetto all'interno di un file.

Con la *serializzazione*, ogni oggetto riceve un numero di serie; se lo stesso oggetto viene salvato due volte la seconda volta salviamo solo il numero di serie. I numeri di serie ripetuti sono interpretati come riferimenti allo stesso oggetto.

Per *serializzare* un oggetto si deve implementare l'interfaccia **Serializable**. Oggetti non *serializzabili* sono ad esempio i **thread** e tutte le classi di tipo **Stream**. Non vengono serializzate né le variabili statiche né le variabili d'istanza dichiarate **transient**.

Poiché la **deserializzazione** potrebbe essere fatta anche da un'altra classe che si trova su un'altra macchina, bisogna verificare la compatibilità tra le classi che serializzano un oggetto e quelle che lo deserializzano attraverso una costante chiamata **SerialVersionUID**, la quale associa alla classe un identificativo univoco di tipo **Long**.

Abbiamo due tipi di stream:

1. flussi di dati binari (**byte stream**);
2. flussi di caratteri (**character stream**).

Ciascun tipo di flusso è gestito da apposite classi inserite nel package **java.io** (`import java.io.*;`):

- per i dati binari, viene usata la classe **InputStream** e la classe **OutputStream**;
- per i caratteri, usiamo la classe **Reader** e la classe **Writer**.

La classe **IOException** viene utilizzata da molti metodi del package **java.io** per segnalare condizioni di errore legati al flusso di I/O.

Classe astratta **InputStream**

La classe astratta **InputStream** dichiara metodi per leggere flussi binari da una sorgente specifica. Tra i metodi più importanti abbiamo:

- **read()** [metodo astratto]: legge un byte alla volta e restituisce un **int** (da 0 a 255) che rappresenta il byte letto oppure -1 se il file è terminato.
- **close()**: chiude il flusso di input, rilascia le risorse associate al flusso e se vengono effettuate operazioni sul flusso chiuso viene provocata una **IOException**.

Tra le sottoclassi di **InputStream** abbiamo:

1. **FileInputStream**: realizzato per la lettura di flussi di byte da un file in filesystem. Il metodo **read()** è invocato su un oggetto di tipo **InputStream** e legge byte fino a quando l'input è a disposizione: ritorna 1 se ha letto il prossimo byte, -1 se ha letto la fine del file.
2. **FilterInputStream**: sovrascrive metodi di **InputStream** oppure ne aggiunge alcune funzionalità, eventualmente trasformando i dati lungo il percorso.
 - a. **DataInputStream**: estende **FilterInputStream** e consente di leggere da un oggetto di tipo **InputStream** i tipi primitivi di Java: **int**, **double**, **String**, etc..
 - b. **BufferedInputStream**: estende **FilterInputStream** in modo tale da rendere la lettura *bufferizzata*.
3. **BufferInputStream**: prende in ingresso un array di byte lo usa come buffer. Un contatore interno tiene traccia del prossimo byte da fornire al metodo **read()**.
4. **ObjectInputStream**: si possono leggere interi oggetti, liste, da qualsiasi stream, evitando cicli inutili in cui andare a scrivere un singolo dato primitivo alla volta. Per fare queste operazioni bisogna **serializzare** gli oggetti che devo leggere, ciò significa che questi oggetti devono estendere **Serializable**.

Classe astratta `OutputStream`

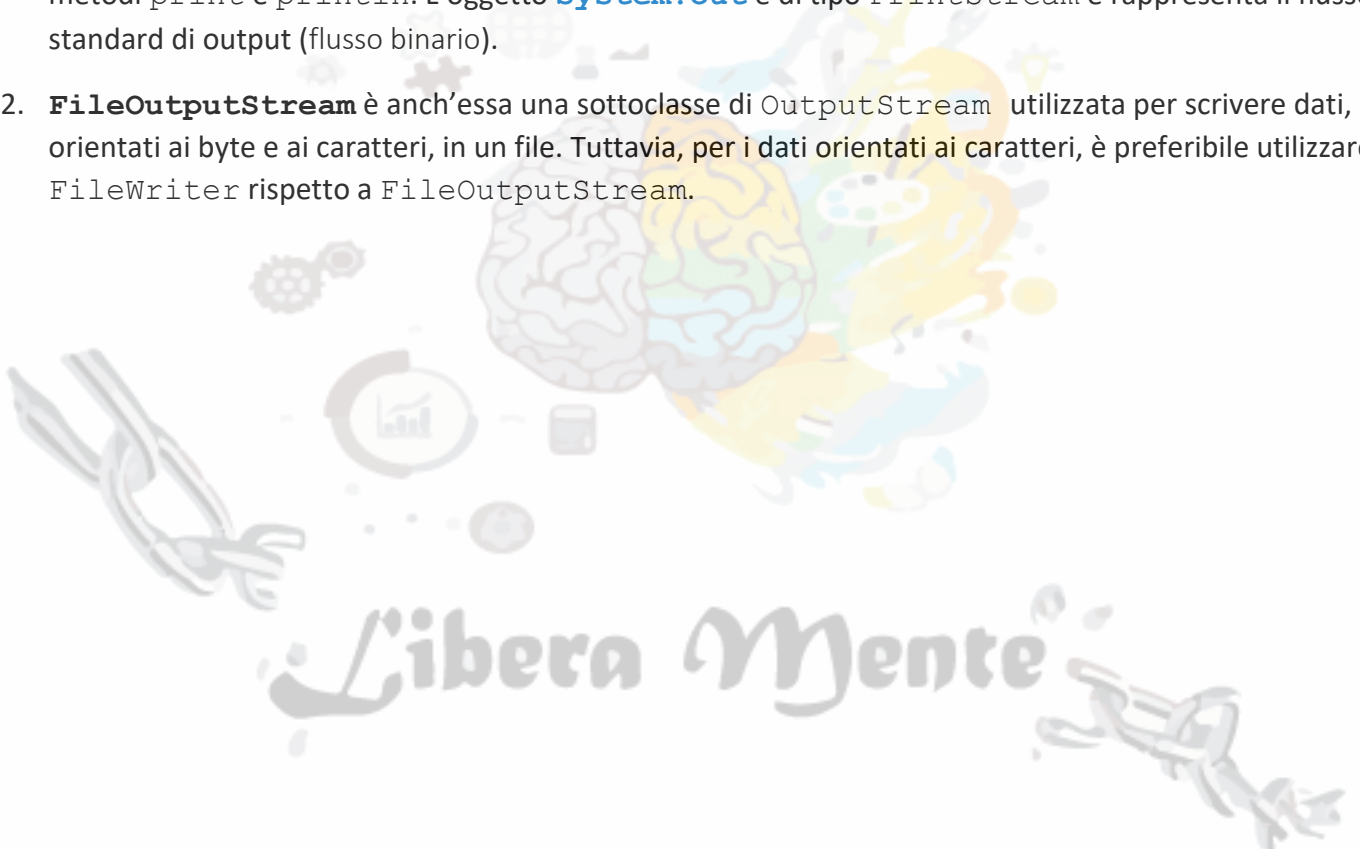
La classe astratta `OutputStream` dichiara metodi per scrivere flussi binari in una destinazione specifica.

Tra i metodi più importanti abbiamo:

- `write(int b)` [*metodo astratto*]: scrive un byte alla volta e il byte è passato come argomento di tipo `int`.
- `close()`: chiude il flusso di output, rilascia le risorse associate al flusso e se vengono effettuate operazioni sul flusso chiuso viene provocata una `IOException`.

Alcune sottoclassi di `OutputStream` sono:

1. **`PrintStream`** è una classe concreta nella discendenza ovvero è una sottoclasse di sottoclasse di `OutputStream`. Aggiunge a `OutputStream` tutti i metodi per stampare vari tipi di dati, come i metodi `print` e `println`. L'oggetto `System.out` è di tipo `PrintStream` e rappresenta il flusso standard di output (flusso binario).
2. **`FileOutputStream`** è anch'essa una sottoclasse di `OutputStream` utilizzata per scrivere dati, orientati ai byte e ai caratteri, in un file. Tuttavia, per i dati orientati ai caratteri, è preferibile utilizzare `FileWriter` rispetto a `FileOutputStream`.



Capitolo 4 – Java RMI

4.1 Introduzione Java RMI

Java Remote Method Invocation (RMI) è una libreria di Java che permette lo sviluppo di applicazioni distribuite, infatti, offre ad un oggetto in esecuzione su una *Java Virtual Machine* la possibilità di invocare metodi di un oggetto in esecuzione in un'altra *JVM*, anche se esso si trova su una macchina differente.

Il ruolo di *Java RMI* è quello di **integration library** (libreria per l'integrazione) situata al di sopra delle librerie base del linguaggio.

Le applicazioni *RMI* seguono un'architettura client-server dove il server crea un certo numero di oggetti server accessibili da remoto e attende che gli oggetti client ne utilizzano i servizi, compiendo invocazioni remote sui metodi.

4.2 Obiettivi di Java RMI

Gli obiettivi di *Java RMI* riguardano la semplicità e l'integrazione dell'implementazione favorendone la diffusione e impedendo usi scorretti del sistema. *Java RMI* è integrato all'interno del linguaggio Java. Altri obiettivi importanti sono:

➤ Invocazione trasparente di metodi remoti

Java RMI deve poter offrire al programmatore un meccanismo semplice per l'invocazione di metodi da parte di un oggetto remoto. L'invocazione trasparente di metodi remoti offre l'illusione che l'invocazione avvenga su un oggetto interno alla *JVM*.

➤ Integrazione completa in Java

Java RMI si integra in maniera naturale all'interno del linguaggio Java. Questo permette di offrire un ambiente familiare allo sviluppatore Java, che può usare gli stessi strumenti, modelli e astrazioni che vengono utilizzati per gli oggetti locali.

Java RMI fornisce un **garbage collector distribuito** integrato con quello locale in modo da gestire i **memory leak** pericolosi soprattutto per le applicazioni distribuite che per quelle locali. Un **memory-leak** è un'allocazione di memoria che non viene deallocata anche se non utilizzata. Questo è particolarmente pericoloso per i server attivi 24/24: anche il più piccolo *memory-leak* può portare in pochi giorni un server a esaurire la memoria a disposizione.

Una tecnica usata per il *garbage collection* è il **reference counting**: ogni oggetto ha un contatore con il numero di riferimenti che puntano ad esso. Quando un oggetto A usa il riferimento ad un altro oggetto B, il contatore di B viene incrementato. Periodicamente viene effettuato il calcolo dei riferimenti e vengono eliminati (restituiti all'heap) le variabili con contatore zero. Lo svantaggio si ha con riferimenti circolari.

La **garbage collection il locale** funziona mantenendo e calcolando il numero di riferimenti ad un oggetto: se un oggetto non è più riferito da nessuno allora può essere eliminato dall'heap alla prossima esecuzione del *garbage collector*.

La **garbage collection per gli oggetti remoti** è simile a quello locale, però in questo caso si introduce il meccanismo di **lease**: ogni riferimento che viene assegnato al client ha un tempo di vita specifico. Al termine del periodo, se non vengono effettuate altre invocazioni, il server considera quel riferimento non più valido e quindi verrà eliminato dal *garbage collector*. Se si vuole evitare che l'oggetto server si elimini, il programmatore deve usare dei metodi per "rinnovare" il *lease*, effettuando delle chiamate fittizie a metodi che non hanno nessun effetto.

➤ Non-trasparenza della natura locale/remota di un oggetto

Un oggetto che sia remoto o locale deve essere chiaro ed evidente, sia in fase di progettazione che in fase di implementazione.

➤ Modalità di invocazioni multiple

Java RMI deve prevedere diversi tipi di invocazione: **unicast**, **multicast**. Inoltre, deve essere possibile che l'oggetto server sia attivato solo al momento dell'invocazione e che i riferimenti ad oggetti persistenti siano persistenti.

➤ Preservare la sicurezza fornita da Java

La **sicurezza** del linguaggio Java consiste nell'eseguire un'applicazione all'interno di una **sandbox**, cioè un ambiente sicuro e controllato all'interno del quale le operazioni che il programma può eseguire non siano pericolose. Java fornisce 4 livelli di sicurezza:

1. **Sicurezza intrinseca.** Java è fortemente tipizzato (strongly typed): tutte le variabili hanno un tipo definito a tempo di compilazione e pochi casting automatici a run-time. Tutte le altre operazioni di casting devono essere esplicitate dal programmatore.
Java gestisce lo spazio di memoria mediante *garbage collection* evitando la formazione di *memory leak*. L'assenza di puntatori non permette accessi illegali in memoria, inoltre, l'accesso alla memoria reale non viene determinato a tempo di compilazione ma tempo di esecuzione, quindi, non si può conoscere in anticipo in che zona di memoria verranno memorizzati gli oggetti e non si può scrivere codice per alterarli. Infine, a tempo di esecuzione vengono controllati limiti di array per prevenire accessi a elementi non esistenti (errori di tipo `IndexOutOfBoundsException`).
2. **Class Loader.** Il suo compito è di caricare la classe a tempo di esecuzione in un namespace separato rispetto a quello delle classi locali, in modo che classi del linguaggio built-in (cioè locali) non possono essere rimpiazzate da altre, in questa maniera non c'è possibilità di **sovrascrivere** una classe di sistema. Infatti, quando si fa riferimento ad una classe, viene prima cercata tra le classi del sistema locale (built-in) e, solo successivamente, nel namespace della classe dalla quale viene riferita.
3. **Bytecode Verifier:** dopo che il *ClassLoader* ha caricato una classe per l'esecuzione, il **Bytecode Verifier** controlla che la classe sia conforme alle specifiche del linguaggio (ad esempio, correttezza degli operandi, casting illegali, etc.), che non ci siano stack underflow/overflow e che non ci siano violazioni ai modificatori di accesso. Questo livello di controllo garantisce che il *bytecode* non sia stato manomesso oppure che non sia stato generato da un compilatore Java di bassa qualità che ha tralasciato alcuni controlli.
4. **Security Manager.** Si occupa di definire i confini della *sandbox*, usando le policy definite dall'utente. Il *Security Manager* viene chiamato dalla macchina virtuale per ciascuna operazione "potenzialmente" pericolosa. In questo modo, l'utente può eseguire un'applicazione sconosciuta con una policy restrittiva. Il *Security Manager* viene caricato a tempo di esecuzione e non può essere esteso, sovrascritto o rimpiazzato.

➤ Rendere minima la complessità di client e server

➤ Livello di trasporto multipli

4.3 Il modello a oggetti distribuiti di Java RMI

Un **oggetto remoto** è un oggetto i cui metodi possono essere acceduti da un altro spazio di indirizzamento, e quindi da un'altra macchina.

I servizi offerti da un oggetto remoto sono contenuti all'interno di un'**interfaccia remota** che è un'interfaccia Java che dichiara **metodi remoti**.

Un'**invocazione di metodi remoti** (Remote Method Invocation) rappresenta l'invocazione di un metodo su un oggetto remoto (specificato nell'interfaccia remota) e ha la stessa sintassi di un'invocazione di un metodo locale.

4.3.1 Struttura delle classi di Java RMI

Java RMI è contenuto in 5 package:

- `java.rmi` e `java.rmi.server` contengono le funzionalità base delle invocazioni remote;
- `java.rmi.activation` è usato per gli oggetti attivabili;
- `java.rmi.dgc` per la garbage collection;
- `java.rmi.registry` per il servizio di localizzazione.

Interfacce ed Eccezioni remote

Prima di definire un oggetto remoto, si deve definire un'**interfaccia remota** per l'oggetto, in modo che vengano esposti i servizi che l'oggetto remoto vuole mettere a disposizione per il client.

Un'interfaccia remota per la Java RMI deve stendere l'interfaccia `java.rmi.Remote` detta **marker**, cioè un'interfaccia vuota che serve solo a indicare che essa definisce dei metodi accessibili dal remoto.

Ogni metodo descritto da un'interfaccia remota deve soddisfare 2 condizioni:

1. Un metodo remoto deve dichiarare esplicitamente l'eccezione `java.rmi.RemoteException`, che è una **checked exception** cioè un'eccezione controllata gestita dal programmatore.
2. I parametri remoti di un metodo remoto devono essere dichiarati tramite l'interfaccia remota. Questo permette di poter passare i riferimenti remoti sia come parametri che come valori restituiti.

Inoltre, il meccanismo dell'interfaccia remota aggiunge un livello in più ai modificatori di accesso dei metodi (`public`, `private`, `protected`, etc.): i metodi remoti dichiarati in un'interfaccia remota sono più accessibili dei metodi `public` che risultano essere accessibili solo dalle invocazioni all'interno della stessa macchina virtuale.

Implementazioni remote

Per realizzare l'**implementazione remota**, che deriva (`implements`) da un'interfaccia remota per offrire verso l'esterno i metodi remoti in essa definiti, si può procedere in 2 modi:

1. il primo, detto di **riuso dell'implementazione remota**, prevede che la classe che contiene l'implementazione dell'oggetto derivi esplicitamente da `java.rmi.server.UnicastRemoteObject` ereditando di conseguenza il comportamento definito dalle classi `java.rmi.server.RemoteObject` e `java.rmi.server.RemoteServer`.

2. La seconda modalità, detta **classe di implementazione locale**, permette che la classe derivi il comportamento da qualche altra classe (non remota) e che si debba occupare esplicitamente di:
 - esportare l'oggetto, tramite il metodo statico `exportObject()` di `java.rmi.server.UnicastRemoteObject`;
 - implementare la semantica di alcune operazioni di `Object` per oggetti remoti che sono ridefiniti in `java.rmi.server.RemoteObject` e `java.rmi.server.RemoteServer`.

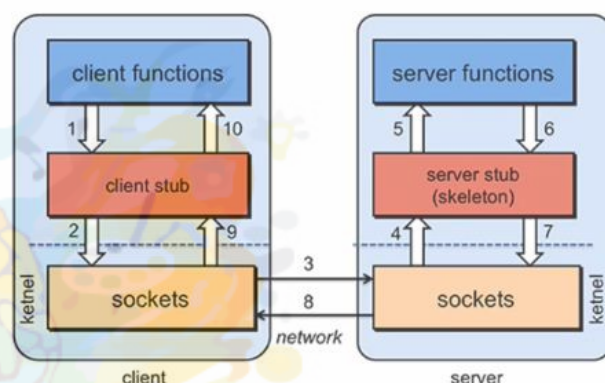
4.3.2 Meccanismo di invocazione remota

Riferimenti remoti

Lo **stub** (lato client) e lo **skeleton** (lato server)

forniscono la rappresentazione dell'oggetto remoto:

- lo **stub** è un oggetto che si trova sul client e agisce come *proxy* consentendo la comunicazione con l'oggetto remoto;
- lo **skeleton** ne consente l'esecuzione sul server.



I client interagiscono con l'oggetto stub che rappresenta l'interfaccia remota dell'oggetto remoto in locale sulla macchina virtuale del client. Inoltre, è possibile caricare dinamicamente lo stub da un server WWW o a tempo di esecuzione, quando si ottiene il riferimento remoto.

Localizzazioni e Invocazioni

Per poter invocare il metodo remoto di un oggetto remoto, l'oggetto client deve avere a disposizione il riferimento remoto, che può essere riferito in due maniere:

1. ottenuto come risultato di altre invocazioni (locali o remote) di metodi;
2. attraverso un servizio di directory.

Per quanto riguarda il secondo metodo, *Java RMI* fornisce un meccanismo di **name server** nella classe `java.rmi.Naming`, che permette di gestire i riferimenti a oggetti remoti attraverso un *ID* (stringa). Tale classe fornisce **metodi statici** per

- **ricercare** → `lookup()`;
- **registrare** → `bind()`, `unbind()`, `rebind()`;
- **elencare** → `list()`;

gli identificativi registrati attraverso un URL. Gli oggetti remoti hanno una sintassi di questo tipo:

host:port/name.

Questo meccanismo è utilizzabile anche come tool da linea di comando, **rmiregistry**, eseguito sulla macchina sulla quale l'oggetto server si trova.

L'invocazione di un metodo remoto ha la stessa sintassi di un'invocazione locale. Poiché i metodi remoti devono includere `RemoteException` nella propria firma, il codice dell'oggetto client viene forzato dal compilatore a gestire questo possibile malfunzionamento della chiamata remota.

Passaggio di parametri

Un metodo remoto può dichiarare solo parametri o valori restituiti che siano serializzabili, cioè che implementino l'interfaccia `Serializable`.

Un **oggetto locale** viene passato per copia, vale a dire che il contenuto dell'oggetto viene copiato prima di essere serializzato.

Quando si passano i riferimenti allo stesso oggetto mediante invocazioni diverse sulla macchina server saranno oggetti diversi. Il meccanismo implementato da *Java RMI* assicura l'**integrità referenziale**: quando vengono passati più riferimenti allo stesso oggetto nella stessa invocazione, allora viene garantito che punteranno allo stesso oggetto sulla macchina server.

Passaggio in metodi diversi

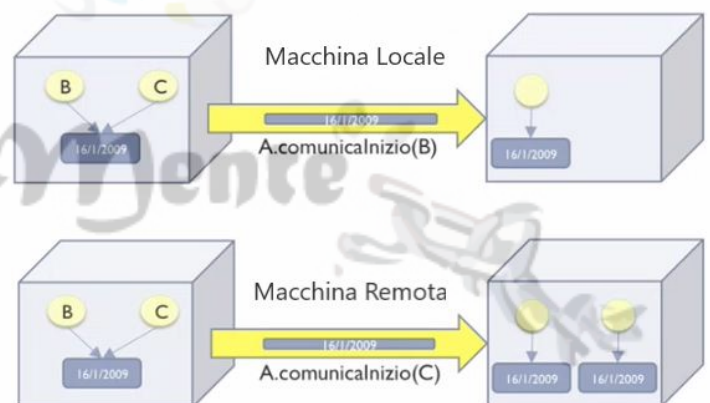
Quando vengono passati come parametri più riferimenti allo stesso oggetto su un'altra macchina virtuale utilizzando invocazioni diverse, allora i riferimenti sulla macchina server saranno ad oggetti distinti.

Esempio:

Se **A** è un **riferimento ad un oggetto remoto** che offre il metodo `comunicaInizio(Date x)`, e se **B** e **C** sono **riferimenti ad oggetti locali** di tipo `Date`, dopo aver eseguito il codice:

```
Date B = new Date();  
Date C = B;  
A.comunicaInizio(B);  
A.comunicaInizio(C);
```

- sulla **macchina remota**, ci saranno due oggetti di tipo `Date()` a cui punteranno le due variabili;
- sulla **macchina locale**, fanno riferimento allo stesso oggetto.



Passaggio nello stesso metodo

Se l'oggetto **A** fornisce il metodo remoto `comunicaInizioFine(Date x, Date y)`, allora dopo avere seguito il codice:

```
Date B = new Date();  
Date C = B;  
A.comunicaInizioFine(B, C);
```

sulla macchina remota ci sarà un solo oggetto `Date` a cui punteranno i due riferimenti passati come parametri.



4.4 Caricamento dinamico delle classi

Java RMI permette il passaggio di oggetti come parametri, valori restituiti o eccezioni, attraverso la **serializzazione**.

Questo si scontra con un'altra caratteristica di Java, quella del **caricamento delle classi a tempo di esecuzione** che risulta più complessa nel momento in cui stiamo passando ad un metodo offerto da un server remoto un parametro che è un'istanza di una sottoclasse della classe dichiarata nella firma del metodo. In questo caso, l'oggetto remoto può trovarsi nella situazione in cui non conosce esattamente com'è strutturata la classe di cui l'oggetto passato è istanza.

Questo viene risolto da *Java RMI* attraverso il **caricamento dinamico delle classi**: quando si fa il marshalling degli oggetti per la trasmissione, essi vengono anche annotati con il **codebase**, cioè con l'*URL* di un server *WWW* da dove è possibile trovare il file `.class`.

Quando viene effettuato l'unmarshalling dell'oggetto, il `ClassLoader` cerca di risolvere il nome della classe nel suo contesto, poi, in caso non sia possibile, viene acceduto il file `.class` per poter ricreare l'oggetto all'altro capo della comunicazione.

Questo meccanismo di caricamento dinamico permette anche il caricamento dinamico degli stub, con lo stesso meccanismo di annotazione che viene effettuato nel *marshalling*. Questo, ad esempio, avviene quando si passa un riferimento remoto, che consiste nell'inviare lo stub.

4.5 Meccanismo di Marshalling

Il **marshalling** è il processo per trasferire un oggetto da un client a un server, l'intento è di avere lo stesso oggetto presente in un programma in esecuzione anche in un altro programma in esecuzione.

La **serializzazione** non ha necessariamente questo scopo poiché si occupa solo di trasformare i dati in uno stream di byte.

Fare il **marshalling** di un oggetto in Java significa:

- effettuare una **serializzazione**;
- inserire lo **stub** dell'oggetto remoto;
- aggiungere informazioni all'oggetto (il codebase della classe dell'oggetto).

Il meccanismo di marshalling di *Java RMI* si basa sul meccanismo di serializzazione effettuato da `ObjectOutputStream`: questa classe offre la possibilità di poter modificare il comportamento tramite il quale gli oggetti vengono scritti come stream di byte.

Il meccanismo di serializzazione per il marshalling avviene modificando 3 metodi della classe `ObjectOutputStream`:

1. il metodo **`replaceObject()`** definisce un metodo alternativo per serializzare un oggetto sullo stream;
2. il metodo **`enableReplaceObject()`** restituisce un booleano che stabilisce se l'istanza deve oppure no specializzare il meccanismo di serializzazione, usando il metodo `replaceObject()`;
3. il metodo **`annotateClass()`** permette di inserire più informazioni sulla classe: viene usato per specificare il **codebase** permettere il caricamento dinamico.

L'operazione più complessa è quella di **`replaceObject()`**. Il meccanismo di serializzazione di RMI specifica (abilitando il Flag booleano restituito da `enableReplaceObject()`) Che questo metodo deve essere richiamato ogni volta che viene invocato **`writeObject()`**.

Il metodo `replaceObject()` fa le seguenti operazioni:

- se l'oggetto da serializzare è una istanza di `java.rmi.Remote` e quindi è un riferimento remoto, e l'oggetto risulta esportato al run time di *RMI*, allora viene restituito dallo stub per quell'oggetto remoto utilizzando il metodo `java.rmi.server.RemoteObject.toStub()`, che è una classe che implementa l'interfaccia `Serializable`.
Nel caso in cui l'oggetto remoto non sia esportato, allora si restituisce l'oggetto remoto stesso.
- se l'oggetto da serializzare non è un oggetto remoto allora viene restituito a `writeObject()`.

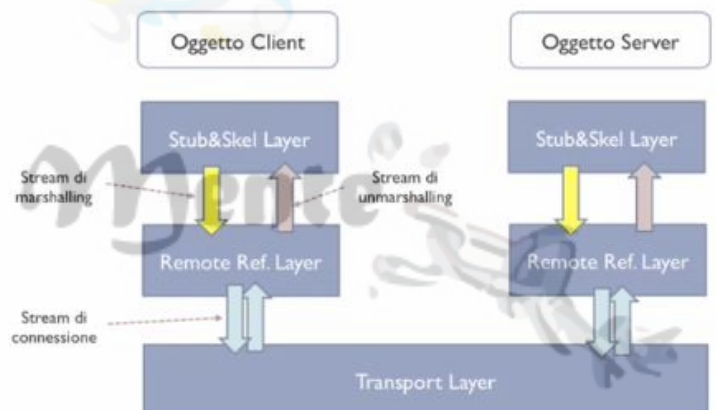
Tramite questo meccanismo viene risolto un problema nelle invocazioni: quando viene passato un riferimento remoto come parametro, questo viene copiato sullo stream, ma non abbiamo il vincolo di dichiarare un riferimento remoto come serializzabile. Infatti, quando si passa un riferimento remoto che è esportato (quindi attivo), esso viene restituito dal suo stub che è un'istanza di `java.rmi.server.RemoteStub`, che è una classe che implementa l'interfaccia `Serializable`.

Questo meccanismo spiega anche la modalità con la quale viene assicurata l'**integrità referenziale**: poiché i parametri di una stessa invocazione remota utilizzano lo stesso stream di output, parametri che si riferiscono allo stesso oggetto nella stessa invocazione verranno serializzati nel flusso come se facessero riferimento allo stesso oggetto, e verranno deserializzati nella stessa maniera all'altro capo dello stream.

4.6 Architettura di Java RMI

Il sistema di *Java RMI* è strutturato su 3 layer:

1. **Stub/Skeleton Layer**: comprende gli stub lato client e gli skeleton lato server;
2. **Remote Reference Layer**: specifica il comportamento dell'invocazione e la semantica del riferimento (unicast, multicast, etc.);
3. **Transport Layer**: si occupa della connessione e della sua gestione.



L'applicazione dell'utente si trova in cima a questi livelli e interagisce con il **livello stub/skeleton**.

Stub/Skeleton Layer

Essendo il livello più alto di *Java RMI*, è l'interfaccia tra l'applicazione (cioè le classi Java scritte dal programmatore) ed il resto del sistema.

Lo **stub** è incaricato di:

- Iniziare la connessione con la macchina virtuale remota, chiamando il **Remote Reference layer**;
- Effettuare il marshalling verso uno stream di marshal, fornito dal layer di **Remote Reference**;
- Attendere il risultato dell'invocazione;
- Effettuare l'unmarshalling degli oggetti;
- Restituire il risultato ad un oggetto client.

Lo **skeleton** è incaricato di effettuare il dispatching verso l'oggetto remoto, cioè si assicura che l'invocazione sia effettuata sull'oggetto remoto in attesa di invocazioni. Lo *skeleton* si occupa di:

- Effettuare l'unmarshalling dal *Remote Reference layer* dei parametri per l'invocazione del metodo remoto;
- Effettuare il marshaling del valore restituito.

Stub e skeleton vengono creati dall'**RMI compiler rmic**, un tool fornito dal *JDK*. Inoltre, è sbagliato dire che, a partire da *Java 2*, siano stati "eliminati" gli *skeleton*, in quanto i compiti che venivano effettuati da essi sono effettuati altrove e, quindi, il layer stub/skeleton è comunque esistente nelle versioni successive.

Remote Reference Layer

Questo layer si occupa di interfacciare il livello di trasporto con quello di stub/skeleton. In particolare, si occupa di varie invocazioni:

- **Invocazioni unicast**: da un solo client verso un solo server;
- **Invocazioni multicast**: un solo client verso una serie di server, in maniera da garantire la ridondanza.
- **Invocazioni di oggetti attivabili**: presenti in memoria secondaria e richiamati durante l'invocazione.

Java RMI, attualmente, implementa solo la **modalità unicast** e la **modalità di invocazione di oggetti attivabili**.

Questo layer fornisce verso lo stub/skeleton l'interfaccia `java.rmi.server.RemoteServer`, che espone un metodo `invoke()` per effettuare l'inoltro dell'invocazione, che viene chiamato dallo *stub*.

Il *Remote Reference layer* interagisce verso il basso con il livello di trasporto, verso il quale utilizza lo stream di connessione.

Transport Layer

Il **livello di trasporto** offre una comunicazione **connection-oriented** (è possibile l'implementazione interna *connectionless*). Questo livello ha il compito di:

- stabilire la connessione verso macchine con indirizzi IP remoti;
- gestire le connessioni e monitorare il loro stato;
- rimanere in ascolto per connessioni in arrivo;
- gestire una tabella di oggetti remoti che sono attivi nella *JVM* locale;
- stabilire una connessione per le chiamate in entrata;
- identificare l'oggetto dispatcher a cui inoltrare la connessione.

Il protocollo utilizzato da *RMI* è chiamato **Java Remote Method Protocol**: protocollo per cercare e usare oggetti remoti.

Ma questa architettura permette l'inserimento di altri protocolli; il più importante è il protocollo di **CORBA**, cioè **Internet-ORB Protocol**, che ha reso compatibile il mondo *Java* (*RMI*) con il mondo *CORBA*.

RMI-IIOP è utilizzato in maniera stabile da *Java Enterprise Edition*, diventando uno standard de-facto per quanto riguarda l'interoperabilità tra applicazioni distribuite *Java*.

4.7 La differenza tra il modello a oggetti locale e quello remoto

Uno degli obiettivi di *Java RMI* è di mantenere simile la programmazione tra **oggetti locali** e **remoti**. Una prima modifica è stata quella dell'implementazione di alcuni metodi della classe `Object` che non sono appropriate per la programmazione distribuita e che sono state rimpiazzate dall'implementazione di `java.rmi.RemoteObject`, che ridefinisce i metodi `equals()`, `hashCode()`, `toString()`:

- Il metodo `X.hashCode()` viene ridefinito in maniera che restituisca lo stesso codice per due stub diversi di oggetti remoti che si riferiscono allo stesso oggetto remoto.
- Il metodo `X.equals()` restituisce un booleano che è vero se il riferimento remoto passato è uguale a quello di `X`: il confronto viene fatto sugli stub e quindi l'uguaglianza è effettuata sui riferimenti e non sul contenuto.
- Il metodo `toString()` su un oggetto remoto restituisce informazioni aggiuntive sulla macchina sulla quale quell'oggetto remoto si trova, oltre alle informazioni che riguardano il nome della classe ed un codice hash.

Infine, a differenza degli oggetti locali, i client di oggetti remoti interagiscono solo con l'interfaccia remota.

Rispetto al **passaggio di parametri**, i riferimenti remoti possono essere passati a (o restituiti da) metodi come parametri (o valori restituiti). La semantica del passaggio di parametri è diversa in quanto argomenti locali a invocazioni remote sono passati per copia, mentre quelli remoti sono passati per riferimento.

Per quanto riguarda la **gestione dei tipi**, si può effettuare il casting Java di un oggetto remoto ad una qualsiasi delle interfacce remote supportate dall'implementazione dell'oggetto e si può utilizzare l'operatore `instanceof` per verificare a tempo di esecuzione le interfacce supportate da un oggetto remoto, applicando l'operatore al riferimento remoto.

Infine, la differenza nelle **invocazioni di metodo** consiste nel fatto che le invocazioni remote forzano il programmatore a dover gestire esplicitamente le eccezioni dei metodi remoti; a differenza delle invocazioni locali che non richiedono l'intervento del programmatore. Questo perché l'invocazione di un metodo remoto è più complesso e malfunzionamenti sono più numerose e difficili da trattare.

Capitolo 5 – Creazione di un programma Java RMI

5.1 Processo di creazione di un programma Java RMI

Un programma *Java RMI* si suddivide in due sotto processi:

1. uno che procede verso lo sviluppo e l'esecuzione del **server**;
2. l'altro che va in direzione dello sviluppo e dell'esecuzione del **client**. In particolare, lo sviluppo del client risulta essere influenzato minimamente da quello del server: infatti, il programmatore lato client alla necessità solo dell'interfaccia remota per poter proseguire nello sviluppo del programma client.



1. Definizione dell'interfaccia remota

Il primo passo consiste nel definire quali sono i servizi che vengono offerti dal **server**, specificandoli all'interno di un'interfaccia. Questa interfaccia rappresenta un "contratto" che vincola il server ad offrire dei servizi. L'implementazione sarà carico dal server e nascosta al client.

L'interfaccia remota deve:

- derivare da `Remote`;
- tutti i metodi remoti devono poter lanciare l'eccezione `java.rmi.RemoteException`.

2. Implementazione del server

Un oggetto remoto in Java deve essere istanza di una classe che:

- implementi uno più interfaccia remota (cioè che estendono `Remote`);
- derivi da `java.rmi.UnicastRemoteObject` il quale a sua volta estende `RemoteServer` che è `Serializable`.

Il fatto che il server derivi da `UnicastRemoteObject` implica il costruttore del nostro server deve esplicitamente essere scritto (anche se vuoto) in quanto è necessario che costruttore della sottoclasse lanci esplicitamente l'eccezione `RemoteException` che viene lanciata dal costruttore della superclasse.

3. Compilazione del server

In questa fase è importante sapere esattamente dove vengono messi i file `.class` che vengono creati dal compilatore `javac`.

4. Generazione Stub e Skeleton

A questo punto si possono generare automaticamente i file *stub* e *skeleton* tramite uno **stub compiler**, chiamato `rmic`.

5. Lato Server: servizio di Naming

A questo punto è necessario che l'oggetto remoto possa essere accessibile dai client che ne vogliono invocare i servizi. A tale scopo, *Java RMI* usa un servizio di **naming** chiamato **rmiregistry**. Questo programma deve essere lanciato prima di eseguire l'oggetto server, in quanto il server all'inizio si registrerà tramite il *naming* con un'etichetta.

Il servizio di naming deve essere lanciato dalla directory dove si trova il file `.class` dello stub, dell'oggetto remoto e dell'interfaccia remota.

6. Esecuzione del server

Ora si può eseguire il server eseguendo con la macchina virtuale la classe appropriata. L'unica cosa da fare è di scegliere una politica di sicurezza per la macchina virtuale attraverso una **policy** fornita da linea di comando.

La scelta di una policy è abbastanza complessa ma nel nostro caso possiamo adottare una politica estremamente liberale nella quale "tutto è permesso".

Il file di policy viene fornito, tramite linea di comando, alla macchina virtuale attraverso il comando `java -Djava.security.policy=nomefile`.

Se ci dimentichiamo di creare il file di policy viene generato un errore che può essere fuorviante.

7. Registrazione del Server sul servizio di Naming

Appena lanciato il server, esso deve registrarsi sul servizio di naming. Vengono usati dei metodi statici del package `java.rmi.Naming`. Per motivi di sicurezza, non è possibile che il server si registri su un servizio di naming che non sia sul suo stesso host, quindi non è possibile avere un servizio di naming "esterno" e questa è una limitazione notevole per i servizi distribuiti.



JAVA EE

(Java Enterprise Edition)

Capitolo 6 – Java EE

6.1 Introduzione a Java EE

Nel mercato odierno, le applicazioni hanno la necessità di accedere a dati, servizi e sistemi esterni, a livello globale. È quello che le aziende stanno inseguendo, cercando allo stesso tempo di minimizzare i costi il più possibile ed utilizzando delle tecnologie robuste che sono in grado di gestire carichi pesanti.

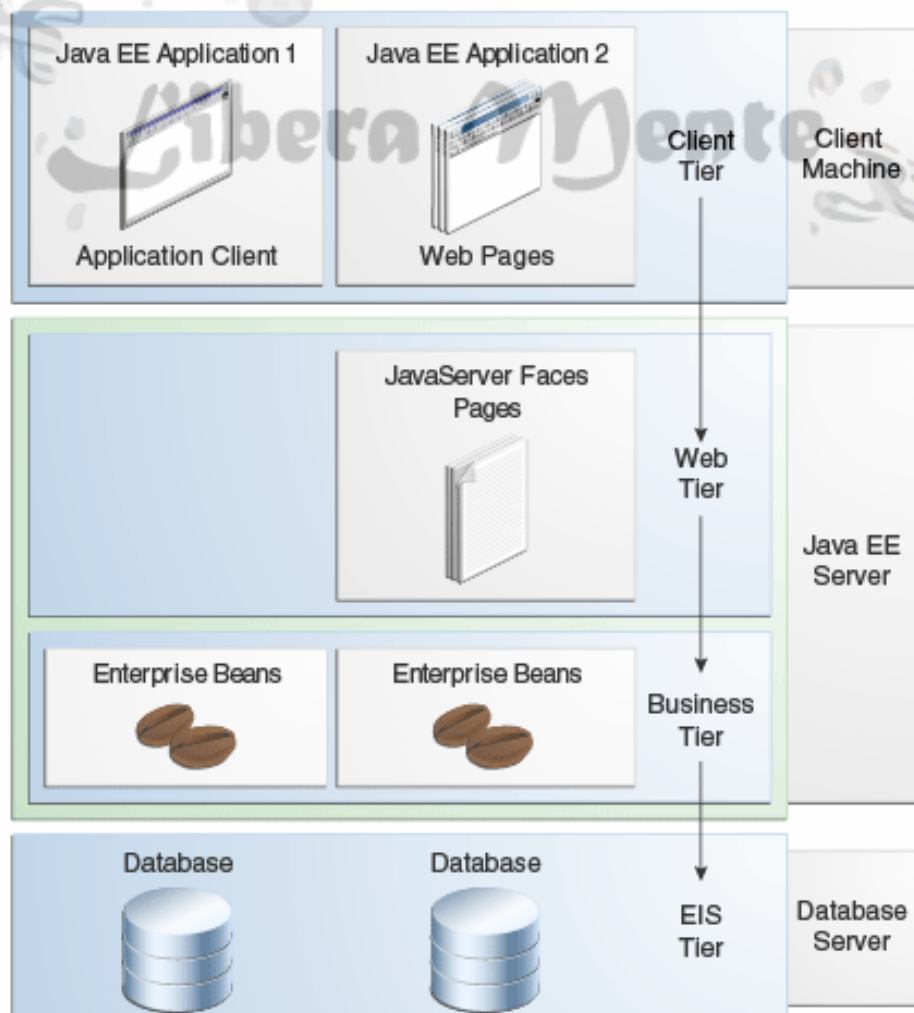
Molte compagnie inoltre devono combinare questi sistemi complessi con sistemi informativi aziendali già esistenti e nello stesso tempo sviluppare applicazioni *B2B* (business to business) e *B2C* (business to customer) per comunicare con i partner commerciali o con i clienti.

Per questo motivo nasce **Java EE**: un **insieme di specifiche** per applicazioni **enterprise** che assicurano l'efficienza, la sicurezza e l'affidabilità della tecnologia lato server. L'obiettivo è avere un'efficienza maggiore con poche risorse e investimenti, garantendo un'alta disponibilità, scalabilità e sicurezza.

6.2 Architettura Multilayer

La piattaforma *Java EE* usa un **modello applicativo distribuito e multilivello** per applicazioni di tipo *enterprise*. La logica dell'applicazione è divisa in **componenti** ognuno con una certa funzionalità, e tutte queste componenti (che creano un'applicazione *Java EE*) sono installati su varie macchine in base al livello a cui quella componente appartiene. Una applicazione *Java EE* è considerata a tre livelli (**three-tier**) perché è distribuita su 3 punti distinti:

1. **Client**;
2. **Server EE**;
3. **Database/legacy**.



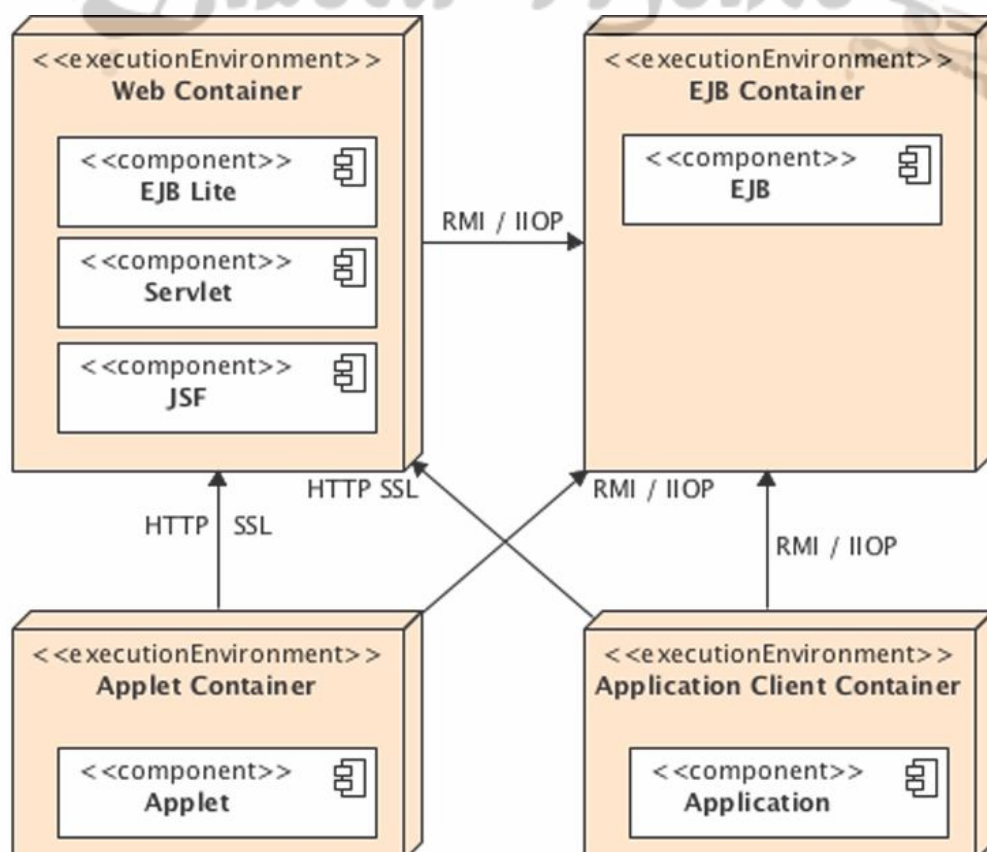
6.2.1 Architettura: Container

Le varie specifiche di *Java EE* sono implementate da diversi **containers**. I *containers* sono ambienti runtime *Java EE* che:

- hanno un ruolo specifico;
- supportano un set di API;
- fungono da interfaccia tra una **componente** e le funzionalità a basso livello che supporta la **componente**. Le componenti possono occuparsi della gestione del ciclo di vita dell'applicazione, l'iniezione di dipendenze, concorrenza, e così via.

Java EE ha quattro tipi di containers:

1. **Applet Containers** sono forniti dalla maggior parte dei Web browser per eseguire i componenti dell'applet. Il container dell'applet utilizza una sicurezza di tipo sandbox; questo significa che il container impedisce a qualsiasi codice scaricato sul computer locale di accedere alle risorse del sistema locale, come processi o file.
2. Gli **application client containers (ACC)** includono una serie di classi Java, librerie e altri file necessarie per portare l'iniezione, la gestione della sicurezza e il servizio di naming alle applicazioni *Java SE*. L'ACC comunica con il **container EJB** che utilizza *RMI-IIOP* e il **web container** con *HTTP*.
3. Il **Web Container** fornisce i servizi per la gestione e l'esecuzione dei componenti web (servlet, EJB Lite, JSP, pagine JSF e servizi Web). È responsabile dell'istanziamento, dell'inizializzazione e dell'invocazione delle servlet e del supporto dell'*HTTP* e *HTTPS*.
4. L'**EJB container** è responsabile della gestione e dell'esecuzione dei **bean enterprise** (bean di sessione e bean a messaggi) contenenti il livello di logica aziendale dell'applicazione *Java EE*. Questo container crea nuove istanze di bean, gestisce il loro ciclo di vita e fornisce servizi come transazione, sicurezza, concorrenza, distribuzione, servizio di naming o la possibilità di essere invocato in modo asincrono.



6.2.2 Architettura: Componenti

Java EE definisce 4 tipi di **componenti** che un'implementazione deve supportare:

1. **Applet**: applicazioni GUI eseguite in un Web browser.
2. **Applications Client**: sono programmi che vengono eseguiti su un client. In genere sono GUI o programmi che hanno accesso a tutte le funzionalità del livello intermedio di *Java EE*.
3. **Web applications** (composte da servlet, pagine JSP e JSF) sono eseguite in un **Web container** e rispondono alle richieste *HTTP* dei client.
4. **Business applications** (composte da Enterprise Java Beans, Java Message Service, Java Transaction API, chiamate asincrone, servizio timer, RMI/IIOP) vengono eseguite in un **EJB container**. Gli *EJB* sono componenti per l'elaborazione della logica di business. Possono essere accessibile localmente e in remoto tramite *RMI*.

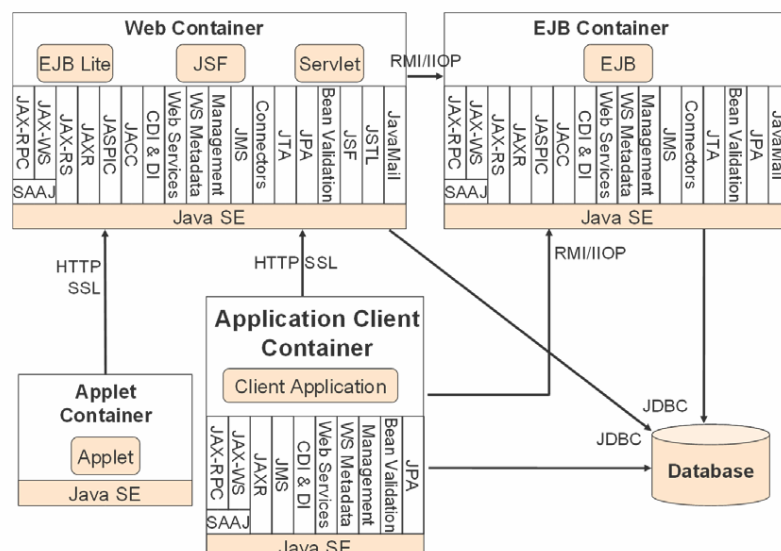
Prima di essere eseguita, una *componente* deve essere assemblata in un **Java EE module** e **deployata** nel suo container. Questo processo specifica per ogni componente i settaggi del container come:

- il servizio di sicurezza permette di configurare l'autenticazione e autorizzazione di componenti web o enterprise bean;
- il servizio di transazioni permette di definire una transazione composta dall'invocazione di diversi metodi;
- *JNDI* fornisce l'unica interfaccia che le componenti usano per accedere ai servizi e risorse del server;
- invocazione remota di metodi.

6.2.3 Architettura: Servizi forniti dal Container

I containers forniscono **servizi** per le componenti deployate. In particolare, i containers consentono allo sviluppatore di concentrarsi sull'implementazione della logica di business piuttosto che sulla risoluzione dei problemi tecnici affrontati nelle applicazioni aziendali. *Java EE* offre i seguenti servizi:

- **Java Transaction API**: abilita le transazioni distribuite;
- **Java Persistence API**: standard API per l'*object-relational mapping* (ORM);
- **Java Message Service**: comunicazione asincrona tra le componenti;
- **Servizi di sicurezza**: cioè servizi di autenticazione e autorizzazione (JAAS);
- **Web Services**: Java API per *XML WEB Services* (JAX-WS) e Java API per *RESTful Web Services* (JAX-RS);
- **Dependency Injection**: le risorse possono essere iniettate nei componenti "managed".

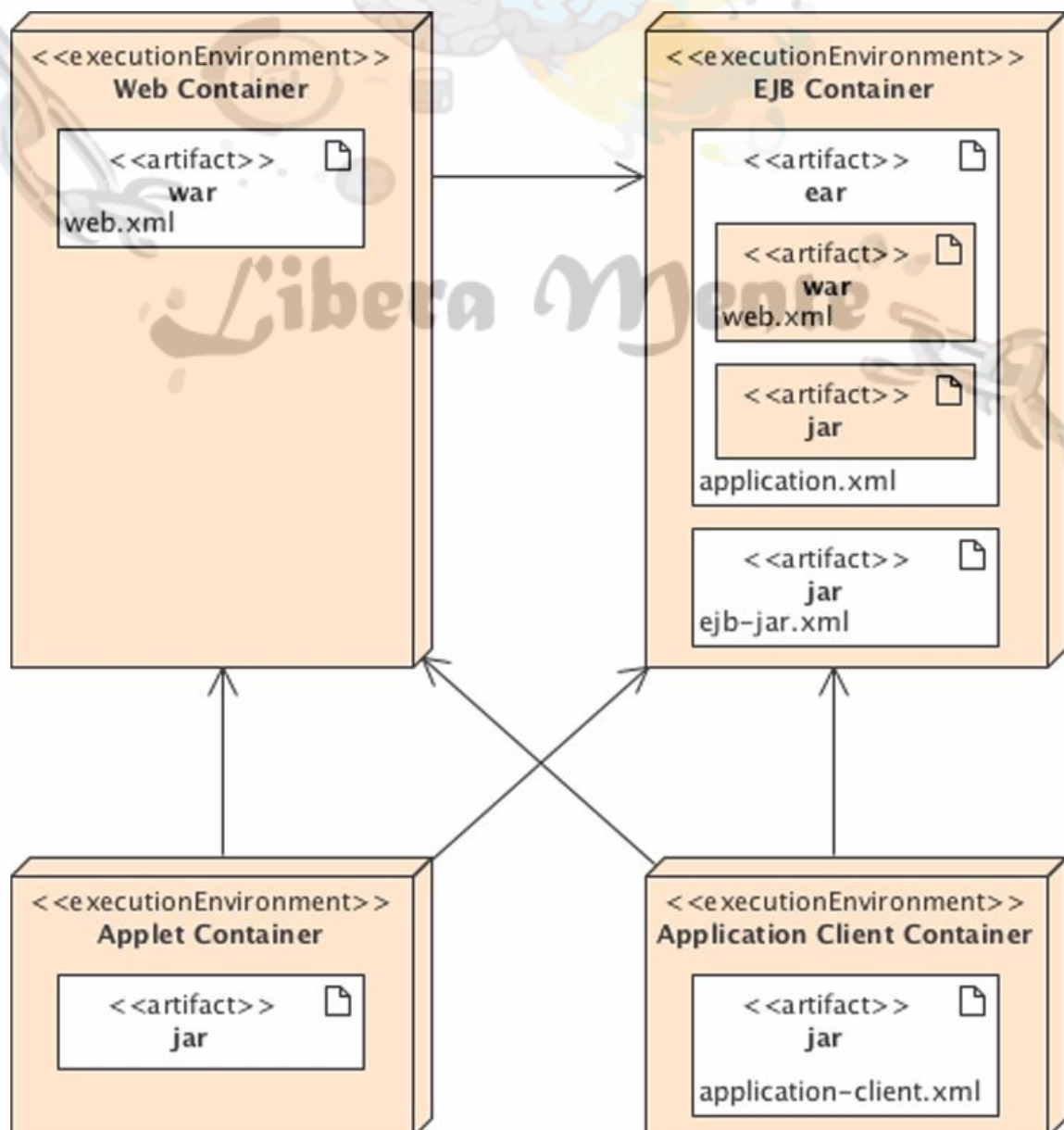


6.2.4 Architettura: Packaging

Per poter fare il deploy in un container, le componenti devono prima essere impacchettate in degli archivi standard formattati. *Java SE* definisce i **file jar** (Java Archive files) che sono usati per aggregare molti file in un solo file compresso.

Java EE definisce tipi di modulo differenti ognuno con un suo formato:

- Un **application client module** contiene classi Java e altri file di risorse impacchettati in un **file jar**. Questo *file jar* può essere eseguito in un ambiente *Java SE* o in un *application client container*. Il *file jar* contiene una directory `META-INF` opzionale per le meta informazioni che descrivono l'archivio.
- Un **EJB module** contiene una o più sessioni e/o **Message-driven beans** (MDB) confezionati in un *file jar*.
- Un **Web application module** contiene servlet, JSP, pagine JSF e servizi Web, nonché qualsiasi altri file relativi al Web (pagine HTML e XHTML, fogli di stile a cascata (CSS), script Java, immagini, video e così via). Tutti questi artefatti sono confezionati in un *file jar* con estensione **.war**.
- Un **enterprise module** può contenere moduli EJB e moduli Web application e altre librerie esterne. Tutto questo è racchiuso in un archivio (un file jar con estensione .ear) in modo che la distribuzione di questi vari moduli avviene simultaneamente e coerentemente.



6.2.5 Architettura: Annotazioni e Deployment Descriptor

Nel paradigma di programmazione, ci sono due approcci: **programmazione imperativa** e **programmazione dichiarativa**.

- La **programmazione imperativa** specifica l'algoritmo per raggiungere un obiettivo (cosa deve essere fatto);
- La **programmazione dichiarativa** specifica come raggiungere questo obiettivo (come deve essere fatto).

In *Java EE*, la **programmazione dichiarativa** è fatta utilizzando i **metadati**, ovvero annotazioni o/e descrittori di distribuzione.

Come abbiamo visto, le componenti vengono eseguiti in un container ed ogni container offre al componente un insieme di servizi. I **metadati** vengono utilizzati per dichiarare e personalizzare questi servizi e associare informazioni aggiuntive insieme a classi Java, interfacce, costruttori, metodi, campi o parametri.

Il vantaggio maggiore delle **annotazioni** è che riducono significativamente l'ammontare del codice che lo sviluppatore deve scrivere e usando le annotazioni si può evitare di avere dei **DD** (deployment descriptor). D'altro canto, i DD sono dei file XML esterni che possono essere cambiati senza necessitare modifiche al codice sorgente e quindi ricompilazione. Se usiamo entrambe le cose XML ha la precedenza sopra le annotazioni.

6.3 Standard

Java EE è basato su degli **standard**. Questo significa che *Java EE* è stato sottoposto ad un processo di standardizzazione sviluppato attraverso il *JCP* (Java Community Process). Infatti, *Java EE* è chiamato **specifica ad ombrello** perché unisce insieme un certo numero di altre specifiche.

Gli standard sono importanti per facilitare la comunicazione e lo scambio. *Java EE* fornisce un ambiente **open source** rendendone estremamente portabile.

JCP

JCP è una organizzazione coinvolta nella definizione delle versioni future e delle *feature* future della piattaforma Java. Quando c'è la necessità di standardizzare un nuovo componente o un API, il capo delle specifiche crea un *JSR* e un gruppo di esperti che devono produrre alcuni documenti da far approvare prima all'EC (committente esecutivo), e poi diventa pronto per essere rilasciato alla community per l'implementazione.

Capitolo 7 – Context e Dependency Injection

7.1 Java Beans

La prima versione di *Java EE* ha introdotto il concetto di **inversion of control** (IoC) che sta a significare che è il container a prendere il controllo del codice appartenente alla logica di business (e che lo esegue) in modo da poter fornire servizi senza che il programmatore debba scriverli.

Java EE 6 ha introdotto il **Context and Dependency Injection** (CDI). Il CDI dà a tutti i **Managed Beans*** un modello di programmazione, trasformando quasi tutte le componenti *Java EE* in beans che sono: **iniettabili**, **intercettabili** e **gestibili**.

Il CDI è un framework costruito sul concetto di “**loose coupling, strong typing**”, che significa che i beans sono poco accoppiati ma molto tipizzati.

Il **disaccoppiamento** permette di poter inserire gli **interceptors**, **decorators** e gli **eventi** all'interno dell'intera piattaforma, omogeneizzando il web layer e il back-end layer.

* Un **bean** è un **Managed Bean** (cioè Bean gestito) quando non abbiamo mai bisogno di maneggiare l'istanza del bean ma è il container che se ne occupa. Quindi quasi tutte le componenti sono *Managed Bean*, infatti non facciamo noi `new` per creare una istanza, e non ci occupiamo noi della *Garbage Collection*.

Java SE ha i suoi **JavaBeans**, *Java EE* ha gli **Enterprise JavaBeans** (che possono essere visti come un Managed Bean + funzioni extra), ma anche altre componenti, tra cui i *Managed Beans* e i **POJO**.

I **POJO** (Plain Old Java Objects) sono degli oggetti che girano all'interno della Java Virtual Machine e che seguono delle convenzioni: naming, getter/setter, costruttore di default.

I JavaBeans di *Java EE* sono:

- eseguiti in un container;
- contengono metadati;
- costruttore non final;
- usufruiscono dei servizi del container (sicurezza, transazioni, etc.).

I Managed Beans sono degli oggetti gestiti dal container che supportano soltanto un piccolo insieme di servizi base, come:

- injection di risorse;
- gestione del ciclo di vita;
- interception.

Inoltre, i managed beans:

- hanno un ciclo di vita migliorato e gestito dal container per oggetti stateful (cioè che conservano un certo stato);
- sono legati a contesti ben definiti;
- portano un approccio type-safe (cioè rispettando il tipo) per la *dependency injection*, *interception* e *decoration*;
- sono specializzati con delle annotazioni `qualifier`;
- possono essere usati nelle expression language.
- ogni classe Java può diventare un Managed Bean che usa i servizi CDI.

7.2 Dependency Injection

* **Design pattern** = una soluzione progettuale generale ad un problema ricorrente.

Il **Dependency Injection (DI)** è un *design pattern* che disaccoppia componenti dipendenti. È una parte dell'**inversione del controllo**, in cui la preoccupazione che nasce spontanea è come ottenere la dipendenza necessaria, dopo aver de-accoppiato.

Un modo per pensare al *Dependency Injection* è pensare al **JNDI** (Java Naming and Directory Interface). Invece di un oggetto che cerca altri oggetti, il container inietta questi oggetti al nostro posto. Questo è chiamato anche **Hollywood Principle**: “Non chiamarci (cercare oggetti), ti chiamiamo noi (iniezione di oggetti)”.

Java EE ha introdotto due nuove specifiche per realizzare un vero Dependency Injection:

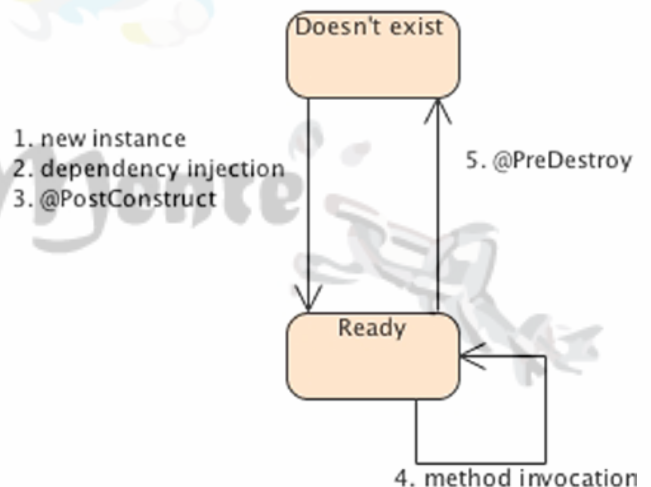
1. **Dependency Injection (DI)**;
2. **Context and Dependency Injection (CDI)**.

7.3 Life-cycle Management

Il **ciclo di vita di un POJO** è molto semplice: lo sviluppatore Java, crea una nuova istanza di una classe con `new` e si attende che il *Garbage Collector* liberi memoria. Ma se volessimo far girare un *CDI Bean* all'interno di un container, non siamo autorizzati ad usare `new`.

Invece, abbiamo bisogno di **iniettare** il bean e il container sarà il responsabile per la gestione del ciclo di vita del bean. Quando iniettiamo un bean, il container:

- è il responsabile per creare una istanza [1];
- risolve poi le dipendenze ed invoca ogni metodo annotato con **@PostConstruct** prima che venga invocato il primo metodo di business sul bean [2] [3] [4];
- quindi la notifica di callback **@PreDestroy** segnala che l'istanza è in procinto di essere rimossa dal contenitore [5].



7.4 Scope and Context

I *Beans CDI* possono essere **stateful**, il che significa che sono in esecuzione in uno **scope** ben definito. I *Beans CDI* hanno di solito degli scope predefiniti, come: **Request**, **Session**, **Application** e **conversation**.

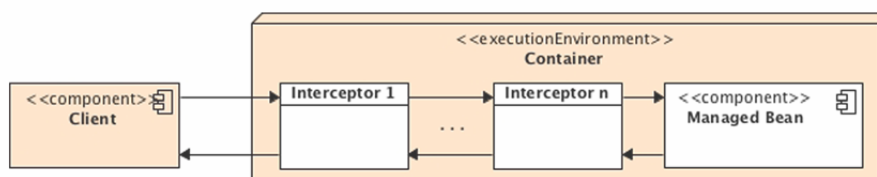
A differenza delle **componenti stateless**, diversi clients di un *bean stateful* vedono il bean in stati differenti. Quando il bean è stateful, è importante capire quale istanza del bean ha il client:

1. Clients in esecuzione nello stesso scope vedranno la stessa istanza del bean;
2. Clients in scope differenti potrebbero vedere istanze diverse.

In tutti i casi, è il container a controllare il ciclo di vita delle istanze, esplicitando la creazione o la rimozione di esse.

7.5 Interception

Gli **interceptors** sono usati per interporli tra un bean e un'invocazione di un metodo su questo bean. Questo aspetto, è simile all'**Aspect-oriented Programming (AOP)**. *AOP* è un paradigma di programmazione che separa quelle che sono le problematiche trasversali dalla logica di business ("cross-cutting concerns"). Il container si occupa di chiamare gli *interceptor* prima o dopo l'invocazione di un metodo.



7.6 Loose Coupling and Strong Typing

Gli *interceptors* rappresentano un mezzo potente per disaccoppiare dettagli tecnici della logica di business. Infatti, la gestione del ciclo di vita di un bean disaccoppia il bean stesso dalla gestione del suo ciclo di vita. Esistono altri modi per disaccoppiare; ad esempio, i beans possono usare event notifications per disaccoppiare event producer da event consumer.

7.7 Deployment Descriptor

Un **deployment descriptor XML** descrive come una componente, modulo o applicazione dovrebbe essere configurata. Con *CDI*, il deployment descriptor è chiamato **beans.xml** ed è obbligatorio. Inoltre, può essere usato per configurare alcune funzionalità (interceptors, decorators, alternative, etc.), ma è essenziale per attivare il *CDI*. Questo perché *CDI* necessita di identificare il bean nel nostro class path (questo viene chiamato bean discovery) ed è durante la fase di scoperta del bean che avviene la magia: cioè quando *CDI* trasforma un POJO in un CDI Bean.

Durante la fase di deployment, *CDI* controlla tutti i `jar` e i `war` dell'applicazione e ogni volta che trova un deployment descriptor `beans.xml`, gestisce tutti i POJO che diventeranno poi CDI Beans. Senza `beans.xml` nel class path (sotto `META-INF` o `WEB-INF`), *CDI* non sarà in grado di usare l'injection, interception, decoration e così via. Senza questo file di markup, il *CDI* non funzionerà.

7.8 Punti di Injection

L'annotazione che permette di iniettare un bean all'interno di un altro è **@Inject**. L'*injection* può avvenire attraverso tre meccanismi (scegliamo quello che preferiamo):

1. su una **proprietà**:

```
@Inject
private NumberGenerator numberGenerator;
```

2. sul **costruttore**:

```
@Inject
public BookService (NumberGenerator numberGenerator) {
    this.numberGenerator = numberGenerator;
}
```

3. sul **setter**:

```
@Inject
public void setNumberGenerator (NumberGenerator numberGenerator) {
    this.numberGenerator = numberGenerator;
}
```

7.9 Qualifiers

Al momento dell'inizializzazione del sistema, il container deve verificare che esista un bean che soddisfi ogni punto di iniezione. Questo significa che se nessuna implementazione di `NumberGenerator` è disponibile, il container ci informerà di una dipendenza non soddisfatta e non effettuerà il deploy dell'applicazione.

- Se c'è solo una implementazione, l'iniezione funzionerà usando il qualifier **@Default**.
- Se più di una implementazione oltre quella di default erano disponibili, il container ci informerà di una dipendenza ambigua e non farà il deploy dell'applicazione. Questo perché l'algoritmo di risoluzione typesafe fallisce quando il container non è in grado di identificare esattamente un bean da iniettare.

Per far capire ad una componente quale implementazione deve essere iniettata, usiamo i **qualificatori** in quanto possiamo rinominare le nostre implementazioni come vogliamo, ma il punto di iniezione non ha necessità di essere alterato.

Un modo per evitare di moltiplicare a dismisura le annotazioni è usando i **membri**.

Invece di scegliere tra diverse implementazioni della stessa interfaccia, può essere necessario dover scegliere un'implementazione completamente **alternativa**, dipendente dallo scenario di deployment.

Una **alternativa** sono beans annotati con un qualificatore speciale

`javax.enterprise.inject.Alternative`. Di default le alternative sono disabilitate e necessitano di essere abilitate nel `beans.xml` descriptor per renderle disponibili per l'istanziamento e iniezione.

7.10 Producers

Quando si inietta un *CDI Bean*, il container tratta quest'ultimo come un *POJO* messo in un package con il file `beans.xml`.

Per impostazione predefinita, non è possibile iniettare classi come `java.util.Date` o `java.lang.String` perché queste classi sono impacchettate nel file **`rt.jar`** (java runtime environment classes) che non contiene il file `beans.xml`.

Una soluzione è usare i **producers**. In particolare aggiungiamo una classe che produce valori, annotati con `javax.enterprise.inject.Producers`, quindi ora questi valori possono essere iniettati.

7.11 Disposers

I producer sono usati per creare tipi di dato o *POJO* da far iniettare: in questo caso non avevamo bisogno di eliminarli.

In alcuni casi è necessario prevedere esplicitamente la distruzione degli oggetti. Infatti, un **disposer** è una notazione (**@Disposes**) che permette di realizzare una pulizia "personalizzata" di un oggetto che era stato restituito da un metodo producer.

7.12 Scope

CDI, oltre che la **dependency injection**, riguarda anche il **context**. Ogni oggetto gestito da CDI ha un *context* (ambito) definito ed un ciclo di vita che è legato all'ambito. In Java, lo scope di un POJO è piuttosto semplice: si crea un'istanza di una classe usando la parola chiave `new` e ci si affida alla Garbage Collection per liberare memoria.

Con il CDI, un bean è legato ad un contesto e rimane attivo in questo contesto fino a quando il container non lo distrugge. Non è possibile rimuovere manualmente un bean da un contesto.

Abbiamo diversi tipi di scope:

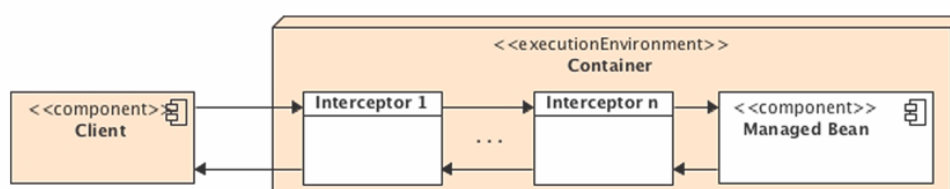
- **Application scope** (`@ApplicationScoped`): si estende per l'intera durata di un'applicazione. Il bean viene creato una sola volta per tutta la durata dell'applicazione e viene distrutto quando l'applicazione termina.
Questo ambito è utile per **classi utility** o **helper**, oppure per oggetti che memorizzano dati condivisi dall'intera applicazione. Gli svantaggi riguardano problemi di efficienza, concorrenza (dati acceduti da diversi thread).
- **Session scope** (`@SessionScoped`): si estende attraverso diverse richieste *HTTP* o diverse invocazioni identificate come una singola sessione utente. Il bean viene creato per la durata di una sessione *HTTP* e viene distrutto al termine della sessione.
Questo ambito è vantaggioso per gli oggetti che sono necessari per la durata di una sessione come user preferences o login credentials.
- **Request scope** (`@RequestScoped`): corrisponde a una singola richiesta *HTTP* o ad un'invocazione di un metodo. Il bean viene creato per tutta la durata dell'invocazione del metodo e viene rimosso quando il metodo finisce. Viene utilizzato per le classi di servizio o i *JSF backing beans* che sono necessari per la durata di una richiesta *HTTP*.
- **Conversation scope** (`@ConversationScoped`): si estende attraverso invocazioni multiple all'interno di sessioni con punti di partenza e fine identificate dall'applicazione.
- **Dependent pseudo-scope** (`@Dependent`): un **dependet bean** viene creato ogni volta che viene iniettato e il riferimento viene rimosso quando l'injection target viene rimosso. Questo è lo scope di default per il CDI.

7.13 Interceptors

Gli **interceptors** ti permettono di aggiungere funzionalità cross-cutting ai bean CDI. Quando un client invoca un metodo su un *managed bean*, il container è capace di intercettare la chiamata tramite uno o più interceptor, applicare una certa logica di business e poi invocare il metodo del bean.

Gli interceptor si dividono in 4 tipi:

1. **Constructor-level interceptors**: interceptor associati ad un costruttore di una classe specifica: `@AroundConstruct`;
2. **Method-level interceptors**: interceptor associati ad uno specifico business method `@AroundInvoke`;
3. **Timeout method interceptors**: interceptor che si interpongono al timeout di metodi con `@AroundTimeout`.
4. **Life-cycle callback interceptors**: interceptor che si interpongono su uno specifico evento del ciclo di vita di una istanza (`@PostConstruct` e `@PreDestroy`).



Requisiti

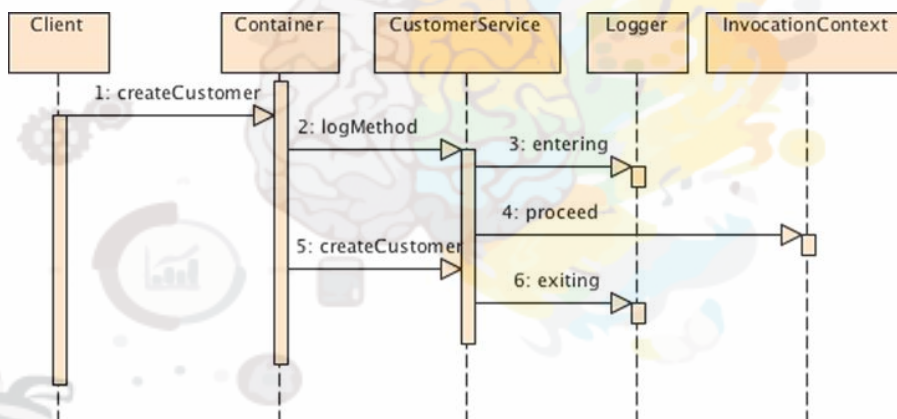
Per poter essere un `Interceptor`, il metodo deve avere questa firma:

@AroundInvoke

```
private Object <METHOD> (InvocationContext ic) throws Exception { ... }
```

Le regole seguenti si applicano ad un metodo *around-invoke*:

- i metodi possono avere i modificatori di accesso `public`, `private`, `protected` ma NON possono essere **static** o **final**.
- i metodi devono avere il contesto come parametro e ritornare un `Object` che è il risultato dell'invocazione del metodo che abbiamo invocato.
- il metodo può lanciare una eccezione controllata.
- il contesto passato permette di concatenare diversi `interceptor` in modo che il contesto sia passato da uno all'altro. Aggiungendo se necessario delle informazioni.



Classi Interceptors

La maggior parte delle volte si vuole isolare quelle che sono delle *cross-cutting concerns* in classi separate e dire al container di intercettare le chiamate su diversi `bean`. Il **Logging** è un esempio tipico di servizio da fornire in maniera trasversale.

Per specificare una **classe interceptor**, abbiamo bisogno di sviluppare una classe separata e istruire il container ad applicarla su uno specifico `bean` o metodo del `bean`.

Se si vuole applicare l'interceptor ad un intero `bean` ad esclusione si può usare l'annotazione:

`javax.interceptor.ExcludeClassInterceptors`.

Life-Cycle Interceptors

Con una annotazione *callback*, possiamo informare il container di invocare un metodo in una certa fase del ciclo di vita del `bean` con `@PostConstruct` e `@PreDestroy`.

Per esempio, se vogliamo effettuare il log di una entry ogni volta che una istanza di un `bean` è creata, abbiamo soltanto bisogno di aggiungere una annotazione `@PostConstruct` su un metodo del `bean` e aggiungere del meccanismo di logging al metodo.

Ma se invece abbiamo bisogno di catturare eventi legati al ciclo di vita tra vari tipi diversi di `beans`?

Una soluzione è di usare i **life-cycle interceptors** che ci permettono di isolare del codice in una classe e di invocarla quando un evento viene intercettato. I *life-cycle interceptors* prendono un parametro

`InvocationContext` e ritornano `void` invece che `Object`.

Binding fra Interceptor

Gli interceptor sono definiti nella loro specifica (JSR 318) e possono essere usati in qualsiasi Managed Bean. Ma la specifica CDI li ha estesi aggiungendo un **interceptor binding**, che sta a significare che l'interceptor binding può essere usato soltanto quando CDI è attivo.

Questo è **typesafe** ma non **loosely coupled** (debolmente accoppiato). CDI ci fornisce dei **interceptor binding** che introducono un livello di indirizzamento e basso accoppiamento. Un tipo di interceptor binding è una annotazione **@InterceptorBinding** che permette all'utente di fare il binding non direttamente verso una classe. Inoltre, come per gli interceptor, l'**interceptor binding** può essere applicato anche ad un solo metodo anziché su tutto il bean. L'uso dell'interceptor va abilitato esplicitamente nel `beans.xml`.

Concatenazione fra Interceptor

Interceptor ci permette anche di concatenare diversi interceptor. Infatti, l'annotazione **@Interceptors** è capace di attaccarsi a più di un interceptor, se gli forniamo una lista di interceptor come parametri, separati da una virgola. Quando è definito più di un interceptor, l'ordine in cui sono invocati è determinato dall'ordine in cui sono stati specificati nell'annotazione **@Interceptors**:

```
@Interceptors ({I1.class, I2.class})
```

7.14 Decorators

Gli interceptor eseguono dei task trasversali che sono perfetti per risolvere delle tematiche legate alla logica di business come la gestione delle transazioni, sicurezza o logging. Di natura gli interceptor non sono consapevoli dell'attuale semantica delle azioni che stanno intercettando e quindi non sono appropriati per separare diverse tematiche legate al business.

L'inverso è vero per i **decorators**. I **Decorators** sono un design pattern comune. L'idea è di prendere una classe e di wrappare un'altra classe intorno con delle funzionalità aggiuntive (decorarla appunto). In questo modo, quando chiamiamo una classe decorata, passiamo sempre attraverso lo strato di decorazione, prima di arrivare alla classe obiettivo. Decorators sono intesi per aggiungere logica addizionale ad un metodo di business. Gli *interceptors* e *decorators*, comunque, sono simili in molti modi, ma sono complementari. Di default i decorator sono disabilitati come le alternative ed interceptor. Se una applicazione ha sia *interceptors* che *decorators*, gli *interceptors* sono invocati per primo.

7.15 Events

Dependency Injection, *interceptors* e *decorators*, permettono un basso accoppiamento aggiungendo un comportamento addizionale che può variare sia a tempo di deploy che a tempo di runtime.

Gli **Event** permettono le interazioni tra i beans senza nessuna dipendenza a tempo di compilazione. Un bean può definire un evento, un altro bean può lanciare l'evento, e un altro bean ancora può gestire l'evento. I bean possono essere in packages separati e persino in tier separati dell'applicazione. Questo schema basilare segue il design pattern **Observe/Observable** (osservatore/osservabile).

Un **produttore** (**Event producer**) lancia metodi usando l'interfaccia **Event**. Un producer attiva eventi chiamando il metodo `fire()`, passa l'oggetto evento, e non è dipendente dall'osservatore.

Un **observer** è un bean con uno o più metodi `observer`. Ognuno di questi metodi `observer` prende un evento di uno specifico tipo come parametro che è annotato con **@Observes** e qualificatori opzionali. Il metodo `observer` è notificato di un evento se l'oggetto evento matcha il tipo dell'evento e tutti i qualifier.

Come per gran parte del CDI, la produzione di event e la sottoscrizione sono typesafe e permettono ai qualificatori di determinare quali eventi observer staranno ad osservare. Un evento può essere assegnato ad uno o più qualifier (con o senza membri), che permettono gli observer di distinguerlo da altri eventi dello stesso tipo.





JPA

(Java Persistence API)

Capitolo 8 – Java Persistence API

8.1 Introduzione

La maggior parte dei dati che le nostre applicazioni manipolano devono essere memorizzati in un database, per poi essere richiamati ed analizzati.

Nella maggior parte dei casi, si parla di **database relazionali**, nei quali ci sono molti concetti associati, come chiavi primarie, esterne e così via. Tutti questi vocaboli però sono estranei al linguaggio Java, il quale ha a che fare solo con oggetti che hanno un comportamento e uno stato (lo stato di un oggetto è accessibile solo durante l'esecuzione della *Java Virtual Machine*).

Alcuni oggetti però necessitano di essere persistenti, e gli oggetti che sono in grado di memorizzare il proprio stato per essere usato successivamente vengono detti **persistenti**.

Il principio dell **ORM** (Object-relational mapping) è di portare il mondo dei database e degli oggetti insieme.

Il **principio di ORM** è quello di delegare a tools esterni o framework il compito di creare una corrispondenza fra oggetti e tabelle, di conseguenza la facoltà di accedere al database relazionale. La mappatura che si andrà a creare tra database ed oggetti sarà **bidirezionale**.

Diversi framework sono in grado di fare ciò, ma quello che tratteremo è **Java Persistence API (JPA)**.

JPA è stato creato con *Java EE 5* ed è un'astrazione di **JDBC** (libreria Java) che lo rende indipendente da *SQL*. JPA è contenuto nel package `javax.persistence` e definisce **ORM**.

Il componente principale di JPA è l'**Entity Manager**, il quale fa operazioni **CRUD** (create, read, update e delete) su *DB*.

8.2 Entities

Quando si parla di mappare oggetti in un database relazionale, si dovrebbe usare il termine “**entità**” invece che oggetto, in quanto:

- gli **oggetti** sono istanze che vivono solo in memoria;
- le **entità** vivono brevemente in memoria e in modo persistente in un database e **possono essere gestite** da **JPA** e **interrogate** da **JPQL** (Java Persistence Query Language).

ORM ci permette di manipolare le entità, mentre dietro le quinte viene fatto l'accesso al database.

Una entità segue un ciclo di vita definito: con i metodi di **callback** e **listeners**, JPA ci permette di agganciare del codice di business ad eventi legati al ciclo di vita. Una entità, grazie al JPA viene vista come un semplice *POJO*, questo significa che può essere dichiarata, istanziata ed usata come una qualsiasi classe Java.

Esempio di POJO in JPA:

```
@Entity
public class Book {
    @Id @GeneratedValue
    private Long id;
    private String title;
    private Float price;
    private String description;
    private String isbn;
    private Integer nbOfPage;
    private Boolean illustrations;

    public Book() {
    }
    // Getters & setters
}
```

- Classe annotata con **@Entity**, in questo modo il *persistence provider* la considera come una classe persistente e non come un *POJO*.
- **@Id** definisce una chiave primaria nel database;
- **@GeneratedValue** sta ad indicare che l'id deve essere generato automaticamente dal *persistence provider*;
- agli altri attributi non annotati sarà applicato un mapping di default;

- ci deve essere un costruttore senza parametri, che sia `public` o `protected`, e se è presente allora sono permessi altri costruttori;
- una *entity class* deve essere una **top-level class**, cioè `enum` o `interface` non possono essere disegnate come entità.
- la classe non deve essere `final` così come i metodi o variabili di istanza;
- se una istanza di una entità deve essere passata per valore attraverso una interfaccia remota, allora la classe della entità deve implementare l'interfaccia `Serializable`.

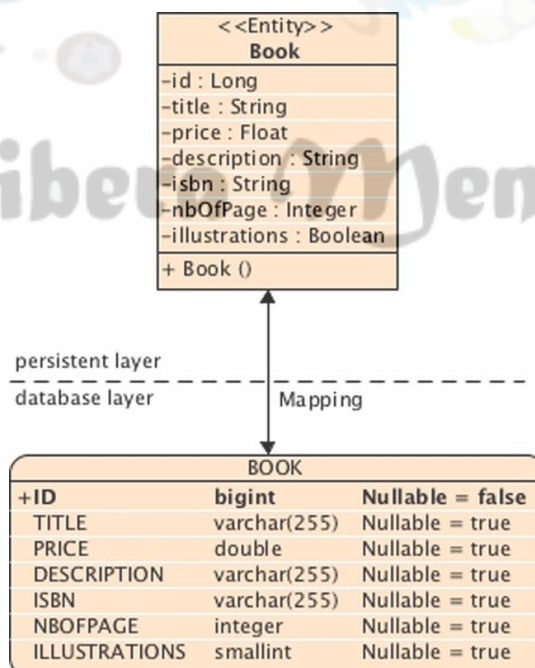
Entità: il ruolo dei metadati di ORM

Quello che **ORM** si propone di fare è delegare a tools esterni o framework (nel nostro caso JPA) il compito di creare una corrispondenza tra oggetti e tabelle, cosicché gli sviluppatori possano maneggiare entità invece che tabelle.

JPA mappa gli oggetti in un database attraverso i **metadata** associati ad ogni entità. I metadata possono essere scritti in due formati differenti:

- **Annotazioni**;
- **XML descriptors**: il mapping è definito in un file *XML* esterno che verrà deployato con le entità.

L'esempio precedente usa le annotazioni, che permettono al persistence provider di sincronizzare i dati delle variabili di istanza con i dati contenuti nelle colonne del database in modo che ad ogni modifica dei dati queste si riflettono anche nel database.



Come possiamo osservare, ogni tipo delle variabili di istanza è stato mappato con un determinato tipo nel database. Questa tecnica, nota come **configuration by exception**, permette di applicare delle regole di default dal container a meno di indicazioni diverse.

Alcune delle regole che vengono applicate di default sono:

- Il nome della entità diventerà il nome della tabella;
- I nomi degli attributi diventano i nomi delle colonne;
- Si applicano le regole di *JDBC* per mappare i tipi primitivi di Java con i tipi di dati contenuti in un database relazionale.

Entità: come fare le query

JPA ci permette di fare delle **query** sulle entità sfruttando il linguaggio *Java* (e non *SQL* per il *DB*). Il pezzo fondamentale dell'API che gestisce le entità è l'**Entity Manager** che ci permette di effettuare delle operazioni **CRUD** (create, read, update, delete) e delle query complesse usando **JPQL** (Java Persistence Query Language).

Tecnicamente l'*entity manager* è un'interfaccia la cui implementazione è fornita dal persistence provider. Il seguente codice ci mostra come ottenere l'*entity manager* e rendere una entità *book* persistente:

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("chapter04PU");
EntityManager em = emf.createEntityManager();
em.persist(book);
```

L'*entity manager* ci permette di fare delle **query** sulle entità. Queste query sono simili alle query che si fanno su un database, ma invece di usare *SQL*, viene utilizzato **JPQL**.

Esempio:

Vogliamo che tutti i libri abbiano il titolo *H2G2*, quindi possiamo scrivere:

```
SELECT b FROM Book b WHERE b.title = 'H2G2'
```

Il **title** non rappresenta la colonna della tabella nel database, ma semplicemente il nome dell'attributo di *Book*. **JPQL** manipola oggetti e attributi, NON tabelle e colonne.

Un'istruzione **JPQL** può essere eseguita con:

- **query dinamiche**: create dinamicamente e run-time;
- **query statiche**: definite staticamente a tempo di compilazione. Le query statiche chiamate anche **named queries**, possono essere definite sia con annotazioni **@NamedQuery** che con **XML metadata**.
- **native SQL statement** o **stored procedures**.

Per esempio, la query che abbiamo scritto prima la possiamo scrivere anche come una *named query* sull'entità *Book*:

@Entity

```
@NamedQuery(name = "findBookH2G2", query = "SELECT b FROM Book b WHERE
    b.title ='H2G2'")
```

```
public class Book {
    @Id @GeneratedValue
    private Long id;
    private String title;
    private Float price;
    private String description;
    private String isbn;
    private Integer nbOfPage;
    private Boolean illustrations;

    // Constructors, getters, setters
}
```

Per usare questa query dobbiamo inserirla in un metodo `main`:

1. **Creare una istanza dell'entità Book:** una entità è soltanto una classe annotata, quindi per creare una nuova istanza facciamo come sempre `new`.
2. **Ottenere l'entity manager e una transazione:** un entity manager è necessario per manipolare le entità, quindi andiamo a creare prima un manager factory per l'unità di persistenza "`chapter04PU`". Questa factory ha il compito di ottenere un entity manager.
3. **Rendere il book persistente nel database:** il codice inizia con una transaction `tx.begin()` e usa `persist()` per rendere l'entità persistente. Quando la transazione è committata tramite `tx.commit()` il dato finisce nel database.
4. **Eseguire la named query:** un entity manager è usato per recuperare il libro usando la named query `findBookH2G2`.
5. **Chiudere l'entity manager e l'entity manager factory.**

```
public class Main {  
    public static void main(String[] args) {  
  
        // 1-Creates an instance of book  
        Book book = new Book("H2G2", "The Hitchhiker's Guide to the Galaxy",  
                               12.5F, "1-84023-742-2", 354, false);  
  
        // 2-Obtains an entity manager and a transaction  
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("chapter04PU");  
        EntityManager em = emf.createEntityManager();  
  
        // 3-Persists the book to the database  
        EntityTransaction tx = em.getTransaction();  
        tx.begin();  
        em.persist(book);  
        tx.commit();  
  
        // 4-Executes the named query  
        book = em.createNamedQuery("findBookH2G2", Book.class).getSingleResult();  
  
        // 5-Closes the entity manager and the factory  
        em.close();  
        emf.close();  
    }  
}
```

8.3 Persistence Unit

Quale driver JDBC bisogna utilizzare?

Come dobbiamo connetterci al database?

Come si chiama il database?

Queste informazioni sono contenute all'interno di una **persistence unit** di cui viene passato il nome all'**EntityManagerFactory**. Questa **persistence unit** indica il tipo di database da usare e i parametri di connessione, i quali sono definiti all'interno del file `persistence.xml`.

Le informazioni che è possibile inserire per ogni *PU* sono:

- **nome** (della Persistence Unit);
- **classe** a cui si riferisce (Entità a cui si riferisce);
- **tipo di database** (per scegliere il giusto driver JDBC);
- **posizione** (URL);
- **modalità per autenticazione**.

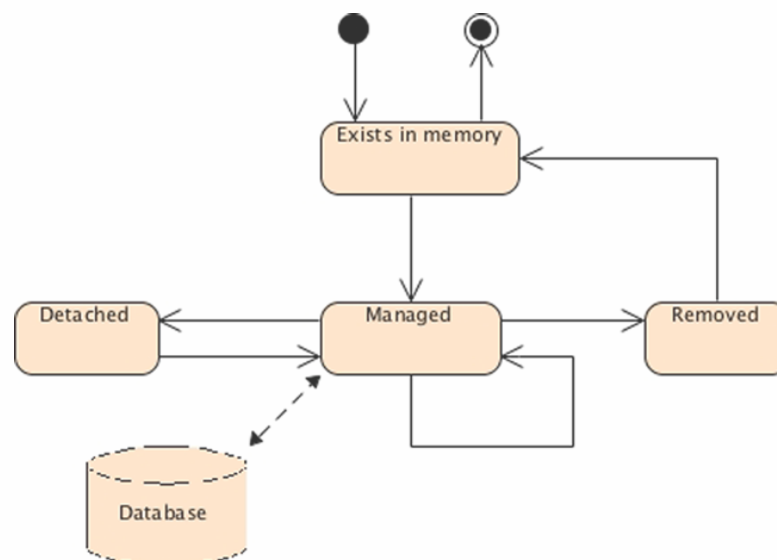
Senza queste specifiche, un *POJO* può essere usato come classe per istanze di oggetti Java tradizionali.

8.4 Ciclo di vita delle entità

Quando gli oggetti sono persistenti essi sono entità e vengono gestiti dall'entity manager che sincronizza il loro stato con il database, mentre quando gli oggetti rimangono *POJO*, vengono trattati come semplici oggetti *Java*. Questo significa che le entità hanno un ciclo di vita.

Quando creiamo una istanza di `Book`, con la keyword `new`, l'oggetto esiste in memoria e *JPA* non sa niente di questa istanza, e quindi il suo ciclo di vita è legato a quello del programma. Quando invece l'entità diventa **managed** dall'entity manager essa viene sincronizzata con il database e quindi il suo stato diventa **persistente**. Chiamando la `remove()`, eliminiamo i dati dal database, ma l'oggetto Java continua a vivere in memoria finché non interviene il garbage collector.

Le operazioni che è possibile eseguire sulle entità rientrano in 4 categorie: **persisting**, **updating**, **removing** e **loading**. Che corrispondono alle operazioni di: **inserting**, **updating**, **deleting** e **selecting** su un database.



Ogni operazione fatta su una entità ha un evento **"pre"** e **"post"** che può essere intercettato dall'entity manager per invocare dei metodi di business. Quindi, abbiamo delle annotazioni **@PrePersist** e **@PostPersist** che possono essere settate sui metodi dell'entità (ovvero metodi di callback) o in classi esterne (ovvero listeners).

8.5 JPA: componenti

Le principali componenti di *JPA* sono:

- **Object-Relational Mapping (ORM)**: meccanismo che permette di mappare oggetti in dati memorizzati in un database.
- **Entity manager API**: per eseguire operazioni *CRUD* su database relazionali.
- **Java Persistence Query Language (JPQL)**: permette di recuperare dati con un object-oriented query language.
- **Transaction e Locking mechanisms** che *Java Transaction API (JTA)* fornisce per gestire l'accesso concorrente ai dati.
- **Callbacks e listeners**: per agganciare la logica di business nel ciclo di vita di un oggetto persistente.

8.5.1 JPA: Reference Implementation

EclipseLink 2.5 è la **reference implementation** open source di *JPA 2.1*: fornisce un framework potente e flessibile per memorizzare oggetti Java in un database relazionale.



Capitolo 9 – ORM (Object-Relational Mapping)

9.1 Mapping di relazioni

Ci possono essere diverse **associazioni** tra le classi. Le associazioni collegano oggetti che possono essere anche di tipo diverso. Le associazioni prima di tutto hanno una direzione e possono essere:

- **Unidirezionali**: da un oggetto possiamo navigare verso un altro oggetto;
- **Bidirezionali**: da un oggetto possiamo navigare verso un altro oggetto e viceversa.

In *UML* ci sono notazioni per esprimere queste associazioni, compreso l'**associazione con molteplicità** (o cardinalità); in questo caso un oggetto della classe di partenza può riferire più oggetti della classe di destinazione:

1. Associazione unidirezionale fra due classi



2. Associazione bidirezionale fra due classi:

Class1 ha un attributo del tipo Class2 e
Class2 ha un attributo del tipo Class1.



3. Associazione con molteplicità: Class1 si riferisce a 0 o più istanze di Class2.

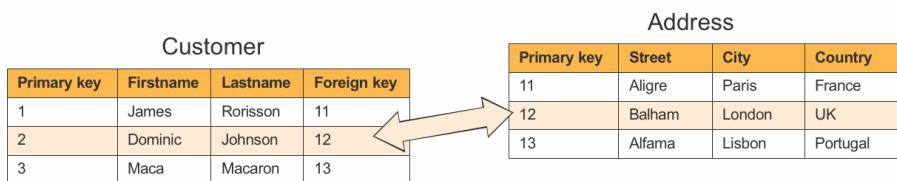


9.2 Relazioni in Database Relazionali

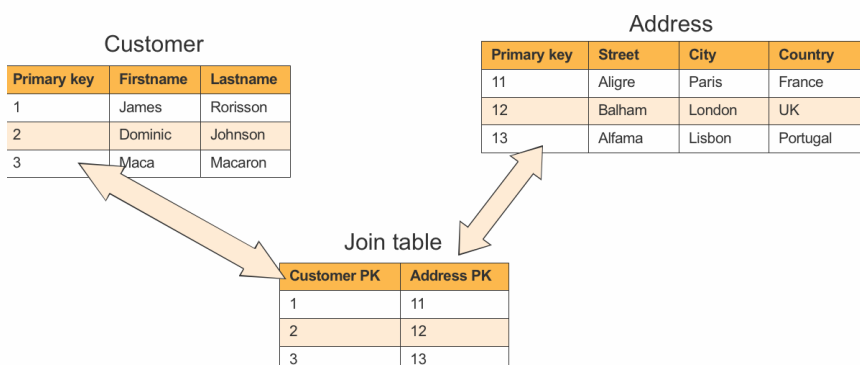
Un **database relazionale** è una collezione di relazioni, dette anche **tabelle** (ogni cosa viene modellata come una tabella). Per modellare una associazione si usano le tabelle.

In *JPA* quando si ha un'associazione tra una classe ed un'altra, nel database avremo un **riferimento in tabella** (*table reference*). Questo riferimento può essere modellato in due modi:

- usando una **chiave esterna** (**foreign key**);
- usando una **tabella di join** (**join table**).



→ foreign key



→ join table

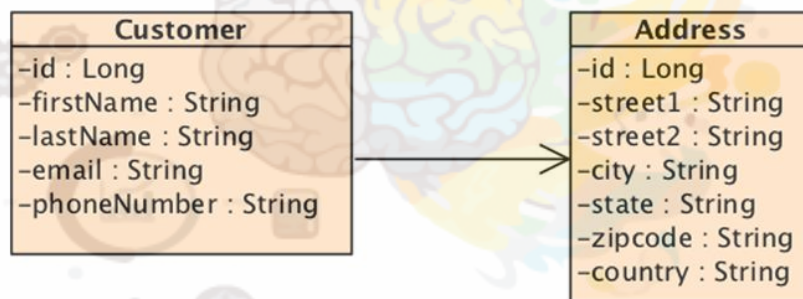
JPA rende possibile mappare associazioni in modo che un'entità può essere linkata ad un'altra in un modello relazionale usando una **configuration-by-exception** per le associazioni: esso ha un modello di default per memorizzare una relazione, ma è possibile personalizzarla tramite annotazioni. La cardinalità tra due entità può essere:

- **uno-ad-uno**, @OneToOne;
- **uno-a-molti**, @OneToMany;
- **molti-a-uno**, @ManyToOne;
- **molti-a-molti**, @ManyToMany.

Ogni annotazione può essere usata unidirezionalmente o bidirezionalmente.

Relazione Unidirezionale

In una **relazione unidirezionale**, una entità Customer ha un attributo di tipo Address. Essendo la *relazione unidirezionale*, Customer è detto il **proprietario della relazione** (ownership). In termini di database, questo significa che la tabella CUSTOMER avrà una **chiave esterna** che punta ad ADDRESS, e quando siamo i proprietari di una relazione, siamo in grado di personalizzare il mapping di questa relazione.



@Entity

```
public class Customer {
    @Id
    @GeneratedValue
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
    private String phoneNumber;
    private Address address;

    public Customer() {}

    // Constructors, getters, setters
    ...
}
```

@Entity

```
public class Address {
    @Id
    @GeneratedValue
    private Long id;
    private String street1;
    private String street2;
    private String city;
    private String state;
    private String zipcode;
    private String country;

    public Address() {}

    // Constructors, getters, setters
    ...
}
```

Per cambiare il nome di una tabella (esempio: Book) si inserisce dopo @Entity:

```
@Table(name = "t_book").
```

Relazione Bidirezionale

Una **relazione bidirezionale**, in Java viene trattata come due relazioni unidirezionali: l'entità Customer ha un riferimento di Address e Address ha un riferimento a Customer.

Per capire chi è l'owner bisogna dirlo esplicitamente tramite l'elemento **mappedBy** delle annotazioni @OneToOne, @OneToMany e @ManyToMany.

Codice:

```
@Entity
public class Customer {

    @Id @GeneratedValue
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
    private String phoneNumber;

    @OneToOne
    @JoinColumn(name = "address_fk")
    private Address address;
}

@Entity
public class Address {

    @Id @GeneratedValue
    private Long id;
    private String street1;
    private String street2;
    private String city;
    private String state;
    private String zipcode;
    private String country;

    @OneToOne(mappedBy = "address")
    private Customer customer;
}
```

- **Customer** ha un attributo **Address** annotato con @OneToOne.
- L'entità **Address** ha un attributo **Customer** annotato con @OneToOne il quale usa l'elemento **mappedBy** sulla sua annotazione (inverse owner).

L'elemento mappedBy indica che la join column (address) è specificata dall'altro lato della relazione. Infatti, nell'altro lato, l'entità **Customer** definisce la join Column con l'annotazione @JoinColumn e rinomina la *foreign key* in address_fk.

Capitolo 10 – Managing Persistent Objects

10.1 Entity Manager

L'**Entity Manager** è il punto centrale di *JPA*. I suoi compiti sono:

- gestire stato e ciclo di vita delle entità;
- fare query di entità all'interno di un *persistence context*;
- proteggere da accessi concorrenti utilizzando tecniche di *locking*.

Quando un *Entity Manager* ottiene un riferimento ad un'entità viene detto **managed**: fino a quel momento l'entità è vista come un regolare *POJO* (*detached*).

La potenza di *JPA* è che le entità possono essere usate come oggetti regolari da differenti layer di un'applicazione e diventare "**managed**" dall'entity manager quando bisogna caricare o inserire dati in un database.

- Quando l'oggetto è "**managed**" si possono eseguire operazioni di persistenza, e l'entity manager automaticamente sincronizzerà lo stato dell'entity con il database.
- Quando l'entity è "**detached**" ritorna ad essere un regolare *POJO* e quindi può essere usato da altri layers senza sincronizzare il suo stato con il database.

10.1.1 Come si ottiene un Entity Manager

L'*entity manager* rappresenta l'interfaccia principale per interagire con le entità, ma prima deve essere ottenuta dall'applicazione. Il codice può cambiare a seconda che l'ambiente sia:

- **Application Managed**: l'applicazione è responsabile per l'istanza specifica di *Entity Manager* e per gestirne il ciclo di vita.
- **Container Managed**: quando l'applicazione è una *Servlet* o un *Enterprise Java Bean* e quindi ci si affida a risorse iniettate. In questo caso le transazioni sono gestite dal container: non si devono scrivere *commit* o *rollback* che sono necessari per un *application-managed enviroment*.

10.2 Persistence Context

Il **persistence context** è un insieme di istanze di entità gestite in un certo tempo per una certa transazione dell'utente. Inoltre, solo un'entità con lo stesso *ID* può esistere in un *persistence context*.

L'*Entity Manager* aggiorna o consulta il *persistence context* quando viene chiamato un metodo dell'interfaccia `javax.persistence.EntityManager`.

Ogni utente ha il suo *persistence context*:

- che ha vita breve, visto che rappresenta la durata di una transazione;
- una sorta di first-level cache;
- è uno spazio dove l'entity manager memorizza entità prima della scrittura (flushing) dei contenuti del database.

Rendere persistente un'Entità

Rendere persistente una entità vuol dire inserire dei dati in un database, quando il quest'ultimo non è già presente nel database (altrimenti si verifica un'eccezione). Per fare questo dobbiamo:

- creare una nuova istanza di una entità usando l'operatore `new`;
- settare gli attributi con dei valori;
- associare l'entità con un'altra nel caso ci sia una associazione;
- chiamare `EntityManager.persist()`.

Codice:

```
Customer customer = new Customer("Antony", "Balla", "tballa@mail.com");
Address address = new Address("Ritherdon Rd", "London", "8QE", "UK");
```

```
customer.setAddress(address);
```

```
tx.begin();
em.persist(customer);
em.persist(address);
tx.commit();
```

```
assertNotNull(customer.getId());
assertNotNull(address.getId());
```

quando la transazione è committata, con `tx.commit()` viene fatto il *flush* dei dati nel database e viene inserita una riga nella tabella ADDRESS e una riga nella tabella CUSTOMER.

Nel database, la *insert* viene fatta rispettando il vincolo di **integrità referenziale**. Quindi dato che customer è il proprietario della relazione, esso custodisce la chiave esterna per address, e quindi viene fatta prima la *insert* di address.

Trovare un'Entità (Finding by ID)

Per cercare una entity tramite il suo identificativo possiamo usare `EntityManager.find()` il quale ha due parametri: la **classe dell'entità** e l'**id univoco**. Se l'entità viene trovata, è ritornata. Altrimenti viene ritornato *null*.

Codice:

```
// Per ID
Customer customer = em.find(Customer.class, 1234L);

if (customer != null) {
    // Process the object
}
```

Un altro metodo è `getReference()`. Questo metodo è simile a `find` eccetto che esso ritorna il riferimento di una entità e non i suoi dati. Possiamo pensare a questo metodo come un **proxy di una entità**, ovvero non otteniamo l'entity stessa.

Codice:

```
// Per riferimento
try {
    Customer customer = em.getReference(Customer.class, 1234L);
    ...
} catch (EntityNotFoundException ex) {
    // Entity not found
}
```

Rimozione di un'Entità

Una entity può essere rimossa con il metodo `EntityManager.remove()`. Una volta rimossa, l'entità è cancellata dal database, ne viene fatto il detach dall'*entity manager* e non può essere sincronizzata con il database. In termini di Java, l'entità continua ad esistere nella JVM finché non ne viene fatto la garbage collection.

Codice:

```
Customer customer = new Customer("Antony", "Balla", "tballa@mail.com");
Address address = new Address("Ritherdon Rd", "London", "8QE", "UK");
customer.setAddress(address);

tx.begin();
em.persist(customer);
em.persist(address);
tx.commit();

tx.begin();
em.remove(customer);
tx.commit();

// The data is removed from the database but the object is still accessible
assertNotNull(customer);
```

1. In questo codice si creano due oggetti e si linkano insieme.
2. In una **transazione** gli oggetti vengono resi persistenti e viene fatto il commit.
3. In un'altra **transazione** viene cancellato il cliente e a seconda della politica di **cascading** l'indirizzo può essere o meno cancellato. Il *POJO*, invece, continua a esistere sempre.

Cascading: rimozione degli orfani

Per la consistenza dei dati, gli **orfani** non sono desiderabili, poiché si troveranno in righe di un database che non sono referenziate da nessuna tabella.

JPA supporta la rimozione automatica degli orfani, infatti, si definisce la **relazione** in modo che alla cancellazione del cliente (Customer) viene cancellato l'indirizzo (Address).

Le associazioni che sono specificate come **uno-ad-uno** oppure **uno-a-molti**, supportano l'uso di questa opzione. Per includere questa opzione dobbiamo aggiungere **orphan = true** alla notazione `@OneToOne`.

Codice:

```
@Entity
public class Customer {
    @Id @GeneratedValue
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
    @OneToOne(fetch = FetchType.LAZY, orphanRemoval = true)
    private Address address;
}
```

Con questo mapping, il codice rimuoverà automaticamente l'entità Address quando il Customer è rimosso o quando la relazione è interrotta.

10.3 Sincronizzazione con il DB: flush e refresh

Non si è certi quando il provider sincronizzerà i dati nel database, ma la cosa certa è che avviene dopo aver fatto il `commit()`. Per sincronizzare il database possiamo usare due metodi:

- **`EntityManager.flush()`**: con questo metodo il *persistence provider* può forzare il flush dei dati esplicitamente nel database ma non fare il `commit` della transazione. Poiché il *flush* non aspetta il `commit`, il codice non funzionerà, perché forzando la *insert* di un'entity violiamo il vincolo di **integrità referenziale** in quanto tale entity avrà una chiave esterna che non punterà a nessun indirizzo.

Codice:

```
tx.begin();
em.persist(customer);
em.persist(address);
tx.commit();
```

Codice funzionante (senza flush)

```
tx.begin();
em.persist(customer);
em.flush();
em.persist(address);
tx.commit();
```

Codice non funzionante (con flush)

- **`EntityManager.refresh()`**: questo metodo è usato per sincronizzare i dati nella direzione opposta del *flush*, ovvero, sovrascrive lo stato corrente della *entity managed* con i dati che sono presenti nel database. Un esempio in cui è usato questo metodo è quando vogliamo riportare la nostra entity in un certo stato contenuto nel database.

Codice:

```
Customer customer = em.find(Customer.class, 1234L);
assertEquals(customer.getFirstName(), "Antony");
customer.setFirstName("William");
em.refresh(customer);
assertEquals(customer.getFirstName(), "Antony");
```

1. In memoria abbiamo il nome "Antony";
2. Settiamo il nuovo nome;
3. Poiché abbiamo fatto la *refresh* abbiamo Antony.

10.4 Interazione con il Persistence Context: Contains, Detach, Context, Merge, Update

Il *persistence context* mantiene le *entity managed*. Con `EntityManager` possiamo controllare che una entità sia attualmente managed, farne il detach, o fare il clear di tutte le entity dal *persistence context*.

- Il metodo **`EntityManager.contains()`** ritorna un booleano e ci permette di controllare se una istanza è *managed* o meno.
- Il metodo **`clear()`** azzerà il *persistence context*, e tutte le entità *managed* diventano **detached**.
- Il metodo **`detach(Object entity)`** rimuove un'entità dal *persistence context*.
- Il metodo **`merge(Object entity)`** gestisce un'entità di cui è stato fatto *detach* e non è più associata ad un *persistence context*.

10.5 Cascading Events

Di default ogni operazione dell'entity manager si applica ad una sola entità passata come argomento dell'operazione, ma a volte vogliamo che gli effetti dovuti ad una azione su questa entità si propaghino in cascata a tutte le sue associazioni. Questa operazione è nota come un **cascading an event**.

Codice:

```
Customer customer = new Customer("Antony", "Balla", "tballa@mail.com");
Address address = new Address("Ritherdon Rd", "London", "8QE", "UK");
customer.setAddress(address);

tx.begin();
em.persist(customer);
em.persist(address);
tx.commit();
```

10.6 Queries

JSQL permette di interrogare entità persistenti indipendentemente dal database utilizzato. E' simile alla sintassi di *SQL*, con la differenza che *JSQL* restituisce non tabelle (con righe e colonne) ma un'entità o una collezione di entità e anche *POJO* facili da gestire. *JSQL* traduce la query *SQL* usando *JDBC* per collegarsi.

JPA ha 5 tipi differenti di query ognuna con uno scopo differente:

1. **Dynamic queries**: è la più semplice delle query, consiste in una query *JSQL* contenuta in una stringa creata dinamicamente a runtime ed è costosa in termini di prestazioni.
2. **Named queries**: query statiche e non modificabili.
3. **Criteria API**: un nuovo tipo di query orientata agli oggetti (introdotta da *JPA 2.0*).
4. **Native queries**: questo tipo di query è utile per eseguire query in *SQL nativo* invece di *JSQL*.
5. **Stored procedure queries**: introdotte con *JPA 2.1*.

Tramite i metodi dell'interfaccia di *Entity Manager* si ottiene una query di un certo tipo dalla quali si vanno a prelevare il risultato (o i risultati). I metodi ritornano una variabile interfaccia di tipo:

- **Query**: quando il risultato è di tipo *Object*.
- **TypedQuery**: quando è preferito un certo tipo come risultato.
- **StoredProcedureQuery**: usata per controllare l'esecuzione dell'esecuzione di una stored procedure.

1. Dinamic queries

- La query restituisce una lista **senza tipo** con il metodo `getResultList()`. Se però è noto che il risultato è una singola entità allora bisogna usare il metodo `getSingleResultList()`.

```
Query query = em.createQuery("SELECT c FROM Customer c");
List<Customer> customers = query.getResultList();
```

- Per restituire una lista con un **tipo specifico** allora utilizzeremo una *TypedQuery*.

```
TypedQuery<Customer> query =
    em.createQuery("SELECT c FROM Customer c", Customer.class);
List<Customer> customers = query.getResultList();
```

Una query puo' anche essere costruita dinamicamente dall'applicazione:

```
String jpqlQuery = "SELECT c FROM Customer c";

if (someCriteria)
    jpqlQuery += "WHERE c.firstName = 'Betty'";

query = em.createQuery(jpqlQuery);

List<Customer> customers = query.getResultList();
```

Un'alternativa per migliorare le query in modo da renderle più pulite e quindi da far rintracciare gli errori facilmente è di utilizzare i **parametri**.

//query 1

```
query = em.createQuery("SELECT c FROM Customer c where c.firstName = :fname");
query.setParameter("fname", "Betty");
List<Customer> customers = query.getResultList();
```

//query 2

```
query = em.createQuery("SELECT c FROM Customer c where c.firstName = ?1");
query.setParameter(1, "Betty");
List<Customer> customers = query.getResultList();
```

Per effettuare una query parziali in cui vengono restituiti solo un certo numero di risultati useremo:

```
query = em.createQuery("SELECT c FROM Customer c", Customer.class);
query.setMaxResults(10);
List<Customer> customers = query.getResultList();
```

2. Named queries

Le **Named queries** sono meno flessibili ma più performanti delle *query dinamiche* perché il persistence provider può tradurre la stringa JPQL in SQL una sola volta quando l'applicazione parte, e non ogni volta che la query deve essere eseguita. Viene utilizzata la notazione **@NamedQuery**.

Codice:

@Entity

```
@NamedQueries({
    @NamedQuery(name = "findAll", query = "select c from Customer c"),
    @NamedQuery(name = "findVincent", query = "select c from Customer c where c.firstName = 'Vincent'"),
    @NamedQuery(name = "findWithParam", query = "select c from Customer c where c.firstName = :fname")
})
```

```
public class Customer { @Id @GeneratedValue
    private Long id;
    private String firstName;
    private String lastName;
    private Integer age;
    private String email;
    @OneToOne
    @JoinColumn(name = "address_fk")
    private Address address;
}
```

- **Entità** con query statiche (efficienti);
- **Query** che selezione tutti i customer dal DB;
- **Query** per un certo utente;
- **Query** con parametro.

Per creare una *Named Query* con **parametri**:

- si crea una query dall'*EM*; `Query query = em.createNamedQuery("findWithParam");`
- si setta un parametro; `query.setParameter("fname", "Vincent");`
- si definisce il massimo numero di risultati; `query.setMaxResults(3);`
`List<Customer> customers = query.getResultList();`
- si esegue.

Bisogna fare attenzione perché il nome delle query ha scope relativo al *persistence unit* e deve essere univoco all'interno di questo scope. Inoltre, errori di query sono riconosciuti run-time essendo la query una stringa: perdiamo la *type safety*.

3. Criteria API (or Object-Oriented Queries)

Usando le stringhe per creare delle query *JPQL* potremmo causare vari errori sintattici di cui ce ne accorgeremmo soltanto a runtime. Con le **Criteria API**, invece, possiamo scrivere qualsiasi query in maniera sintatticamente corretta e orientata agli oggetti, dato che molti degli errori che potrebbe fare uno sviluppatore nello scrivere una query sono rilevati a tempo di compilazione e non a tempo di esecuzione.

Per esempio, vediamo una query che prende tutti i customer che si chiamano "Vincent".

- In *JPQL* sarebbe: `SELECT c FROM Customer c WHERE c.firstName = "Vincent"`

- Mentre con **Criteria API** sarebbe:

```
CriteriaBuilder builder = em.getCriteriaBuilder();
CriteriaQuery<Customer> criteriaQuery = builder.createQuery(Customer.class);
Root<Customer> c = criteriaQuery.from(Customer.class);
criteriaQuery.select(c).where(builder.equal(c.get("firstName"), "Vincent"));
Query query = em.createQuery(criteriaQuery).getResultList();
List<Customer> customers = query.getResultList();
```

Da come si vede `select`, `where`, `from` sono rappresentate tramite dei metodi dell'API: `select()`, `where()`, `from()`. Lo stesso vale anche per tutte le altre keyword in *JPQL*.

4. Native Queries

JPA ci permette di eseguire *SQL nativo* invece di *JPQL* usando delle **query native**. Queste query prendono come parametro una **native SQL statement** (`SELECT`, `UPDATE` o `DELETE`) e ritornano una istanza di tipo **Query**. Queste query non sono portabili tra i database.

```
Query query = em.createNativeQuery("SELECT * FROM t_customer", Customer.class);
List<Customer> customers = query.getResultList();
```

Come le *named queried*, le *native queries* possono usare le annotazioni per definire query *SQL* statiche.

Le **named native queries** sono definite usando l'annotazione **@NamedNativeQuery** (posizionata sull'entità). Il nome della query deve essere unico all'interno del *persistence unit*.

```
@Entity
@NamedNativeQuery(name = "findAll", query = "select * from t_customer")
@Table(name = "t_customer")
public class Customer {...}
```

5. Stored Procedure Queries

Le **stored procedure queries** sono memorizzate nel database stesso e vengono eseguite all'interno del database. È una subroutine disponibile per le applicazioni che hanno accesso ad un database relazionale. Un uso tipico potrebbe essere relativo ad azioni complesse che richiedono l'esecuzione di *diversi statement SQL*, azioni che magari sono anche ripetitive.

Questo porta i seguenti vantaggi:

- migliori prestazioni per la precompilazione;
- riduce l'ammontare dei dati che viaggiano su una rete, mantenendo il codice sul server;
- possono essere usate anche da linguaggi diversi da *Java*;
- trasparenza dei dati, favorendo l'accesso solo alle *stored procedures*.

Codice (Procedura in SQL):

```
CREATE PROCEDURE sp_archive_books @archiveDate DATE, @warehouseCode
    VARCHAR AS
    UPDATE T_Inventory
    SET Number_Of_Books_Left - 1
    WHERE Archive_Date < @archiveDate AND Warehouse_Code = @warehouseCode;

    UPDATE T_Transport
    SET Warehouse_To_Take_Books_From = @warehouseCode;
END
```

Questa stored procedure può essere invocata attraverso il suo nome `sp_archive_books`.

È possibile invocare una *stored procedure* con annotazioni (`@NamedStoredProcedureQuery`) o dinamicamente. L'annotazione `NamedStoredProcedureQuery` specifica:

- il nome dello *stored procedure* da invocare;
- i tipi di tutti i parametri (`Date.class` e `String.class`);
- i loro corrispondenti parameter modes (IN, OUT, INOUT, etc...).

Codice (Entity che dichiara la stored procedure):

```
@Entity
@NamedStoredProcedureQuery(name = "archiveOldBooks",
    procedureName = "sp_archive_books",
    parameters = {
        @StoredProcedureParameter(name = "archiveDate", mode = IN, type = Date.class),
        @StoredProcedureParameter(name = "warehouse", mode = IN, type = String.class)
    })
public class Book {
    @Id @GeneratedValue
    private Long id;
    private String title;
    //..etc..etc..
}
```

Per invocare la *store procedure* `sp_archive_books` è necessario:

- usare l'*entity manager*;
- creare una named stored procedure query passando il suo nome (`archiveOldBooks`);
- questo restituisce una `StoredProcedureQuery query` sulla quale settare i parametri ed eseguire.

Codice:

```
StoredProcedureQuery query = em.createStoredProcedureQuery("archiveOldBooks");
query.setParameter("archiveDate", new Date());
query.setParameter("maxBookArchived", 1000);
query.execute();
```

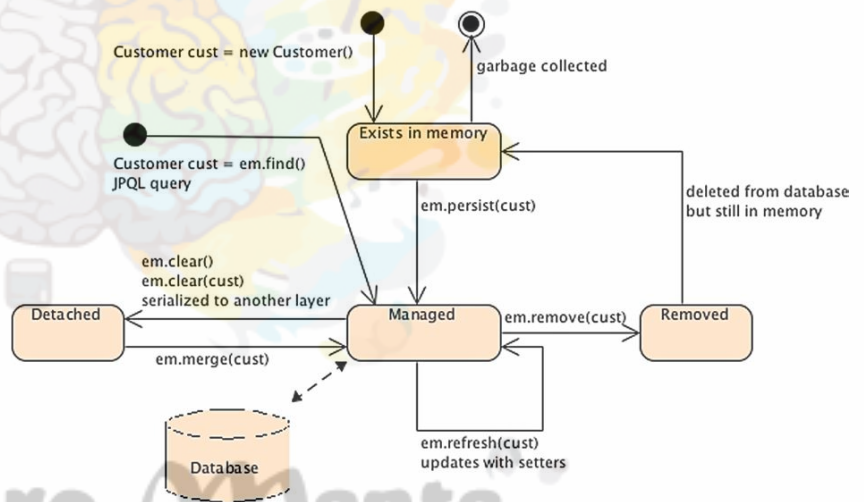
10.7 Ciclo di Vita di un Entity

Quando una entità è istanziata (con la keyword `new`) è vista come un semplice *POJO* dalla *JVM* e può essere usata come un oggetto regolare dall'applicazione; se l'oggetto non viene usato, verrà liberato dal garbage collector ed il ciclo di vita termina.

Invece, quando viene invocato il metodo `EntityManager.persist()`, l'entità è resa persistente e diventa **managed**; quindi, il suo stato sarà sincronizzato con il database.

In questa fase detta **managed state**, è possibile fare delle operazioni con questa entità che saranno riflesse nel database.

Nel *managed state* è possibile invocare il metodo `EntityManager.remove()` e l'entità verrà cancellata dal database: l'entità non sarà più gestita, ma l'oggetto Java continua a risiedere in memoria fin quando non interverrà il *garbage collector*.

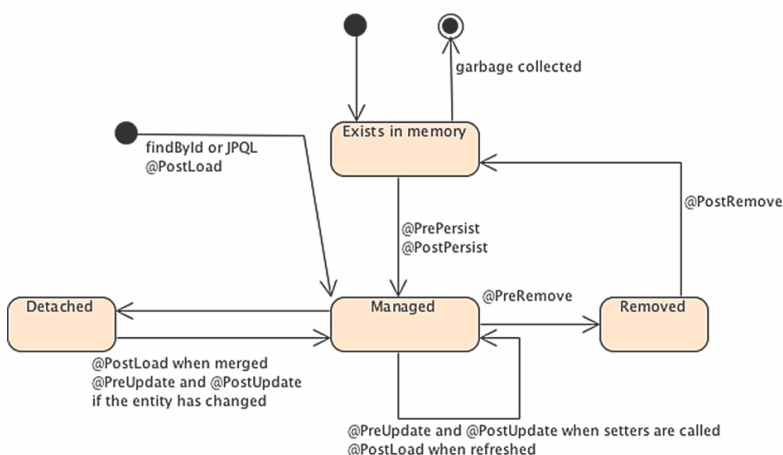


10.7.1 Callback

Il *ciclo di vita* di una entità ricade in una di queste 4 categorie:

- **Persisting;**
- **Updating;**
- **Removing;**
- **Loading.**

Che corrispondono alle operazioni del database di **inserimento**, **aggiornamento**, **rimozione** e **selezione**. Ogni ciclo ha un evento **precedente** "*pre*" ed uno quando l'operazione si è **conclusa** "*post*", che possono essere intercettati dall'entity manager per invocare un metodo di business.



Prima di inserire una entità nel database, l'entity manager chiama il metodo annotato con `@PrePersist`.

Se l'inserimento va a buon fine l'entità è resa persistente e la sua identità è inizializzata e il metodo annotato con `@PostPersist` viene invocato.

Lo stesso discorso vale per *update* e *delete*.

10.8 Listeners

I metodi di *callback* in una entità funzionano bene quando abbiamo della logica di business che è legata solo a quell'entità. I **listeners**, invece, sono una generalizzazione di *callback* in quanto sono usati per estrapolare la logica di business in una classe separata e condividerla tra altre entità. Un **entity listener** è semplicemente un *POJO* su cui possiamo definire uno o più metodi di *callback* legati al ciclo di vita. Per registrare un *listener*, la entity necessita dell'annotazione **@EntityListeners**.

Abbiamo alcune regole che i *listener* devono rispettare:

- costruttore pubblico senza argomenti;
- i metodi di *callback* devono avere un parametro del tipo dell'entità (che viene passato automaticamente);
- se ha parametro *Object* può essere chiamato su diverse entità altrimenti ha il tipo specificato.





EJB

(Enterprise Java Beans)

Capitolo 11 – Enterprise Java Beans

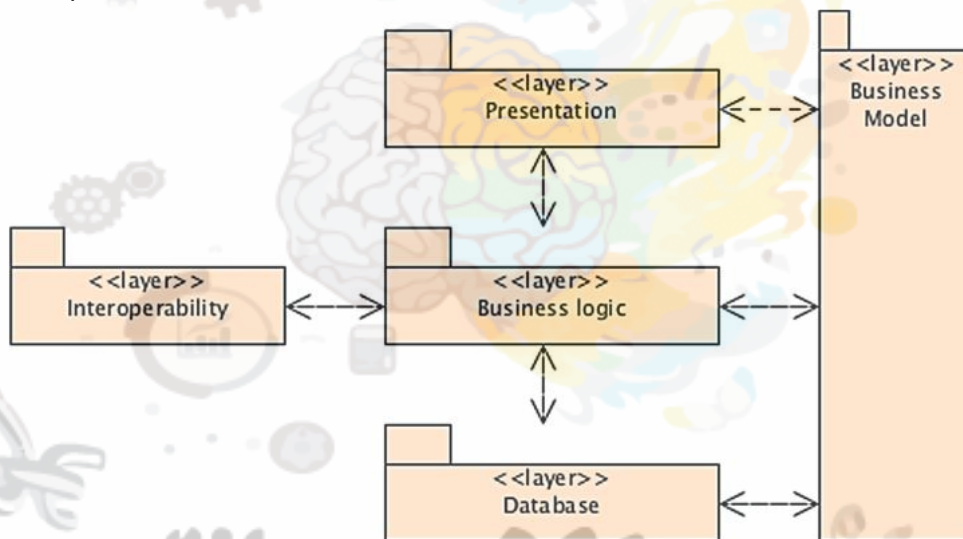
11.1 Introduzione EJB

Le entità non sono fatte per rappresentare dei task complessi. Infatti, il *persistance layer* e il *presentation layer* non sono adatti per implementare la **logica di business**. Per questo motivo abbiamo bisogno di un **business layer**: in *Java EE* implementiamo questo layer usando i **Java Beans Enterprise (EJB)**.

Seguendo un approccio *bottom-up*, la parte relativa a *JPA* modella il **dominio delle classi**, definendo gli attori del sistema (Artista, CD, Libro, Customer, etc.).

Invece, sopra questo *domain layer*, c'è il *business layer* che modella invece le azioni dell'applicazione. Il compito del *business layer* è di:

- interagire con i servizi web esterni;
- inviare messaggi asincroni ad altri sistemi usando *JMS*;
- servire il layer di presentazione.



Gli **EJB** sono delle **componenti lato server** che incapsulano la **logica di business** e si prendono cura delle transazioni e della sicurezza, apportano riusabilità e scalabilità alle applicazioni enterprise. Si integrano con qualsiasi altra tecnologia *Java SE* e *Java EE*, ed è per questo che vengono usati per costruire il livello relativo alla *logica di business*.

Tutto questo lo otteniamo semplicemente annotando un *POJO* che verrà deployato in un **EJB container**. Un **EJB container** è un ambiente di esecuzione che fornisce servizi, come:

- Gestione delle transazioni;
- Controllo della concorrenza;
- Pooling;
- Autorizzazioni per la sicurezza.

In questo modo gli sviluppatori di *EJB* possono occuparsi soltanto della logica di business mentre il container si occupa di tutto il resto.

11.2 Servizi forniti dal container

Non importa se il container è integrato o se gira in un processo separato, esso fornisce servizi basilari comuni a molte applicazioni enterprise come:

- **Remote client communication**: un *EJB client* può invocare metodi da remoto attraverso protocolli standard.

- **Dependency injection:** il container può iniettare delle risorse in un *EJB*, così come in un *POJO* grazie al *CDI*.
- **State management:** per i *beans stateful*, il container gestisce il loro stato in modo trasparente.
- **Pooling:** per i *beans stateless*, il container crea un pool di istanze che può essere condiviso tra client diversi. Una volta invocato, un *EJB* ritorna al pool per essere riusato invece che distrutto.
- **Component life-cycle:** il container è responsabile per la gestione del ciclo di vita di ciascun componente.
- **Messaggistica.**
- **Gestione delle transazioni:** mediante delle annotazioni l'*EJB* può informare il container riguardo le politiche di transazione che deve usare. Il container si prende carico della commit o del rollback.
- **Sicurezza:** controlli di accesso a livello di classe o di metodi possono essere specificati su un *EJB*.
- **Supporto della concorrenza:** tutti i *EJB* sono **thread safe** per natura tranne per i singleton, dove delle dichiarazioni sulla concorrenza sono specificate.
- **Interceptors:** problematiche trasversali possono essere dentro gli *interceptors*, che possono essere invocati automaticamente dal container.
- **Invocazione asincrona di metodi.**

Gli *EJB* sono dei **managed beans**. Quando un client invoca un *EJB*, questo non lavora direttamente su una istanza di un *EJB* bensì invoca un **proxy** su di esso. Ogni volta che un client invoca un metodo su un *EJB*, la chiamata è filtrata da questo *proxy* e intercettata dal container, che fornisce servizi per conto dell'istanza del bean. Questo meccanismo è trasparente al client.

In una applicazione *Java EE*, l'*EJB container* interagisce di solito con altri container:

- *Servlet container*;
- *Application client container*;
- *Message broker*;
- Etc...

11.3 Tipi di *EJB*

Un *session bean* può avere diversi stati:

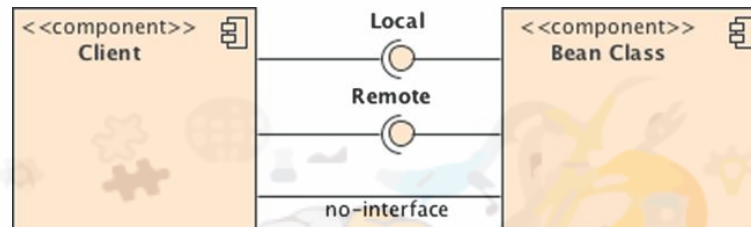
- **Stateless:** il *session bean* non conserva nessuno stato sulle conversazioni avvenute tra metodi e ogni istanza può essere usata per qualsiasi client. È usato per gestire task che possono concludersi all'interno di una singola chiamata di un metodo.
- **Stateful:** il *session bean* conserva uno stato sulle conversazioni avvenute, che deve essere conservato tra diversi metodi per un singolo utente. È utile per task che necessitano di eseguire diversi step per arrivare al completamento.
- **Singleton:** un singolo *session bean* è condiviso tra i client e supporta accessi concorrenti. Il container farà in modo che soltanto una istanza esista per l'intera applicazione.

I *session bean* sono delle componenti gestite dal container e quindi devono essere impacchettate in un archivio (*jar*, *war*, o *ear*) e deployato nel container.

11.4 Anatomia di un EJB

Un *EJB* è composto dai seguenti elementi:

- Una **classe bean**: contiene l'implementazione dei metodi di business e può implementare o meno diverse interfacce di business. Il *session bean* deve essere annotato con `@Stateless`, `@Stateful` o `@Singleton` in base al tipo.
- **Interfaccia di Business**: queste interfacce contengono la dichiarazione dei metodi di business che sono visibili al client e implementati dalla classe bean. Un session bean può avere delle **interfacce locali**, **interfacce remote** o **nessuna interfaccia**.



Session Bean

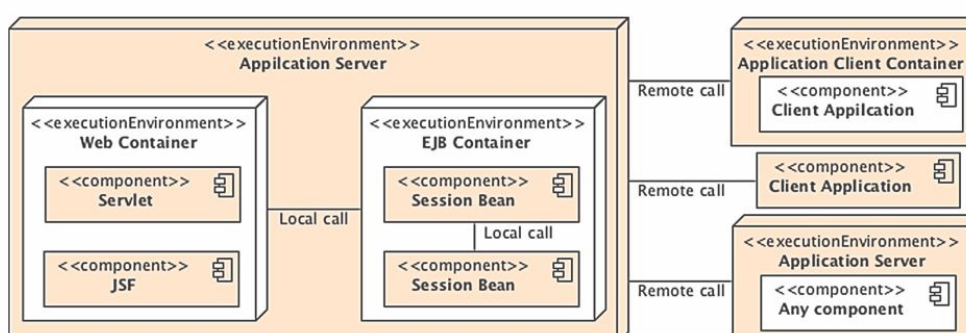
Una classe **session bean** è una classe *Java* standard che implementa della logica di business. I requisiti per sviluppare un *session bean* sono i seguenti:

- La classe deve essere annotata con `@Stateless`, `@Stateful`, `@Singleton` o un deployment descriptor equivalente in *XML*.
- Deve implementare i metodi delle sue interfacce (se esistono).
- La classe deve essere **public**.
- Ci deve essere un costruttore senza argomenti che il container userà per creare le istanze.
- La classe non deve definire il metodo `finalize()`.
- I nomi dei metodi di business non devono iniziare con *ejb* e non possono essere `final` o `static`.
- I valori di ritorno e gli argomenti di un metodo remoto devono essere dei tipi *RMI* validi.

11.5 Remote, Local e No-Interface Views

A seconda di dove un client invoca un *session bean*, la classe bean potrà implementare un'interfaccia:

- **Remote view**: se il client risiede al di fuori del container *EJB*. In questo caso i client dovranno invocare i metodi del *session bean* tramite *RMI*.
- **Local view**: se il bean e il client sono in esecuzione nella stessa *JVM*. Inoltre, è possibile usare sia chiamate locali che remote sullo stesso *session bean*.
- **No-interface view**: è una vista senza interfaccia simile alla *local interface*. Espone tutti i metodi pubblici di business della classe bean localmente senza l'utilizzo di un'interfaccia separata.



Una **interfaccia di business** è una interfaccia standard che non estende nessuna interfaccia. Come per ogni interfaccia *Java*, essa definisce una serie di metodi che saranno disponibili per l'applicazione client.

Queste interfacce devono usare le annotazioni seguenti:

- **@Remote**: denota una **interfaccia di business remota**. I parametri dei metodi sono passati per valore e necessitano di essere serializzati essendo parte del protocollo *RMI*.
- **@Local**: denota una **interfaccia di business locale**. I parametri dei metodi sono passati per riferimento dal client al bean.

Non è possibile usare più di una annotazione per la stessa interfaccia.

Interfacce EJB

1. Annotazione per **interfaccia locale**

```
@Local
public interface ItemLocal {
    List<Book> findBooks();
    List<CD> findCDs();
}
```

2. Annotazione per **interfaccia remota**

```
@Remote
public interface ItemRemote {
    List<Book> findBooks();
    List<CD> findCDs();
    Book createBook(Book book);
    CD createCD(CD cd);
}
```

3. Utilizzo delle interfacce annotate precedentemente

```
@Stateless
public class ItemEJB
    implements ItemLocal, ItemRemote {
    // ...
}
```

È possibile definire le interfacce stesso nel *bean* in questo modo:

```
@Stateless
@Remote(ItemRemote.class)
@Local(ItemLocal.class)
@LocalBean → \\@LocalBean aggiunge la no-interface view
public class ItemEJB implements ItemLocal, ItemRemote {
    // ...
}
```

11.6 EJB con JNDI: scope

Alla creazione di un *EJB* viene creato automaticamente un nome **Java Naming and Directory Interface** (JNDI).

La sintassi è la seguente: `java:<scope>[/<app-name>]/<module-name>/<bean-name>[!<fully-qualified-interface-name>]`

- **<scope>** definisce una serie di namespaces standard che mappano diversi scope di una applicazione *Java EE*: **global**, **app**, **module**.

- **<app-name>** è richiesto soltanto se il *session bean* è impacchettato all'interno di un file `war` o `ear`. Se è questo il caso allora avrà di default il nome del file.
- **<module-name>** è il nome del modulo in cui il *session bean* è impacchettato.
- **<bean-name>** è il nome del session bean.
- **<fully-qualified-interface-name>** è il nome di ogni interfaccia business.

11.7 Tipi di EJB

Stateless Beans

Gli **stateless bean** sono i *bean* più popolari perché sono semplici e potenti. **Stateless** sta ad indicare che un task deve essere completato in una singola invocazione di un metodo avendo fornito tutti i parametri necessari.

Gli *stateless session bean* possono essere raggruppati e condivisi tra diversi client, proprio perché non tengono traccia di uno stato. Questo significa che per ogni *EJB stateless*, il container tiene un certo numero di istanze (un pool) in memoria e le condivide tra i clients.

Quando un client invoca un metodo su uno *stateless bean*, il container prende un'istanza dal pool e l'asigna al client. Quando la richiesta del client finisce, l'istanza ritorna nel pool per essere riutilizzata. Questo significa che necessitiamo soltanto di un piccolo numero di *beans* per gestire una serie di clients. Ovviamente il container non garantisce la stessa istanza per lo stesso client.

Codice:

@Stateless

```
public class ItemEJB {

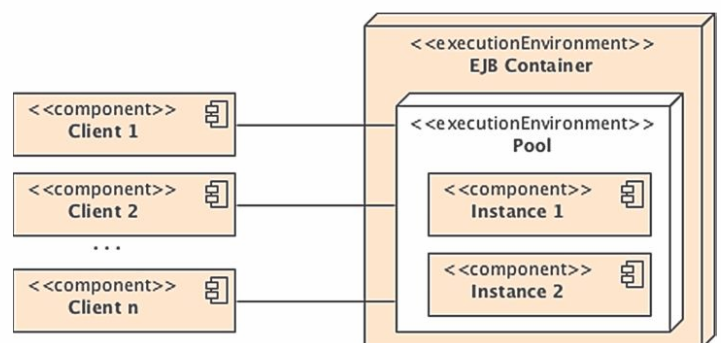
    @PersistenceContext(unitName = "chapter07PU")
    private EntityManager em;

    public List<Book> findBooks() {
        TypedQuery<Book> query = em.createNamedQuery(Book.FIND_ALL, Book.class);
        return query.getResultList();
    }

    public List<CD> findCDs() {
        TypedQuery<CD> query = em.createNamedQuery(CD.FIND_ALL, CD.class);
        return query.getResultList();
    }

    public Book createBook(Book book) {
        em.persist(book);
        return book;
    }

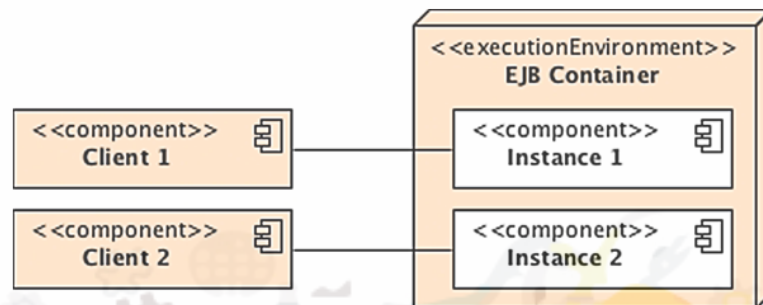
    public CD createCD(CD cd) {
        em.persist(cd);
        return cd;
    }
}
```



Stateful Beans

Gli *EJB stateless* abbiamo detto che forniscono dei metodi di business ai loro clients ma non mantengono uno stato tra loro.

I **session bean stateful**, invece, mantengono lo stato della conversazione. Sono utili per task che necessitano di essere eseguiti in una serie di step, ognuno dei quali fa affidamento sullo stato mantenuto immediatamente prima; quindi, riducono il carico di lavoro.



Prendiamo come esempio un carrello per la spesa su un sito e-commerce. Un customer fa il log-in avviando la sua sessione, sceglie vari libri e li aggiunge al carrello. Alla fine, il customer fa il checkout dei libri, li paga, fa il log-out e la sessione termina. Il carrello tiene traccia dello stato di quanti libri il customer ha scelto durante le varie interazioni. Queste interazioni con una componente *stateful* potrebbero essere scritte come segue:

```
@Stateful
@StatefulTimeout(value = 20, unit = TimeUnit.SECONDS)
public class ShoppingCartEJB {
    private List<Item> cartItems = new ArrayList<>();

    public void addItem(Item item) {
        if (!cartItems.contains(item))
            cartItems.add(item);
    }

    public void removeItem(Item item) {
        if (cartItems.contains(item))
            cartItems.remove(item);
    }

    public Integer getNumberOfItems() {
        if (cartItems == null || cartItems.isEmpty())
            return 0;
        return cartItems.size();
    }

    public Float getTotal() {
        if (cartItems == null || cartItems.isEmpty())
            return 0f;
        Float total = 0f;
        for (Item cartItem : cartItems) {
            total += (cartItem.getPrice());
        }
        return total;
    }

    public void empty() {
        cartItems.clear();
    }
}
```

```

@Remove
public void checkout() {
    // Do some business logic
    cartItems.clear();
}
}

```

Il codice mostra come un *session bean stateful* funziona: due libri sono creati e aggiunti al carrello di una componente *stateful*. Alla fine, il metodo `checkOutShoppingCart()` fa affidamento sullo stato mantenuto e può fare il check-out dei due libri. Quando il client invoca un *session bean stateful* nel server, il *container EJB* fornisce la stessa istanza per ogni successiva invocazione di metodi. I *session bean stateful* non possono essere riutilizzati da altri client.

Passivazione e attivazione

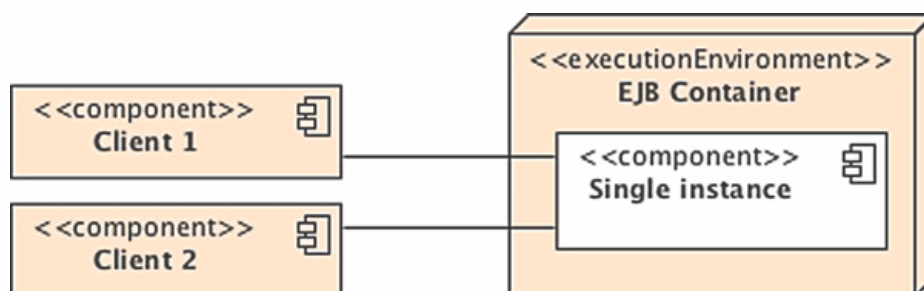
Questa correlazione **uno ad uno** ha un prezzo però, dato che se abbiamo un milione di client, allora avremo in memoria un milione di *session bean stateful*. Per evitare questo sovraccarico in memoria, il container pulisce temporaneamente i *bean stateful* dalla memoria prima che la prossima richiesta dal client li riporta indietro. Questa tecnica è chiamata **passivazione** e **attivazione**.

- **Passivazione** è il processo di rimuovere una istanza dalla memoria e di salvarla in una locazione persistente (un file sul disco, un db ecc.). Questo aiuta a liberare memoria e rilasciare risorse.
- L'**attivazione** è il processo inverso di ripristino.

Passivazione e attivazione sono fatti in automatico dal container.

Singleton

Un **singleton** è un *session bean* istanziato una sola volta per applicazione. Un *singleton* assicura che soltanto una istanza di una classe esista in tutta l'applicazione e fornisce un punto di accesso globale a questa. Ci sono molte situazioni in cui vogliamo un oggetto singleton: ad esempio, se si vuole gestire una cache di oggetti (HashMap) per tutta l'applicazione.



In un'**application managed environment** bisogna fare diverse modifiche per trasformare un *POJO* in un *bean singleton*.

1. Innanzitutto, bisogna fornire un costruttore privato per impedire la creazione di nuove istanze.
2. Per ottenere un'istanza, si deve avere il **metodo statico pubblico** `getInstance()`, il quale ritornerà una singola istanza della classe `CacheSingleton`.
3. Se una classe client necessita di aggiungere un oggetto alla cache usando il singleton deve chiamare il metodo: `CacheSingleton.getInstance().addToCache(myobject)`.

11.8 Packaging e deployment

Per eseguire gli *EJB* abbiamo bisogno di un container in cui eseguirli. Per fare ciò useremo il **packaging**, necessario per mettere insieme:

- Classi *EJB*, interfacce *EJB*;
- Superclassi/superinterfacce;
- Eccezioni, classi ausiliare;
- Un *deployment descriptor* opzionale `ejb-jar.xml`.

Tramite un file **EAR** (Enterprise Archive) vengono raggruppati gli *EJB* che devono essere deployati insieme.

11.9 Invocazione EJB

Il client di un *session bean* può essere una qualsiasi componente. Il client non istanzia direttamente il bean, bensì necessita di un riferimento del bean o di una sua interfaccia. Possiamo ottenere questo riferimento tramite **dependency injection** o **JNDI**.

La **dependency injection** permette al container di iniettare automaticamente il riferimento a tempo di deployment.

Invocazione con Injection

L'annotazione `@EJB` inietta un riferimento del *session bean* nel codice del client.

Codice:

```
@Stateless
public class ItemEJB {...}

// Client code injecting a reference to the EJB
@EJB ItemEJB itemEJB;
```

Se il *session bean* implementa diverse interfacce, il client deve specificare di quale vuole il riferimento. Nel codice seguente `ItemEJB` implementa due interfacce e grazie all'annotazione `@LocalBean`, espone anche una vista senza interfaccia. Il client può invocare l'*EJB* attraverso la sua interfaccia locale, remota o nessuna interfaccia.

Codice:

```
@Stateless
@Remote(ItemRemote.class)
@Local(ItemLocal.class)
@LocalBean
public class ItemEJB implements ItemLocal, ItemRemote {...}

// Client code injecting several references to the EJB or interfaces
@EJB ItemEJB itemEJB;
@EJB ItemLocal itemEJBLocal;
@EJB ItemRemote itemEJBRemote;
```

Invocazione con CDI

Possiamo usare le `@Inject` di **CDI** al posto di `@EJB`. In tal caso non si può passare la stringa di lookup (non consentita per CDI) e bisogna produrre il *remote EJB* da iniettare:

```
// Code producing a remote EJB
@Produces @EJB(lookup = "java:global/classes/ItemEJB") ItemRemote itemEJBRemote;

// Client code injecting the produced remote EJB
@Inject ItemRemote itemEJBRemote;
```

Per *EJB remoti*, potremmo avere bisogno di una stringa **JNDI** per cercarlo:

```
@EJB(lookup = "java:global/classes/ItemEJB") ItemRemote itemEJBRemote;
```

Invocazione diretta di JNDI

I *session bean* possono anche essere cercati usando **JNDI**. **JNDI** è principalmente usato per gli **accessi remoti** quando un client non è *container managed* e non può usare la dependency injection.

Ma **JNDI** può anche essere usato su **client locali** anche se la dependency injection è sempre più semplice. La *injection* è fatta a tempo di deploy. **JNDI** è una alternativa all'*injection*.

Codice:

```
Context ctx = new InitialContext();
ItemRemote itemEJB = (ItemRemote)
    ctx.lookup("java:global/cdbookstore/ItemEJB!org.agoncal.book.javaee7.ItemRemote");
```

11.10 Ciclo di vita dei Session Bean

Il **ciclo di vita degli EJB** è gestito dal **container**; infatti, un client non può creare un'istanza di un *session bean* con un `new`.

I riferimenti vengono ottenuti attraverso l'**iniezione di dipendenze** o attraverso **lookup JNDI**. Inoltre, tutti i bean passano almeno attraverso creazione e distruzione.

I *bean stateful* passano attraverso gli stati di attivazione e passivazione.

Transizioni per Stateless e Singleton Bean

I **bean stateless** hanno in comune con i **bean singleton** il fatto che non mantengono uno stato di conversazione con un client. Entrambi possono essere accessi da qualsiasi client.

I **bean stateless** fanno questo per ogni istanza, mentre un *singleton* fornisce un accesso concorrente ad una singola istanza. Entrambi condividono lo stesso **ciclo di vita**.

1. Il ciclo di vita inizia quando un client richiede un riferimento al bean (nel caso di un singleton, può iniziare anche quando viene fatto il bootstrap del container). Il container crea una nuova istanza di un session bean.
2. Alla creazione vengono anche iniettate le dipendenze richieste.
3. Se l'istanza appena creata ha un metodo annotato con `@PostConstruct`, il container lo invoca.
4. L'istanza del bean processa la chiamata invocata dal client e sta in modalità ready per processare chiamate future. I bean stateless stanno in modalità ready finché il container non libera spazio nel pool. I singleton stanno in modalità ready finché il container non viene spento.

5. Il container non ha più bisogno dell'istanza e invoca il metodo annotato con `@PreDestroy` (se esiste) e termina il ciclo di vita dell'istanza del bean.

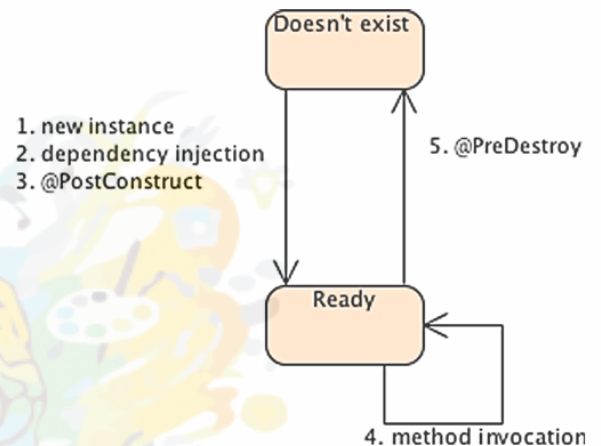
I **session bean stateless** e **singleton** condividono lo stesso ciclo di vita ma ci sono delle differenze nel modo in cui sono creati e distrutti.

Quando facciamo il deploy di un **session bean stateless**, il container crea diverse istanze e le aggiunge in un pool. Quando un client chiama un metodo su un *stateless session bean*, il container seleziona una istanza dal pool ed elimina l'invocazione del metodo di quella istanza. Quando il container non necessita più dell'istanza, la distrugge.

Per i **singleton** invece, la creazione avviene se sono istanziati al bootstrap oppure se dipendono da altri *singleton* che sono stati creati prima.

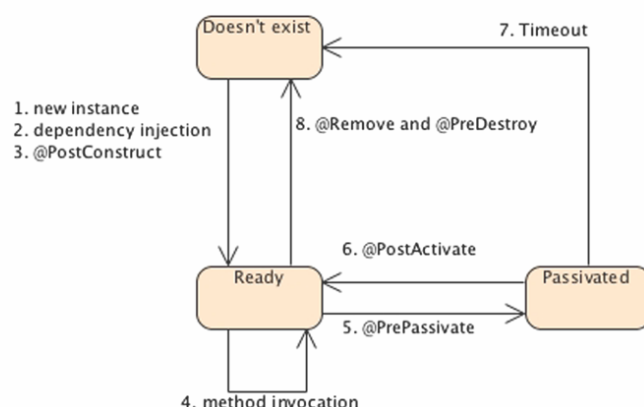
Se così sarà, il container creerà un'istanza a tempo di deploy. Altrimenti, il container creerà un'istanza quando un client invoca un metodo di business.

Dato che i *singleton* durano per tutta la durata dell'applicazione, l'istanza sarà distrutta quando il container si spegne.



Transizioni per *Stateful Bean*

1. Il ciclo di vita inizia quando il client richiede un riferimento al bean. Il container crea una nuova istanza e la memorizza in memoria.
2. Alla creazione, vengono anche iniettate le dipendenze richieste.
3. Se l'istanza appena creata ha un metodo annotato con `@PostConstruct` il container lo invoca.
4. Invocazioni dei metodi da parte dei client.
5. Se il client resta idle per un certo periodo di tempo, il container invoca il metodo annotato con `@PrePassivate` e rende **passivo** il bean in uno storage permanente.
6. Se un client invoca un **bean passivated**, il container lo attiva riportandolo in memoria ed invoca il metodo annotato con `@PostActivate`, se esiste.
7. Se un client non invoca il bean all'interno del *session timeout period*, il container lo elimina.
8. In modo alternato al passo 7, se un client chiama un metodo annotato con `@Remove`, il container invocherà il metodo annotato con `@PreDestroy`, se esiste, e termina il ciclo di vita del bean.



11.11 Autorizzazioni

Lo scopo principale del **modello di sicurezza EJB** è di controllare l'accesso al codice di business.

In *Java EE* l'**autenticazione** è gestita dal **layer di presentazione** (web) o dall'**applicazione client**.

L'EJB controlla se l'autenticazione dell'utente è permessa per accedere ai metodi in base al suo ruolo.

L'autorizzazione può essere:

- **autorizzazione dichiarativa**, in cui il controllo degli accessi è fatto dal *container EJB*;
- **autorizzazione da programma**, attraverso *Java Authentication and Authorization Service* (JAAS API).

Autorizzazione dichiarativa

L'**autenticazione dichiarativa** può essere definita nel bean usando **annotazioni** oppure in **XML deployment descriptor**. Essa richiede di:

- definire ruoli;
- assegnare permessi ai metodi o all'intero bean;
- o cambiare temporaneamente una security identity.

Questi controlli sono fatti dalle annotazioni e ognuna di esse può essere usata in un *bean* o/e su un *metodo*.

Annotation	Bean	Method	Description
@PermitAll	X	X	Indicates that the given method (or the entire bean) is accessible by everyone (all roles are permitted).
@DenyAll	X	X	Indicates that no role is permitted to execute the specified method or all methods of the bean (all roles are denied). This can be useful if you want to deny access to a method in a certain environment (e.g., the method <code>launchNuclearWar()</code> should only be allowed in production but not in a test environment).
@RolesAllowed	X	X	Indicates that a list of roles is allowed to execute the given method (or the entire bean).
@DeclareRoles	X		Defines roles for security checking.
@RunAs	X		Temporarily assigns a new role to a principal.

Autorizzazione da Programma

Permette una grana più fine nel controllo (ad esempio per blocchi di programma, oppure per uno specifico utente). Si usano direttamente le API di *Java Authentication and Authorization Service* (JAAS API).

L'interfaccia `SessionContext` offre metodi per verificare se chi chiama il metodo ha un certo ruolo, oppure verificare l'utente stesso.

11.12 Transazioni

Le *transazioni* sono alla base del business e quindi la loro gestione permette alle applicazioni di avere dati consistenti e di processarli in maniera affidabile. Una **transazione** rappresenta un insieme di operazioni logiche che devono essere realizzate come una singola unità di lavoro.

Le operazioni vanno realizzate separatamente e:

- se vanno tutto a buon fine, la transazione va a buon fine (**commit**);
- altrimenti si riporta la situazione a prima che la transazione iniziasse (**rollback**).

Transazioni: proprietà ACID

Le *transazioni* devono garantire l'affidabilità e la robustezza delle **proprietà ACID**:

Atomicity: una transazione è composta da una o più operazioni raggruppate in un'unità di lavoro. Alla fine della transazione, queste operazioni o sono eseguite tutte con successo (**commit**) oppure se accade qualcosa di imprevisto o irrecuperabile nessuna di esse viene eseguito del tutto (**rollback**).

Consistency: alla fine della transazione, i dati vengono lasciati in uno stato coerente.

Isolation: lo stato intermedio di una transazione non è visibile alle applicazioni esterne.

Durability: una volta che la transazione è stata confermata, le modifiche apportate ai dati sono visibili ad altre applicazioni.

Transazioni locali in JTA

Quando sviluppiamo la logica di business con gli *EJB* non ci dobbiamo preoccupare delle *transazioni* perché **JTA** ci fornisce un'astrazione di tutta la complessità sottostante.

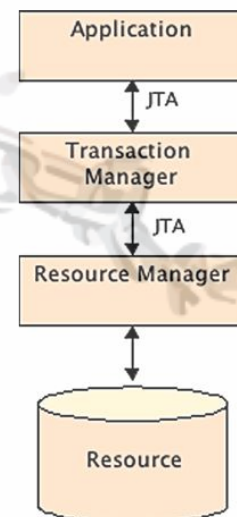
Con gli *EJB* possiamo sviluppare delle applicazioni transazionali lasciando al container implementare il tutto. Un *container EJB* è un **transaction manager** che supporta **JTA**.

I metodi degli *EJB* sono wrappati in una transazione in modo automatico e questo comportamento di default è chiamato **CMT** (container-managed transaction), perché le transazioni sono gestite dal *container EJB*.

Esempio:

Semplice di applicazione su una risorsa singola (DB).

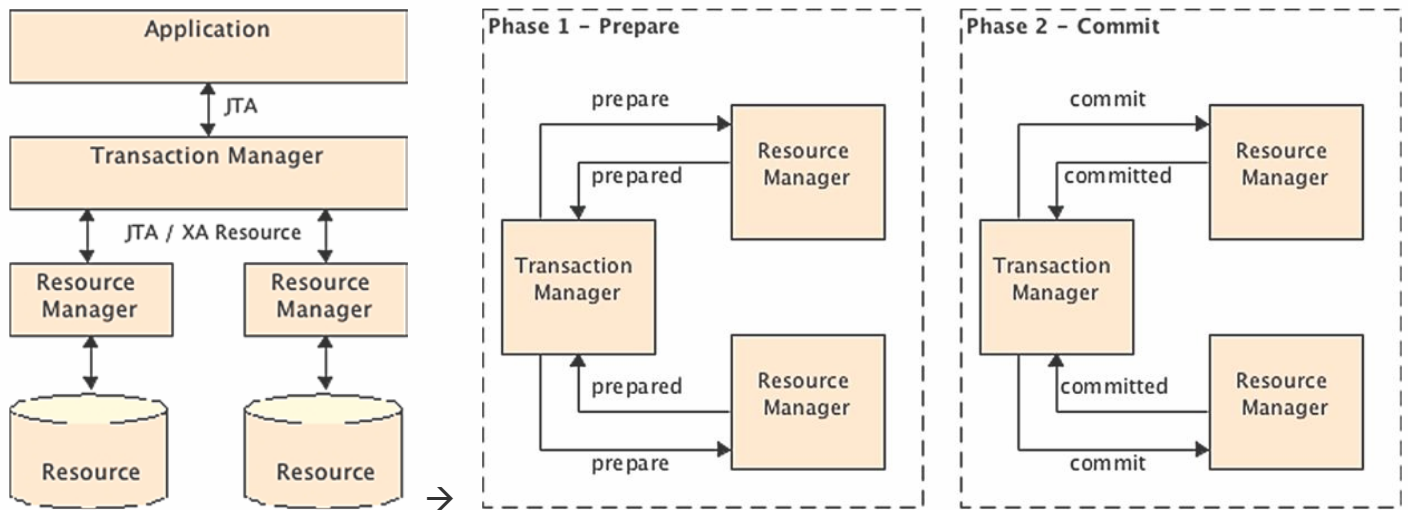
- **Transaction Manager (TM)**: componente centrale che gestisce e operazioni, informa il *RM* chi partecipa nella transazione.
- **Resource Manager (RM)**: responsabile della gestione delle risorse e della loro registrazione con il *transaction manager* (ad esempio un driver, una risorsa JMS, etc.).
- **Risorsa**: *persistent storage* da cui leggere o in cui scrivere (esempio DB).



Transazioni distribuite

Le **transazioni distribuite** avvengono quando ci sono più risorse coinvolte, ad esempio diversi DB. Il coordinamento viene effettuato usando *Extended Architecture (XA)* per *Distributed Transaction Processing*.

XA implementa un algoritmo chiamato **two-phase commit**. Durante la **prima fase**, ogni *Resource Manager* viene notificato, dal *Transaction Manager*, attraverso un comando "**prepare**" che un commit sta per essere eseguito. Se tutti dichiarano che sono *prepared*, la transazione può procedere: ai *resource managers* viene chiesto di fare commit nella seconda fase.



Transazione distribuita

Two-phase commit

Transazioni distribuite attraverso la rete

Sono applicazioni diverse su network che condividono la transazione. I *TM* lavorano insieme per assicurare la *two-phase commit*.

Transazioni Container-Managed

Per default le *transazioni* sono gestite dal container:

- il programmatore non sviluppa codice *JTA*;
- i servizi vengono forniti ai session beans;
- gli *EJB* sono per loro natura transazionali: *configuration by exception*.





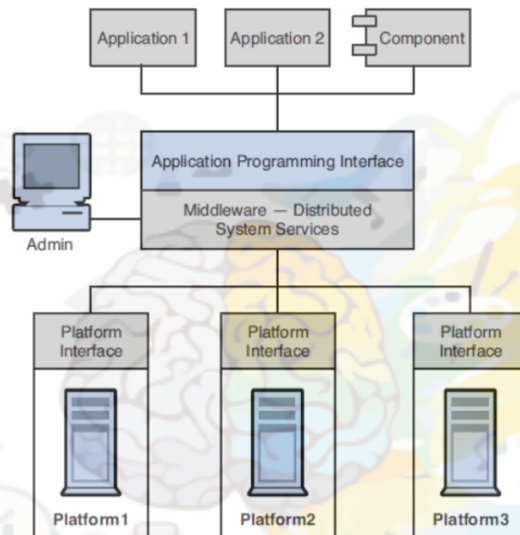
JMS API

(Java Messaging Service API)

Capitolo 12 – Java Messaging Service API

12.1 Introduzione *Middleware*

Dati i continui cambiamenti della tecnologia i sistemi software devono riuscire ad assecondare questi cambiamenti, integrando nuove componenti o scalando quelle già presenti. Il modo più efficiente per integrare componenti eterogenee è quello di fornire un livello che permette loro di comunicare malgrado le differenze; questo livello è chiamato **middleware**. Questo **middleware** permette a componenti differenti, sviluppate indipendentemente e che sono in esecuzione su piattaforme differenti, di interagire tra di loro. Il *middleware* risiede tra il livello di applicazione e il livello di piattaforma.



Le applicazioni distribuite su nodi di rete differenti necessitano l'uso di una **Application Programming Interface (API)** per comunicare. In più, fornendo una interfaccia amministrativa, questo nuovo sistema interconnesso di applicazioni può essere reso sicuro ed affidabile.

I *middleware* possono essere raggruppati in queste categorie:

- **Middleware basati su RPC** (Remote Procedure Call), che permettono alle procedure di una applicazione di chiamare procedure in applicazioni remote come se fossero chiamate a procedure locali. Il middleware implementa un meccanismo di collegamento che individua le procedure remote e rende queste disponibili al chiamante in modo trasparente.
- **Middleware basati su ORB** (Object request Broker), che abilita gli oggetti di una applicazione di essere distribuiti e condivisi tra reti eterogenee.
- **Middleware Message Oriented** (MOM).

Nei sistemi basati su *RPC* e *ORB* quando una procedura ne chiama un'altra, deve aspettare che la procedura invocata ritorni prima che può fare qualsiasi altra cosa. Realizzando una **comunicazione sincrona**.

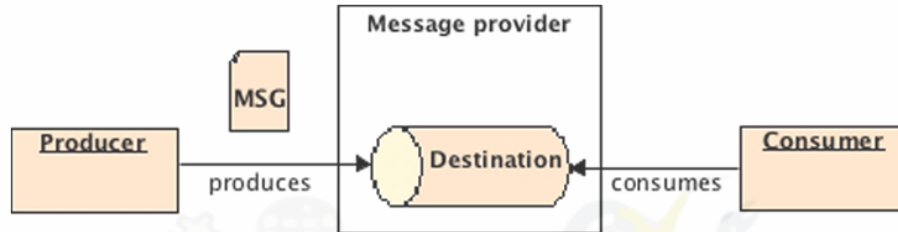
I sistemi basati su **MOM** permettono una **comunicazione asincrona** tramite messaggi.

12.2 Message-Oriented Middleware

MOM (Message-Oriented middleware) è un software (provider) che permette lo scambio di messaggi asincroni fra sistemi eterogenei. Può essere visto come un buffer che produce e consuma messaggi.

È **loosely coupled** e i produttori non sanno chi è all'altra estremità del canale di comunicazione a consumare il messaggio, inoltre, produttore consumatore non devono essere disponibili contemporaneamente per comunicare.

- Quando un messaggio viene inviato, il software che memorizza e invia il messaggio è detto **Provider** (a volte chiamato broker).
- Il *sender* del messaggio è chiamato **Producer** e la locazione in cui il messaggio è memorizzato è detta **destinazione**.
- La *componente* che riceve il messaggio è detta **Consumer** ed ogni *componente* interessata ad un messaggio in una particolare destinazione può consumarlo.



12.3 Java Message Service (JMS)

In *Java EE*, l'API che gestisce questi concetti è **Java Message Service (JMS)**: un set di interfacce e classi per

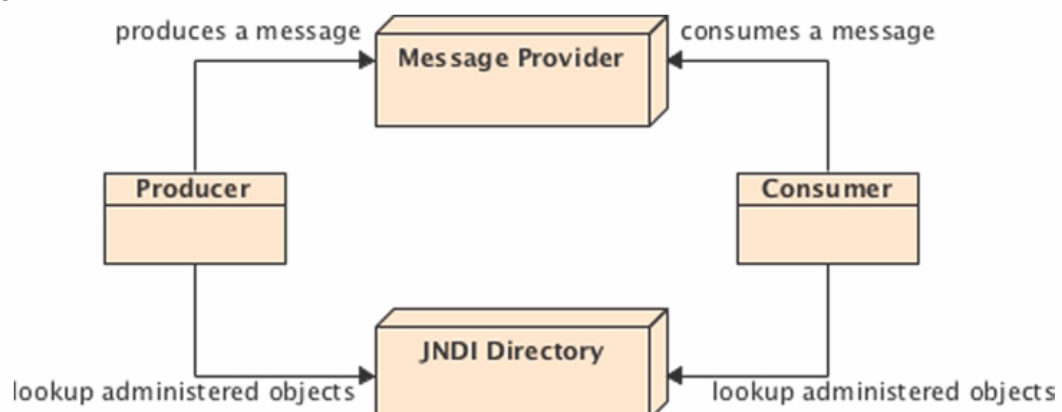
- connettersi ad un provider;
- creare un messaggio;
- inviare e ricevere un messaggio.

In un *EJB container*, **Message-Driven Beans (MDBs)** possono essere usati per ricevere i messaggi in maniera *container-managed*.

12.4 Architettura di Messaging

Un'architettura di messaging è composta da 4 componenti principali:

- **Provider**: è la componente necessaria per instradare messaggi; gestisce buffering e delivery dei messaggi;
- **Clients**: è una qualunque applicazione Java o una componente che produce o consuma messaggi per/da un provider. il termine client si usa genericamente per producer, sender, publisher, receiver, subscriber;
- **Messages**: oggetti che i clients inviano/ricevono dal provider;
- **Administered objects**: oggetti (connection factories e destinazioni) fornite attraverso JNDI per lookups o injection.



Il *provider* permette una **comunicazione asincrona** fornendo una **destinazione** dove i messaggi possono essere mantenuti finché non vengono instradati verso un client.

Esistono due tipi di *destinazione*:

1. **Point-to-point** (P2P) *model*: la *destination* è chiamata **coda**. Il client inserisce un messaggio in coda, mentre un altro client riceve il messaggio. Una volta fatto l'acknowledge, il *message provider* rimuove il messaggio dalla coda.
2. **Publish-subscribe** (pub-sub) *model*: la *destination* è chiamata **topic**. Il client pubblica un messaggio con un *topic*, e tutti i sottoscrittori al *topic* riceveranno il messaggio.

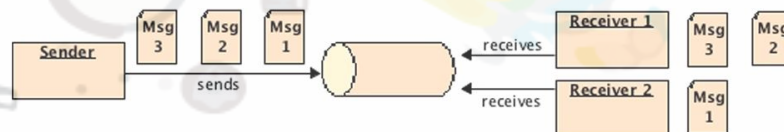
Modello Point-to-Point

Con il modello **point-to-point** il messaggio viaggia da un singolo *producer* verso un singolo *consumer*. Ogni messaggio viene inviato ad una specifica coda, ed il *receiver* riceve il messaggio dalla coda. La coda mantiene i messaggi finché non vengono consumati o scadono.



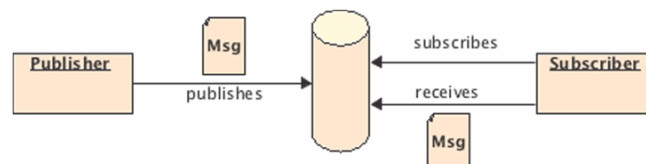
Nel modello P2P, esiste un solo *receiver* per ogni messaggio, invece, una coda può avere *consumers* multipli ma quando un *receiver* consuma il messaggio, questo viene tolto dalla coda e nessun altro *receiver* potrà consumarlo.

Il modello P2P non garantisce che i messaggi siano instradati in un particolare ordine: un *provider* può riceverle in un particolare ordine, o random, o in qualunque altro ordine.



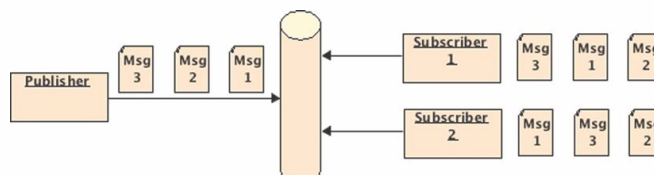
Modello Pub-sub

Nel modello **pub-sub**, un singolo messaggio è inviato da un *producer* a diversi *consumers*. Il modello è costruito intorno ai concetti di **topic**, **publishers** e **subscribers**. I *consumers* sono chiamati *subscribers* e hanno la necessità di sottoscrivere ad un *topic* e a partire dal momento della loro iscrizione ricevono tutti i messaggi che vengono inviati a questa *topic*.



Il *topic* conserva i messaggi fino a quando non vengono distribuiti a tutti i *subscribers*. I *subscribers* possono anche definire delle preferenze per cui scartare alcuni messaggi. C'è una **dipendenza temporale** nel senso che il *subscriber* riceve solo i messaggi da quando si iscrive e non quelli prima. Inoltre, multipli *subscribers* possono consumare lo stesso messaggio.

Questo modello è utile per applicazioni di tipo **broadcast** in cui un singolo messaggio viene recapitato a diversi consumatori.



12.5 Administered Objects

Gli **administered objects** sono oggetti che sono configurati da un amministratore di sistema, e quindi, non sono sotto il controllo del programmatore. Il *provider* permette di configurare questi oggetti e li rende disponibili nello spazio dei nomi *JNDI*. Come i *JDBC datasources*, questi oggetti vengono creati una sola volta.

I due tipi di *administered objects* sono:

1. **Connection factory**: usato dai clients per creare una connessione ad una destinazione;
2. **Destinazioni**: punti di distribuzione del messaggio che ricevono, mantengono e distribuiscono messaggi. Le destinazioni possono essere **code** (P2P) o **topic** (pub-sub).

Connection Factory

Le **connection factory** sono *administered objects* gestite tramite l'interfaccia `javax.jms.ConnectionFactory` che incapsula i parametri definiti da un amministratore. Per usare un *administered object* come una **ConnectionFactory**, il client deve eseguire una *JNDI lookup* (o usare injection).

Esempio:

```
Context ctx = new InitialContext();
ConnectionFactory ConnectionFactory = (ConnectionFactory)
    ctx.lookup("jms/javaee7/ConnectionFactory");
```

Destination

Una **destination** è un *administered object* che contiene un **provider-specific configuration information** come ad esempio un *destination address*. Questo meccanismo è nascosto al *JMS client* attraverso l'uso dell'interfaccia `javax.jms.Destination`. Come per le **connection factory**, è necessaria una *JNDI lookup* per restituire tali oggetti:

```
Context ctx = new InitialContext();
Destination queue = (Destination) ctx.lookup("jms/javaee7/Queue");
```

12.6 Java Messaging Service API

JMS è un insieme di standard *Java API* che permette alle applicazioni di creare, inviare, ricevere e leggere messaggi in maniera asincrona. *JMS* definisce un insieme di interfacce e classi per la comunicazione con altri *message provider*. *JMS* è analogo a *JDBC*:

- **JDBC** permette la connessione a diversi databases (Derby, MySQL, Oracle, etc.);
- **JMS** permette la connessione a diversi providers (OpenMQ, MQSeries, SonicMQ, etc.).

Altre interfacce

Le altre API di *JMS* sono:

- **JMSContext**: una connessione attiva ad un *JMS provider* e un *context single-threaded* per **inviare** e **ricevere** messaggi;
- **JMSProducer**: oggetto creato da un *JMSContext* per **inviare** messaggi ad una **code** o ad un **topic**;
- **JMSConsumer**: oggetto creato da un *JMSContext* per **ricevere** messaggi inviati ad una **code** o ad un **topic**.

Message Producers

Con le nuove API di *JMS 2.0*, la scrittura di *produttori* e *consumatori* diventa meno complicata. Rimangono gli *administered objects* *ConnectionFactory* e *Destination* e, a seconda se si è fuori o dentro un container, si usa **JNDI lookup** oppure **injection**.

Abbiamo 3 esempi:

1. Produttore fuori da un container

Un oggetto *JMSProducer* viene creato da un *JMSContext* ed usato per inviare messaggi. I passi da eseguire sono:

- Ottenere una *connection factory* ed una coda con *JNDI*;
- Creare un *JMSContext* usando la factory;
- Creare un *JMSProducer* usando il contesto;
- Inviare il messaggio usando il metodo *send()* del producer per inviare il messaggio.

Codice:

```
public class Producer {
    public static void main(String[] args) {
        try {
            Context jndiContext = new InitialContext();
            ConnectionFactory connectionFactory = (ConnectionFactory)
                jndiContext.lookup("jms/javaee7/ConnectionFactory");
            Destination queue = (Destination)
                jndiContext.lookup("jms/javaee7/Queue");
            try (JMSContext context = connectionFactory.createContext()) {
                context.createProducer().send(queue, "Text message sent at " + new
                    Date());
            }
        } catch (NamingException e) {
            e.printStackTrace();
        }
    }
}
```

- Prendiamo il **contesto** da JNDI e cerchiamo l'*administered object* **ConnectionFactory**;
- Preleviamo la **coda** dal JNDI;
- Successivamente si cerca di creare il contesto; se fallisce chiude il contesto;
- Dal contesto creiamo il producer e a questo producer chiediamo di **inviare** sulla coda questo messaggio di testo. Nel momento in cui il *send* termina, il messaggio è stato preso in carico dal broker che si occuperà lui del resto.

2. Produttore in un container

Possiamo trasformare il produttore in un **EJB Stateless**, che essendo in un container chiede risorse con le annotazioni **@Resource**. Chiede in particolare una *queue* e una *Connection factory*. Prendiamo poi tutto il codice di prima e lo mettiamo in un metodo *sendMessage* che fa la stessa cosa di prima.

Codice:

```
@Stateless
public class ProducerEJB {

    @Resource(lookup = "jms/javaee7/ConnectionFactory")
    private ConnectionFactory connectionFactory;
```

```

@Resource(lookup = "jms/javaee7/Queue")
private Queue queue;

public void sendMessage() {
    try (JMSContext context = connectionFactory.createContext()) {
        context.createProducer().send(queue, "Text message sent at " + new Date());
    }
}
}

```

3. Produttore in un container con CDI

Possiamo chiedere ad un oggetto standard che non sia un *EJB* di iniettare una factory e cerchiamo la coda usando *JNDI* con `@Resource`.

Codice:

```

public class Producer {

    @Inject
    @JMSConnectionFactory("jms/javaee7/ConnectionFactory")
    private JMSContext context;

    @Resource(lookup = "jms/javaee7/Queue")
    private Queue queue;

    public void sendMessage() {
        context.createProducer().send(queue, "Text message sent at " + new Date());
    }
}

```

Message Consumers

I consumer possono essere di tipi diversi:

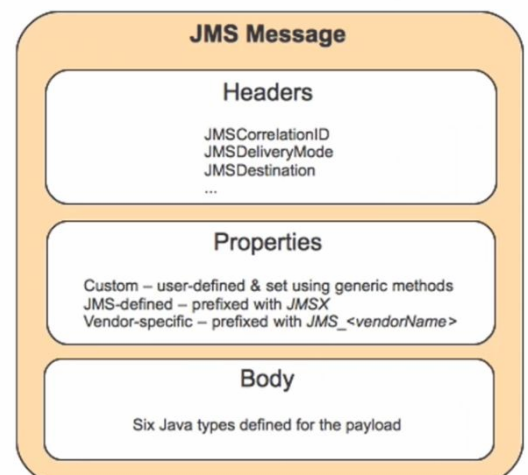
- **Sincroni:** il ricevitore esplicitamente preleva il messaggio dalla destinazione, invocando `receive()`;
- **Asincrono:** il ricevitore si registra all'evento di arrivo di un messaggio e implementa `MessageListener`, in modo che all'arrivo del messaggio viene invocato il suo metodo `onMessage()`;

12.7 Meccanismi di affidabilità

Un **JMS Message** consiste di 3 parti:

1. **Headers:** metadati predefiniti;
2. **Properties:** metadati non standard;
3. **Body:** può essere empty, String, Map, bytes, Stream, Object.

Per assicurare un recapito affidabile, *JMS* definisce diversi **livelli di affidabilità** per assicurare che i messaggi siano instradati correttamente (anche se il provider va in crash o è sotto carico elevato).



I meccanismi sono i seguenti:

1. Filtering messages

Usando i selettori è possibile ricevere solo i messaggi che si desiderano. Il *filtering* è un meccanismo che permette di far arrivare i messaggi solo ai *consumer* che hanno espresso una preferenza per quegli specifici messaggi. Infatti, il messaggio è inviato in **broadcast** a diversi client; si definisce poi un **selector** in modo che venga consumato solo da customer interessati.

Il vantaggio è che non c'è spreco di banda perché non si ricevono messaggi di non interesse.

La preferenza può essere espressa in due modi, tramite:

- i **metadati** presenti nell'header;
- agendo sulle **proprietà** (le proprietà le scegliamo noi).

Il **message selector** è una stringa che contiene un'espressione SQL:

```
context.createConsumer(queue, "JMSPriority < 6").receive();
context.createConsumer(queue, "JMSPriority < 6 AND orderAmount < 200").receive();
context.createConsumer(queue, "orderAmount BETWEEN 1000 AND 2000").receive();
```

Dal lato del *producer*, possiamo quindi settare una **property** e un **metadato** in questa maniera:

```
context.createTextMessage().setIntProperty("orderAmount", 1530);
context.createTextMessage().setJMSPriority(5);
```

2. Setting message time-to-live

Si sceglie il **time-to-live** (expiration time) in modo da non intradare i messaggi obsoleti. Il tempo viene settato in millisecondi, passato il quale, il *provider* (broker) rimuove il messaggio. Per fare ciò si utilizza il metodo del *producer*:

```
context.createProducer().setTimeToLive(1000).send(queue, message);
```

3. Gestire la persistenza dei messaggi

Possiamo chiedere diversi **livelli di persistenza** al *provider*, ovviamente più la qualità del servizio che richiediamo è alta, più le prestazioni del *provider* degradano. JMS supporta 2 modalità di *message delivery*:

1. **Persistent delivery**: il messaggio viene salvato sul provider (disco o database). Il *persistent delivery* è il valore di default che può essere "degradato" per migliorare le prestazioni. Per fare ciò dobbiamo settare il **delivery mode**:

```
context.createProducer().setDeliveryMode(DeliveryMode.NON_PERSISTENT).send(queue, message);
```

2. **Non-persistent delivery**: il messaggio non viene salvato.

4. Controllo degli Acknowledgment

Si vuole ricevere una verifica del recapito del messaggio al destinatario, ovvero ricevere un **ACK**.

Questo ACK lo possiamo fornire in diverse modalità:

- **AUTO_ACKNOWLEDGE**: la sessione automaticamente fa ack di un messaggio.
- **CLIENT_ACKNOWLEDGE**: ack esplicito del client chiamando il metodo `Message.acknowledge()`.

Esempio di Producer:

usiamo l'annotazione `@JMSSessionMode` per settare l'*acknowledgment mode*

```
@Inject
@JMSConnectionFactory("jms/connectionFactory")
@JMSSessionMode(JMSContext.AUTO_ACKNOWLEDGE)
private JMSContext context;

...
context.createProducer().send(queue, message);
```

Esempio di consumer:

fa esplicito ack del messaggio chiamando il metodo `message.acknowledge()`

5. Durable Consumer

Nel modello **pub-sub**, un *consumer* che non è in esecuzione perde i messaggi che vengono postati sul *topic*. Mentre invece con **durable consumer**, gli vengono recapitati anche i messaggi che nel frattempo sono stati persi. Quindi, con *durable consumer* tutti i messaggi vengono mantenuti dal *provider* fino a quando tutti i *consumer* li hanno ricevuti.

Questo lo si può fare attraverso `JMSContext` con un **id** univoco che caratterizza questo consumer:
`context.createDurableConsumer(topic, "uniqueID").receive();`

A questo punto il client inizia la connessione e riceve i messaggi. Il nome "**uniqueID**" è usato come identificatore della *durable subscription*. Inoltre, ogni *durable consumer* deve avere un `unique ID`:

- che corrisponde alla dichiarazione di un'unica *connection factory*;
- per ogni potenziale durable consumer.

6. Scegliere la priorità del messaggio

Per scegliere la **priorità del messaggio** si istruisce il *JMS* a fare il delivery di messaggi urgenti per primi. Le priorità sono contenute nell'header del messaggio con valori che vanno da **0** (bassa priorità) a **9** (alta priorità).

12.8 Message-Driven Beans

I **Message-Driven Beans** (MDBs) sono messaggi asincroni dei consumer eseguiti in un **EJB container**.

Quindi, l'*EJB container* si occupa dei servizi (transactions, security, concurrency, message acknowledgment, etc.), mentre l'*MDB* si occupa di consumare messaggi.

I *MDB* sono **stateless** (non mantengono stato attraverso invocazioni separate), mentre, l'*EJB container* può avere numerose istanze, eseguite in concorrenza per processare messaggi provenienti da diversi producers.

In generale gli *MDBs* sono in ascolto su una *destination* (queue o topic) e, quando il messaggio arriva, lo consuma e lo processa.

I *MDBs* rispondono a messaggi ricevuti dal container laddove gli *stateless session beans* rispondono a richieste client attraverso un'interfaccia appropriata (local, remote o no-interface).

Inoltre, tramite *CDI* può accedere a tutte le risorse come *EJB*, *JDBC*, ecc. in quanto è comunque un **bean**.

Il vantaggio di usare un *MDB* (rispetto ad un *JMS Client*) è che la transazione, il multithread, la sicurezza, etc., sono gestite dal container.

MDB: come è fatto

Un *MDB* non è parte del modello *EJB Lite*: serve un'implementazione *full EE*. Tra i requisiti abbiamo:

- Annotato con `@MessageDrive`;
- Deve implementare l'interfaccia del *listener* definita come `public`, **NON** `final` o `abstract`;
- Deve esserci un costruttore senza argomenti altrimenti il container non sa crearlo;
- La classe non deve avere il metodo `finalize()`.

MDB: Dependency Injection

Come per tutti gli *EJB*, gli **MDBs** possono usare **dependency injection** per ottenere riferimenti a risorse come *JDBC datasources*, *EJBs*, o altri oggetti.

Injection è il meccanismo attraverso il quale il container inserisce le dipendenze automaticamente dopo aver creato l'oggetto: queste risorse devono essere disponibili nel container oppure nell'*enviroment context*.

MDB: MessageDrivenContext

Questa interfaccia fornisce accesso al *runtime context*, che il container fornisce per un'istanza di un *MDB*. Il container passa l'interfaccia *MessageDrivenContext* all'istanza che rimane associata per il tempo di vita dell'*MDB*. Inoltre, tale interfaccia permette all'*MDB* di fare **roll back** di una transazione, ottenere il caller, ecc...

MDB: Ciclo di vita

Il **ciclo di vita** è simile a quello di uno *stateless*. È possibile inserire degli **interceptors** sui metodi ed è possibile per un *MDB* ricevere e, contemporaneamente, inviare messaggi.

MDB come Consumer Sincrono

Per natura, gli *MDBs* sono progettati per funzionare come **consumatori di messaggi asincroni**. Infatti, implementano l'interfaccia *message listener*, che viene risvegliata (triggered) dal container quando un messaggio arriva.

Un *MDB* può essere anche un **consumer sincrónico** ma non è raccomandato poiché tali consumers bloccano le risorse del server, in particolare, gli *EJBs* si bloccheranno in un loop senza eseguire nessun lavoro ed il container non sarà in grado di liberarli.

Gli *MDBs*, come gli *stateless session beans*, vivono in un pool di una certa taglia: quando il container ha bisogno di un'istanza la prende dal pool e la usa. Se l'istanza va in un loop infinito, il pool si svuoterà e tutte le risorse saranno bloccate in un **busy looping**.

L'*EJB container* può generare nuove istanze di *MDB* incrementando il pool ma aumentando così il consumo di memoria. Per questa ragione, *session beans* e *MDBs* non dovrebbero essere usati come **synchronous message consumers**.

MDB come Producer

Gli *MDBs* possono diventare anche **message producers**. Abbiamo un workflow che prevede che essi ricevano messaggi da una destinazione, li processino, e li rinviino ad un'altra destinazione. Per aggiungere questa capacità bisogna usare le *API* di *JMS*.

MDB: transazione per i messaggi

Le **transazioni per i messaggi** stanno ad indicare che un certo numero di messaggi vanno recapitati tutti insieme o nessuno. Le transazioni possono essere **Bean-managed** oppure **Container-managed**.



Web Services

Libera Mente

Capitolo 13 – Web Services

13.1 Service-Oriented Architecture

La **tecnologia (middleware) a servizi** è una tecnologia per l'integrazione di applicazioni distribuite con lo scopo di risolvere problemi pratici di interoperabilità. Tale tecnologia è basata su standard accettati dalla maggior parte dei produttori di software e focalizza l'attenzione sul concetto di **servizio**.

L'**architettura orientata ai servizi (SOA)** è un tipo di progettazione software con lo scopo di riutilizzare dei componenti software attraverso interfacce di servizio che usano un linguaggio di comunicazione comune in una rete. Fornisce, quindi, una metodologia per utilizzare al meglio la **tecnologia basata su servizi**. Infatti, i **web services** rappresentano la tecnologia più importante in questo ambito.

I **servizi** sono un insieme di funzionalità, con lo scopo di completare un'attività specifica come il recupero di determinate informazioni o l'esecuzione di un'operazione. Inoltre, sono scritti in modo da essere **indipendenti** dal contesto in cui sono usati, infatti, *service providers* e *consumers* sono debolmente accoppiati.

A livello middleware il **loosely coupling** (basso accoppiamento) richiede che l'approccio *service-oriented* sia indipendente dalle specifiche tecnologie o dai sistemi operativi. Inoltre, servizi e composizione di servizi sono indipendenti dai linguaggi di programmazione utilizzati.

Affinché si possa parlare di *Service-Oriented Architecture*, i servizi devono essere:

- **neutrali** rispetto alla tecnologia: devono poter essere invocati attraverso tecnologie *standard compliant* (protocolli, meccanismi di discovery).
- **loosely coupled**: non bisogna conoscere la struttura interna o context sia a livello client che server.
- supportare **location transparency**: le definizioni e le informazioni di localizzazione devono essere memorizzate in una repository ed accessibile a diversi client che possono invocarli indipendentemente dalla locazione.

I **servizi** possono essere forniti su una singola macchina o su un gran numero di dispositivi distribuiti su una rete locale o su una rete distribuita (incluso mobile e ad hoc networks).

I **servizi** che usano Internet come *communication medium* e *open Internet standards* sono i **Web Services**.

13.2 Web Services

I **Web services** sono un qualcosa di accessibile sul Web ed offrono un servizio tramite una pagina *HTML*, una *sevlet* o altro; è il concetto fondamentale dell'**architettura orientata ai servizi** (SOA).

I *Web services* posso accedere ai servizi in due modi, in base a due famiglie di protocolli:

1. **SOAP** (Simple Object Access Protocol);
2. **REST** (Representational State Transfer).

Il concetto chiave è quello del **loosely coupled**. I *Web Services* sono **debolmente accoppiati** cioè permettono di usare un servizio senza che il *consumer* sappia qualcosa sull'implementazione del servizio.

Il *basso accoppiamento* viene raggiunto usando il **protocollo HTTP** come protocollo di trasporto per il *middleware*. Il protocollo *HTTP* ha una serie di caratteristiche vantaggioso:

- protocollo stateless, robusto, semplice;
- indipendente dalla piattaforma;
- scalabile.

I *Web Services* possono essere invocati sia da singoli client che da **Business-to-Business** (B2B), ovvero del software che invoca dei *Web Services* altrove.

I *Web Services*, quindi, permettono lo sviluppo di applicazioni distribuite che siano:

- veloci;
- basso costo;
- di facile composizione con altri servizi simili, in modo da creare servizi più complessi.

Tramite *Web Services* possiamo esporre la **logica di applicazione** sulla rete internet. Ogni porzione di codice e componente può essere trasformata in un *web service*, ed è questo il cuore dell'approccio orientato ai servizi, ovvero concepire l'applicazione come un insieme di servizi che sono disponibili sulla rete.

Web Services: Requisiti

Sono **neutri rispetto alla tecnologia**. Quindi abbiamo un *Client* scritto con una tecnologia x, e dei servizi scritti con una tecnologia y ed è possibile per il Client sfruttare comunque i servizi.

Sono **debolmente accoppiati**: nessuna conoscenza con le strutture interne, oppure il contesto, da parte del client.

I *Web Services*, inoltre, sono **trasparenti alla locazione**: ovvero sono stati pensati per essere disponibili attraverso registri pubblici.

Web Services: Definizione

Un *WS* è un sistema software progettato per supportare interazioni interoperabili tra computer su una rete. L'interfaccia è specificata in una descrizione analizzabile automaticamente (come Web Service Definition Language). L'interazione con gli altri sistemi avviene mediante **messaggi SOAP** (Simple Object Access Protocol), tipicamente attraverso una **richiesta HTTP** con **serializzazione XML**.

Quindi in sostanza un **Web Service** è:

- acceduto tramite **HTTP**;
- viene definito tramite **WSDL** che definisce l'interfaccia;
- la maniera in cui facciamo interagire *consumer* e *producer* di un servizio è attraverso il protocollo **SOAP**.

Web Services Vs Servizi Accessibili via Web

I **servizi accessibili via Web** usano un'interfaccia universale (browser) ma non sono necessariamente dei *WS*. I **WS** sono utilizzabili sia automaticamente da altre applicazioni sia da utenti. I *Web Services* vengono dalla rivoluzione introdotta dal concetto di **Software as a Service**: l'idea è quella di fornire il software come servizio.

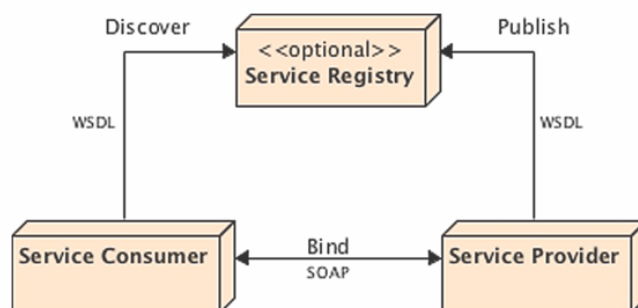
Il **Software as a service** (SaaS) è un modello di servizio del software applicativo dove un produttore di software sviluppa, opera e gestisce un'applicazione web, mettendola a disposizione dei propri clienti via Internet. Spesso si tratta di un servizio di cloud computing.

Web Services: Ruoli

Service Provider: organizzazione che fornisce i servizi.

Service Registry: fornisce un *repository di service descriptions* (WSDL) pubblicati dal *Service Provider*.

Service Consumer: fruitore dei servizi.



Le interazioni con i Web Services

Ci sono diversi modi con cui possiamo interagire con i *Web Services*:

- I *Web Services* possono essere registrati in un registro con **Universal Descriptor Discovery and Integration** (UDDI).
- Posso interagire con un *WS* una volta che il consumer viene a conoscenza dell'interfaccia del servizio e il formato del messaggio, e quindi può inviare una richiesta al provider del servizio per ricevere una risposta.

Il *consumer* e il *provider* parlano tra di loro in **SOAP**. Il servizio può essere pubblicato su un registro di servizio tramite **WSDL** e il *consumer* può scoprire questo servizio tramite **WSDL** su questo registro di servizio.

Gli Standard

Gli **standard** servono per rendere omogenee le architetture. Lo scopo dei *WS* è quello di rendere le piattaforme aperte e quindi disponibili a tutti i potenziali utenti senza vincolare lo sviluppo.

Infatti, l'idea è che se si usa uno standard indipendente dalla piattaforma si riesce a raggiungere l'indipendenza.

Tutti gli standard dei *Web Services* sono scritti in **XML** e sono: **SOAP** **WSDL** **UDDI**

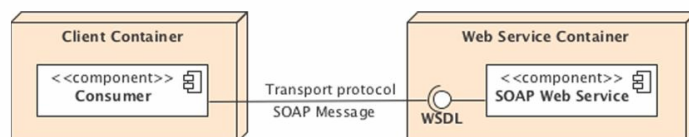
Il **protocollo di comunicazione** è **HTTP** il quale da protocollo di applicazione diventa protocollo di trasporto per il middleware. È possibile usare anche altri protocolli come *SMTP* e *JMS*.

❖ WSDL

WSDL (Web Service Definition Language) serve per definire l'interfaccia di un *Web Service*, ovvero il **contratto** tra chi fornisce il servizio e chi ne usufruisce.

WSDL permette sia a tutti i servizi di essere interoperabili ma anche di fornire una serie di informazioni:

- tipo di messaggio;
- porta;
- protocollo di comunicazione;
- operazioni supportate;
- posizione;
- il valore di ritorno del servizio che si attende.



All'interno di *Java Enterprise* ci sono dei container che facilitano la vita tra chi usa il servizio e chi lo eroga. Nel **Web Service Container** abbiamo quindi un messaggio *SOAP* e una definizione dell'interfaccia in **WSDL**. **WSDL** e **SOAP** permettono la totale indipendenza.

❖ SOAP

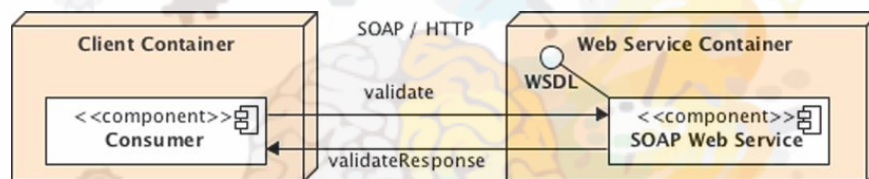
SOAP (Simple Object Access Protocol) è il protocollo usato per scambiare messaggi. È fortemente basato su XML nel senso che i messaggi scambiati sono strutturati in XML in diversi elementi.

In particolare, un **messaggio SOAP** è diviso in 4 parti:

1. **SOAP envelope**: descrive cosa c'è nel messaggio e come processarlo, inoltre, identifica il documento XML come messaggio SOAP;
2. **SOAP header** (opzionale): contiene informazioni aggiuntive per processare il messaggio.
3. **SOAP body**: è il contenuto vero e proprio del messaggio e contiene le chiamate e le risposte.
4. **SOAP fault**: contiene informazioni su eventuali errori verificatosi durante il processing.

SOAP usa il protocollo HTTP come protocollo di scambio:

- la richiesta HTTP corrisponde all'invio della richiesta del servizio;
- la risposta HTTP corrisponde al risultato della invocazione.



❖ UDDI

UDDI è il servizio di localizzazione dei servizi. Il **provider** ha un servizio e vuole renderlo disponibile; il **consumer** si rivolge ad un servizio di naming, ovvero un servizio **UDDI** per scoprire questi servizi. Un servizio può essere scoperto in questa maniera in modo da scaricare il **WSDL** e usare il servizio.

UDDI è opzionale quando si conosce già la locazione del servizio. **UDDI** è basato su XML come tutti gli altri protocolli:

- Viene interrogato da messaggi SOAP e fornisce collegamenti a documenti WSDL;
- Contiene informazioni aggiuntive sul servizio (come disponibilità, costo, etc.);
- Attualmente è poco utilizzato ed in disuso: WS un po' meno loose coupled.

Web Service: implementazione di riferimento in Java

L'**implementazione di riferimento** per il mondo Java è **Metro** (Open Source) che permette di consumare ed implementare WS in Java. Con queste implementazioni possiamo mascherare completamente SOAP dagli scopi del programmatore.

Un WS si può sviluppare in 2 maniere complementari: possiamo partire dall'alto (specifiche) verso il basso (codice) oppure in direzione opposta.

1. approccio **top-down**, chiamato **contract first**: si parte dal contratto in WSDL, definendo operazioni, messaggi e poi arrivo al codice. Consiste dunque nella generazione automatica delle classi da WSDL fornite da Metro (`wsimport`);
2. approccio **bottom-up**, chiamato **program first**: scrivo prima il codice e poi le specifiche. Corrisponde alla costruzione di un'interfaccia WSDL di quello che fa questo il codice Java. Possiamo farlo con `wsgen` di Metro, il quale genera le interfacce WSDL.

In entrambi i casi il compito del programmatore è facilitato da JAX-WS, che permette di annotare un POJO e farlo "diventare" Web Service.

Anatomia di un SOAP WS

La classe per essere un **SOAP WS** deve:

- essere annotata con `@WebService`;
- deve implementare zero o più interfacce annotate con `@WebService`;
- avere un costruttore pubblico di default;
- non avere un `finalize()`;
- essere pubblica (non final o astratta);
- essere annotata anche con `@Stateless` o `@Singleton` se si vuole renderla anche un *EJB endpoint*.

Ovviamente avere un *EJB* ha il vantaggio di offrire servizi di transazioni e sicurezza in maniera automatica. I *Web Service* come piccolo svantaggio non sono molto efficienti in quanto per ogni richiesta e risposta di un *Web Service* abbiamo una componente di codifica e decodifica di *XML*. All'aumentare della qualità del servizio offerto la qualità decade un po'.

Web Services in Java

I *Web Services* sono definiti tramite degli standard che possono risultare tedious da configurare. Per questo motivo esistono dei framework per semplificarci la vita, come **JAX-WS** e **JAXB**.

Anche in questo caso per definire i *Web Services* possiamo percorrere due strade, tramite due tipi di annotazioni:

1. **mapping WSDL**: ci consente di scrivere codice in Java che poi viene mappato in *XML* (mapping WSDL/Java). Nel package `javax.jws` ci sono le annotazioni per personalizzare i metodi esposti: `@WebMethod`, `@WebResult`, `@WebParam` e `@OneWay`.
2. **invio di messaggi SOAP**, ovvero scriviamo qualcosa che viene riportato sul messaggio che viaggia in *SOAP*. Le annotazioni appartengono al package `javax.jws.soap`.

13.3 SOAP Binding

Un **SOAP binding** deve scegliere fra due differenti formati di serializzazione/deserializzazione:

- **Literal**: dati serializzati secondo uno schema *XML*;
- **Encoded**: il *SOAP encoding* specifica come oggetti, strutture, array devono essere serializzati.

13.4 Eccezioni e Fault

In Java, quando si verifica un errore, si lanciano le eccezioni e qualche classe dovrà gestirle.

Con *SOAP* non si può procedere nello stesso modo perché *consumer* e *service* potrebbero essere scritti in linguaggi diversi e separati dalla rete. L'idea è di usare **SOAP fault** in un **SOAP message**.

JAX-WS runtime converte automaticamente le eccezioni Java in **SOAP fault messages**, restituiti al client. In questo modo il programmatore ha meno lavoro da svolgere.

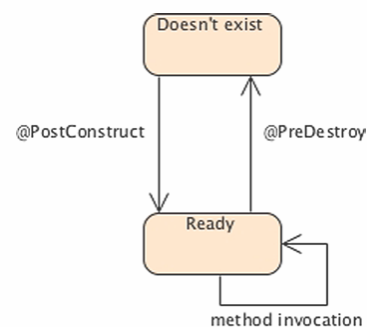
13.5 Ciclo di vita di un WS

Il **ciclo di vita** di un *Web Services* è molto simile a quello dei **bean stateless**. Un Servizio esiste grazie alla transizione verso lo stato di *Ready*.

I metodi con `PostConstruct` vengono invocati dopo che il WS viene istanziato.

I metodi con `PreDestroy` vengono invocati prima di chiuderlo.

A differenza degli *EJB* non ci sono *Interceptor*. Non possiamo mettere un *Web Service* davanti un altro *Web Service*.



13.6 Contesto di un WS

Un **SOAP web service** ha un *environment context* e può accedervi iniettando un riferimento di `javax.xml.ws.WebServiceContext` con l'annotazione `@Resource`. In questo modo può avere informazioni runtime.

13.7 Deployment Descriptor

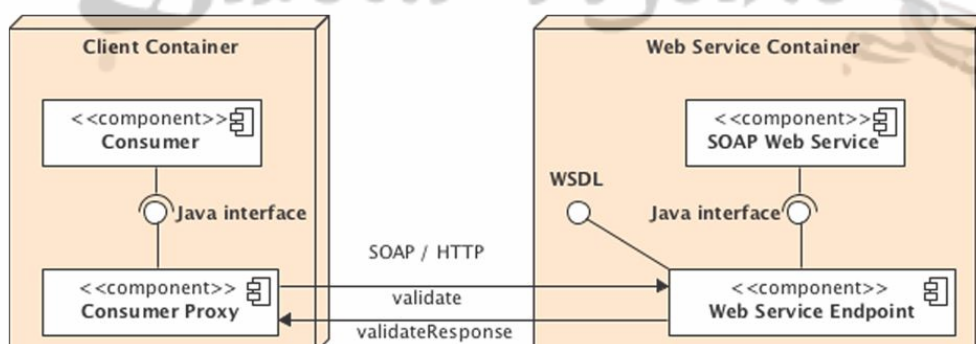
SOAP web service permette di definire i metadati usando le annotazioni oppure via XML.

Il **deployment descriptor** (localizzato sotto la directory `WEB-INF`) si chiama `webservices.xml` ed è opzionale.

13.8 Invocazione di un Web Service

Usando il *WSDL* e alcuni *stub* (proxy) generati automaticamente dal sistema, si possono usare i *WS* anche senza usare *HTTP* e *SOAP*. Il meccanismo è molto simile a *RMI* ma la differenza rispetto ad esso è che il server può essere scritto in un linguaggio diverso da Java.

Per fare ciò, usiamo **Metro** che fornisce il tool *WSDL-to-Java* (`wsimport`) per generare interfacce e classi il Java partendo da un file *WSDL* e usando i proxy si possono convertire i parametri di un messaggio *SOAP* e inviarlo all'endpoint del *WS*.



13.9 Anatomia di un SOAP Consumer

Poiché *JAX-WS* è disponibile in *Java SE*, un **SOAP web service consumer** può essere:

- un qualunque tipo di codice Java con una *classe main* in esecuzione nella *JVM*;
- una qualunque *Java EE component* in esecuzione in un container (Web, EJB o application client container).

Se eseguita in un container, il consumer può ottenere un'istanza del proxy attraverso **injection** oppure **programmaticamente** (creandola).

Per iniettare un *web service*, bisogna usare la notazione `@javax.xml.ws.WebServiceRef` o un *CDI producer*.

***High Performance
Computing
&
Cloud Computing***

Capitolo 14 – High Performance Computing

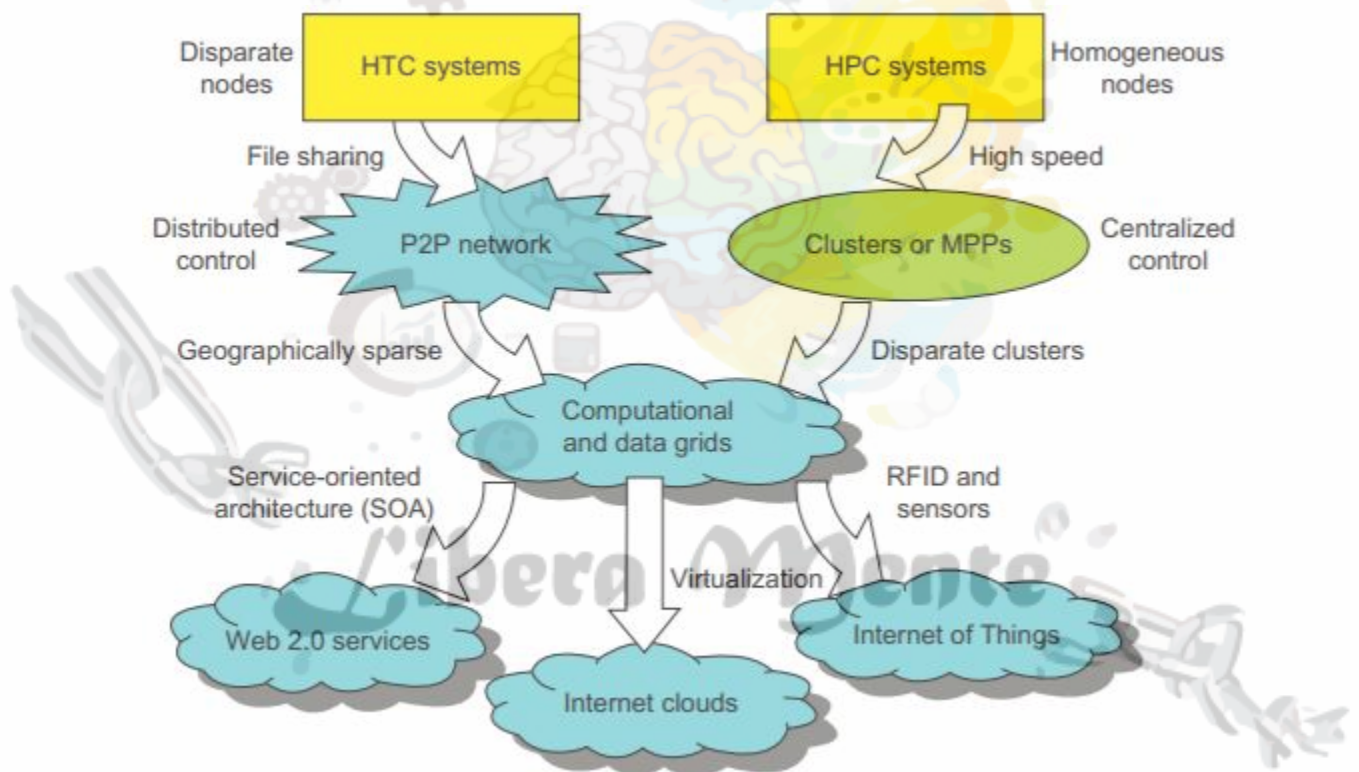
14.1 Computazioni scalabili

In questo capitolo, vengono analizzati i cambiamenti evolutivi dell'architettura della macchina, del sistema operativo, della connessione della rete e del carico di lavoro delle applicazioni.

Infatti, il percorso che le computazioni hanno seguito va dal calcolo monolitico a quello **distribuito**; siamo passati da un'architettura *monolitica* ad un'architettura di tipo **network-centric**. L'obiettivo principale è la **scalabilità**. L'app deve poter fornire buone prestazioni di calcolo al crescere dei dati.

Invece di usare un computer centralizzato per risolvere problemi computazionali, si è passati ad un **sistema di calcolo parallelo e distribuito** il quale utilizza molteplici computer per risolvere problemi su larga scala, quindi su Internet. Pertanto, il calcolo distribuito diventa **data-intensive** e **network-centric**.

Tramite i sistemi HTC e HPC arriviamo al **computational and data grids**: progenitore del cloud.



High Performance Computing

Con **High Performance Computing (HPC)** ci si riferisce alle tecnologie utilizzate da **computer cluster** (ovvero un insieme di computer connessi tramite una rete telematica) per creare dei sistemi di elaborazione con prestazioni molto elevate nell'ordine dei **PetaFLOPS**, ricorrendo tipicamente al **calcolo parallelo**. Questo tipo di calcolo è qualcosa di omogeneo, con controllo centralizzato.

Per creare un'architettura **HPC**, i server di calcolo sono collegati in rete in un **cluster**. I programmi software e gli algoritmi vengono eseguiti contemporaneamente sui server del cluster. Il cluster viene collegato in rete al sistema di storage per acquisire il risultato. Per funzionare al massimo delle performance, ogni componente deve essere sincronizzato con gli altri.

Un **cluster HPC** è costituito da centinaia o migliaia di server di calcolo collegati in rete. Ogni server è chiamato nodo. I nodi di ciascun cluster lavorano in parallelo, aumentando la velocità di elaborazione per fornire un ambiente di **HPC**.

High-Throughput Computing

Il paradigma **HTC** presta maggiore attenzione al **throughput**, cioè il numero di attività completate per unità di tempo. L'obiettivo dell'**HTC** è di eseguire un'attività computazionale usando più risorse possibili per lunghi periodi di tempo. Nell'**HTC** i nodi sono eterogenei, distribuiti e il calcolo è di tipo cooperativo, cioè tante macchine decidono di collaborare per risolvere problemi di grandi dimensioni (es p2p sharing).

Esistono molte differenze tra l'**HPC** e l'**HTC**:

- le **attività HPC** sono caratterizzate dal bisogno di grandi quantità di potenza di calcolo per brevi periodi di tempo;
- le **attività HTC** richiedono grandi quantità di elaborazione, ma per tempi molto più lunghi (mesi e anni, anziché ore e giorni). Inoltre, l'**HTC** non considera i **FLOPS** ma il numero di task completati per unità di tempo.

14.2 Cloud Computing

Il concetto di **cloud computing** non è un concetto nuovo dal punto di vista tecnologico. Già nel 1961 John McCarthy cominciò a parlare di **utility computing**, in cui immaginava un futuro in cui i computer venissero offerti sotto forma di *utility pubblica*, come ad esempio un telefono pubblico.

L'**utility computing** si concentra su un modello di business in cui i clienti ricevono risorse informatiche da un fornitore di servizi a pagamento. Tutte le **piattaforme grid/cloud** sono considerate fornitori di servizi di utilità.

Il **Cloud** è un ambiente IT ben delimitato, progettato per fornire a distanza risorse IT scalabili e misurabili. Il termine *Cloud* nasce inizialmente come metafora di internet, che veniva rappresentato graficamente come una nuvola. Ovviamente sono due concetti ben diversi tra loro:

- **Internet** permette accesso pubblico a una moltitudine di risorse IT web-based, che sono tipicamente content-based e pubblicate attraverso il world wide web.
- Un **cloud** è invece privato, offre accesso a risorse IT misurabili e non è detto che siano web-based anche se vengono fruite attraverso protocolli e tecnologie di internet.

I **vantaggi** nell'utilizzo del cloud sono i seguenti:

- **Abbassamento dei costi**: non c'è più bisogno di investimenti iniziali e successivi su HW e SW, come quelli riguardanti all'acquisto, configurazione, installazione, manutenzione e dismissione. E' possibile accedere ai servizi del cloud senza avere macchine particolari, perché programmi e dati risiedono nel cloud e sono gestiti da personale qualificato. Questo permette da un lato di adeguarsi alle esigenze dell'organizzazione in modo tempestivo e dall'altro libera il personale dell'organizzazione che in passato si occupava della gestione dell'infrastruttura ora presente nel cloud.
- **Aumento della scalabilità**: è possibile richiedere o rilasciare risorse in base alle richieste.
- **Indipendenza dall'hardware**: non si è vincolati all'utilizzo di determinati dispositivi o configurazioni.
- **Accesso ubiquo**: ormai numerosi cloud supportano l'accesso da diversi dispositivi.

Gli **svantaggi** invece sono:

- **Aumento delle vulnerabilità**: spostare i dati nel cloud, significa condividere la responsabilità della sicurezza dei dati con il cloud provider, quindi, la sicurezza di questi dipende anche dalle policy di sicurezza che il cloud adotta.

- **Posizione geografica:** i cloud possono essere dislocati in altri paesi che trattano i dati in modi diversi dai quali noi ci aspettiamo, cosa che di norma non succede per le risorse on-premise.
- **Portabilità:** a causa di mancati standard per quanto riguarda l'industria del cloud computing, è possibile avere problemi nella portabilità delle risorse tra i diversi cloud. La portabilità è di fatto il metro per determinare l'impatto che avrebbe spostare le risorse IT di un consumer in un altro cloud.

14.2.1 Modelli di distribuzione

Un **modello di distribuzione** è una specifica combinazione di risorse IT offerte da un *cloud provider*. I modelli più famosi sono i seguenti:

- ❖ **Infrastructure as a Service (IaaS):** l'utente noleggia un'infrastruttura.
- ❖ **Platform as a Service (PaaS):** l'utente noleggia un ambiente per poter sviluppare ed eseguire applicazioni su cloud.
- ❖ **Software as a Service (SaaS):** l'utente noleggia l'uso di software fornito, via web, dal cloud.

14.2.2 Modelli di deployment

Un **modello di deployment** rappresenta uno specifico ambiente cloud, caratterizzato da proprietà, dimensione e accesso. Ci sono 4 principali *modelli di deployment*:

1. **Public cloud:** è un ambiente cloud accessibile da chiunque (pubblico) e di proprietà di un *cloud provider* che si occupa della sua creazione e del mantenimento delle sue risorse, che sono organizzate secondo i modelli di distribuzione visti in precedenza.
2. **Community cloud:** è simile ad una *public cloud*, con la differenza che l'accesso è limitato ad uno specifico gruppo (community) di *cloud consumer*. Il cloud può essere di proprietà del gruppo che vi accede oppure di proprietà di un terzo *cloud provider* che ne regola l'accesso.
3. **Private cloud:** è di proprietà di una singola organizzazione che utilizza il cloud per centralizzare l'accesso a risorse IT da parte di differenti dipartimenti, sedi o edifici.
4. **Hybrid cloud:** è un cloud il cui modello di deployment è costituito da due o più dei modelli visti in precedenza. Ad esempio, è possibile gestire i dati sensibili attraverso una private cloud e quelli meno critici con un public cloud.

Capitolo 15 – Microservizi e Serverless Computing

15.1 La morte del “Big Software”

I progetti di grandi dimensioni (che usano waterfall) falliscono più facilmente sia per ragioni di tipo umano che di tipo tecnico.

Il **big software** è morto perché non è più prodotto all'interno di grandi aziende le quali, solitamente, si appoggiano a prodotti standard e i software che sviluppano hanno un'**architettura monolitica**. Il *cloud* è una buona soluzione per ridurre i costi di gestione.

Dalle “ceneri” del big software, rinasce il **software “small”** in cui diversi servizi sono sviluppati e mantenuti in maniera indipendente. La comunicazione avviene tramite protocolli leggeri e indipendenti dalla piattaforma e l'infrastruttura non è centralizzata.

Di conseguenza vengono utilizzati i **microservizi** che hanno diversi vantaggi:

- Deployment continuo (CD/CI);
- Le sostituzioni e le modifiche sono ora possibili senza influenzare il resto;
- Possibile il delivery su container (Docker);
- Gestione dell'applicazione in mano al cliente.

15.2 Microservizi

I **microservizi** non sono servizi piccoli bensì uno stile di sviluppo di un'applicazione come insieme di piccoli servizi, ciascuno in esecuzione su un proprio processo che collaborano attraverso protocolli leggeri come **HTTP**. Può esserci un minimo di gestione centralizzata ma possono essere realizzati in tecnologie differenti.

I **microservices** permettono di gestire e scalare in maniera indipendente i servizi. L'idea è che al posto di scalare l'intero server vengono replicati solo dei singoli servizi. All'inizio si paga in termini di progettazione di *microservices*. Man mano che aumenta la complessità la progettazione diventa più produttiva rispetto a quella *monolitica*.

Caratteristiche dei microservizi

Componenti come servizi

- sono indipendentemente deployabili;
- la composizione aiuta ad eliminare la coesione;
- esplicitazione delle interfacce dei servizi

Sono presenti degli svantaggi: bisogna scegliere la “grana” delle invocazioni.

Organizzazione attorno alle competenze di business

Secondo la **legge di Conway**, un'organizzazione che progetta un sistema, produrrà un progetto la cui struttura è una copia della struttura di comunicazione dell'organizzazione (es. se ho dei team partizionati, la struttura del progetto “copia” la struttura organizzativa).

Con i *microservizi* i team sono **cross-funzionali** dove in ogni team ci sono persone che si occupano di competenze diverse. Ogni team ha al suo interno tutte le skill necessarie per affrontare un piccolo progetto e quindi avere una visione globale del sistema.

Prodotti, non progetti

“You build it, you run it”

Un **microservice** è una funzionalità di business. Il team ha la responsabilità completa del software, anche in produzione. Bisogna realizzare un prodotto completo e funzionante e non un singolo progetto. Questo approccio è possibile anche con l'architettura monolitica, ma è molto più semplice con i *microservizi*. La responsabilizzazione del team è fondamentale in questo caso.

Disaccoppiamento e coesione

I *microservices* devono essere intelligenti ma devono agire da filtri. I servizi possono avere una coreografia per costruire servizi più complessi. Posso usare HTTP ma anche protocolli di comunicazione basati su messaggi.

Gestione decentralizzata del calcolo e dei dati

Ogni team è autorizzato a scegliere la tecnologia che preferisce. Diventa importante definire delle interfacce. Con i *microservices* i database sono distribuiti e specifici per ogni servizio.

Automatizzazione del deployment

Il continuous deployment è fondamentale per ogni microservizio. Vengono sfruttate tutte le tecniche di testing automatico per consentire un deploy automatizzato.

Progettati per i malfunzionamenti

Ogni team ha chiaro in mente che altri *microservizi* possano non funzionare e devono prevedere il non funzionamento del *microservizio* che si chiama. Bisogna progettare le chiamate sapendo che possono non andare a buon fine.

Per ovviare a questo problema vengono usati anche dei servizi di real time monitoring dei servizi con azioni automatiche di restore del servizio.

Progettazione evolutiva

Il servizio deve essere sostituibile. Se dovendo cambiare un servizio *A* bisogna cambiare anche *B* vuol dire che c'è stato un errore di progettazione (probabilmente i due servizi dovevano essere un unico servizio). Il **release planning** è effettuato per *microservice* invece che per applicazione. Bisogna gestire correttamente il **versioning** dei servizi gestendo correttamente le interfacce.

Quando usare un'architettura monolitica?

L'**architettura monolitica** è importante nel caso in cui è necessaria l'efficienza. Ad esempio, sistemi di comunicazione real time. Quando si parla di software di grandi dimensioni l'architettura a *microservizi* è privilegiata.

15.3 Serverless Computing

Il **serverless** è l'interpretazione in cloud dei *microservizi* "Function as a Service". Il *cloud provider* fornisce una gestione completa del container su cui la funzione viene eseguita. Esistono diversi tipi di eventi da trattare:

- Richieste API;
- Oggetti inseriti/ricercati nello storage;
- Cambi al DB;
- Schedule;
- Comandi vocali, bots etc

In un **microservizio serverless** ogni processo è indipendente ed **event based**. Ciò rende l'infrastruttura molto scalabile. Il costo è bassissimo rispetto all'infrastruttura classica. Ovviamente ci sono dei limiti anche nel serverless:

- Non va bene quando bisogna mantenere il controllo dell'infrastruttura;
- Non è utile per procedure batch;
- *Vendor lock-in*: chi sviluppa soluzioni diventa "ostaggio" da parte di chi mette a disposizione la tecnologia.
- *Cold start*: le funzioni quando non vengono usate per molto tempo vengono passivate. Si potrebbe incontrare un ritardo dopo tanto che la funzione non viene fatta partire. Si può fare un ping schedato alla funzione.
- Infrastruttura condivisa.

Libera Mente